SF2957 STATISTICAL MACHINE LEARNING (7.5 HP)

LECTURE NOTES

Lecture 1

Literature:

- Casella and Berger, 2002, Chapter 7.

In this lecture we have a look at the mathematical foundation of statistics. Throughout the course one can work with discrete or continuous densities. We will mostly use the notation of continuous densities, but these can just as well be replaced by discrete ones and integrals replaced by sums.

1. Classical and Bayesian statistics

Usually one starts with a number of observed data, $X = (X_1, \dots, X_n)$, where X_i are random variables with values in $\mathbb{R}$. The distribution of X is unknown but we assume it has a density that depends on an unknown parameter. We write Θ for the unknown parameter and think of it as a random variable representing the uncertainty of its value and assume that it takes values in the parameter space $\mathit{\Theta}$. One may assume that there is a single "true value" θ of the parameter but this value is unknown.

We write $p_{X|\Theta}(\cdot \mid \theta)$ for the conditional probability density of X given $\Theta = \theta$. The density $p_{X|\Theta}(\cdot \mid \theta)$ is the basis of classical statistics. If one observes $X = x$, then the function $\theta \mapsto p_{X|\Theta}(x \mid \theta)$ is called the *likelihood function* and is used for making inferences.

Example 1 (Independent observations)**.** Suppose we have n observations of independent random variables $X_1, \dots, X_n$, each with density $p_{X_i|\Theta}(x_i \mid \theta)$ when $\Theta = \theta$. In this case the vector $X = (X_1, \dots, X_n)$ has density

$$p_{X|\Theta}(x \mid \theta) = p_{X|\Theta}(x_1, \dots, x_n \mid \theta) = \prod_{i=1}^{n} p_{X_i|\Theta}(x_i \mid \theta).$$

After observing $(X_1, \dots, X_n) = (x_1, \dots, x_n)$ the likelihood function $L(\theta)$ is the function $\theta \mapsto \prod_{i=1}^{n} p_{X_i|\Theta}(x_i \mid \theta)$.

Example 2 (Independent and identically distributed observations)**.** If we in addition suppose that the independent random variables are identically distributed, then $p_{X_i|\Theta}(x_i \mid \theta) = p_{X_1|\theta}(x_i \mid \theta)$, for each i, and the vector $X = (X_1, \dots, X_n)$ has density

$$p_{X|\theta}(x \mid \theta) = \prod_{i=1}^{n} p_{X_1|\theta}(x_i \mid \theta).$$

1.1. **General introduction.** Let us now take a somewhat more formal look at the setting, which provides a general framework where we can study the elements of statistics.

To start let us take an underlying probability space $(\Omega, \mathcal{F}, \mathrm{P})$. Ω is the abstract space of outcomes, $\mathcal{F}$ is a σ-field, and P a probability measure. Suppose we do an experiment where the collected data takes values in the sample space $\mathcal{X}$ which has a σ-field $\mathcal{A}$. This space is denoted $(\mathcal{X}, \mathcal{A})$. The observed data is denoted by X, where $X : \Omega \to \mathcal{X}$ is a random variable (i.e. it is a measurable mapping, which means $X^{-1}(A) = \{\omega \in \Omega : X(\omega) \in A\} \in \mathcal{F}$ for every $A \in \mathcal{A}$). We will use the term *random variable* in a general sense. That is, $\mathcal{X}$ could be a general space. Often $\mathcal{X}$ will be some familiar Euclidean space, for instance, X could be a vector of random variables $X = (X_1, \ldots, X_n)$, in which case $\mathcal{X} = \mathbb{R}^n$, but it may also be a continuous stochastic process, in which case $\mathcal{X}$ is the space $C[0,1]$ of continuous functions. The distribution of X (which is a probability measure on $\mathcal{A}$) is assumed to belong to a parametric family of probability measures $\mathcal{P}_0$ on $\mathcal{A}$. The probability measures in the family $\mathcal{P}_0$ are indexed by a parameter θ taking values in the parameter space Θ with σ-field $\mathcal{B}$. It is assumed that the parametric family $\mathcal{P}_0$ can be written as $\mathcal{P}_0 = \{p_{X|\Theta}(\cdot \mid \theta) : \theta \in \Theta\}$.

Note that we have introduced $\mathcal{P}_0$ to be a family of conditional distributions. An alternative is to represent the uncertainty of the parameter by viewing Θ as a random variable, i.e. as a measurable mapping from Ω to Θ. The joint distribution of (X, Θ) is then a probability $p_{X,\Theta}(x, \theta)$. Then one can define the conditional distribution of X given $\Theta = \theta$, also called the likelihood, as

$$p_{X|\Theta}(x \mid \theta) = \frac{p_{X,\Theta}(x, \theta)}{p_\Theta(\theta)},$$

where

$$p_\Theta(\theta) = \int p_{X,\Theta}(x, \theta) dx,$$

is the marginal distribution of Θ. We call p_Θ the prior distribution on Θ.

It should be noted that in the classical setup it is sufficient to start directly with the family $\mathcal{P}_0$ without first defining Θ as a random variable. For instance, the classical paradigm never uses the joint distribution of (X, Θ) or the marginal distribution of X or Θ. However, to fit the classical and the Bayesian into the same framework we will think about Θ as a random variable and $p_{X|\Theta}(x \mid \theta)$ as the conditional distribution of X given $\Theta = \theta$.

2. Bayesian statistics

In the Bayesian paradigm it is assumed that Θ is a random variable and some prior knowledge of the parameter Θ is available. The information about Θ is put into the model by specifying the *prior distribution* with density p_Θ. The likelihood $p_{X|\Theta}$ and prior p_Θ can be combined to obtain the joint density of (X, Θ) given by

$$p_{X,\Theta}(x, \theta) = p_{X|\Theta}(x \mid \theta) p_\Theta(\theta).$$

Once the joint density is specified we can also derive the marginal density of X

$$p_X(x) = \int p_{X|\Theta}(x \mid \theta) p_\Theta(\theta) d\theta.$$

An important ingredient in the Bayesian paradigm is the *posterior distribution* given the observation $X = x$. Its density is given by Bayes' theorem as

$$p_{\Theta|X}(\theta \mid x) = \frac{p_{X,\Theta}(x,\theta)}{p_X(x)} = \frac{p_{X|\Theta}(x \mid \theta)p_\Theta(\theta)}{\int p_{X|\Theta}(x \mid \vartheta)p_\Theta(\vartheta)d\vartheta}.$$

The posterior distribution of Θ given $X = x$ can be thought of as the updated beliefs about Θ after taking into account the observation $X = x$.

In Bayesian statistics all inference is based on the posterior distribution. Note that the difference from classical statistics is that the posterior density is just the likelihood function multiplied by the prior density and then normalized to become a probability distribution.

2.1. **Posterior predictive distribution of future values.** Suppose that $X_1, X_2, \ldots$ are random variables and we have observed the values $X_1 = x_1, \ldots, X_n = x_n$. To compute the probability of future events, the Bayesian methodology proposes to use

$$\begin{aligned}
&p_{X_{n+1},\ldots,X_{n+k}|X_1,\ldots,X_n}(x_{n+1},\ldots,x_{n+k} \mid x_1,\ldots,x_n) \\
&\quad= \frac{p_{X_1,\ldots,X_{n+k}}(x_1,\ldots,x_{n+k})}{p_{X_1,\ldots,X_n}(x_1,\ldots,x_n)} \\
&\quad= \int \frac{p_{X_1,\ldots,X_{n+k},\Theta}(x_1,\ldots,x_{n+k},\theta)}{p_{X_1,\ldots,X_n}(x_1,\ldots,x_n)}d\theta \\
&\quad= \int p_{X_{n+1},\ldots,X_{n+k}|X_1,\ldots,X_n,\Theta}(x_{n+1},\ldots,x_{n+k} \mid x_1,\ldots,x_n,\theta)\frac{p_{X_1,\ldots,X_n,\Theta}(x_1,\ldots,x_n,\theta)}{p_{X_1,\ldots,X_n}(x_1,\ldots,x_n)}d\theta \\
&\quad= \int p_{X_{n+1},\ldots,X_{n+k}|X_1,\ldots,X_n,\Theta}(x_{n+1},\ldots,x_{n+k} \mid x_1,\ldots,x_n,\theta)p_{\Theta|X_1,\ldots,X_n}(\theta \mid x_1,\ldots,x_n)d\theta.
\end{aligned}$$

This distribution is called the *posterior predictive distribution of future values.*

Example 3. If the X_i's are assumed conditionally IID given $\Theta = \theta$ then the posterior predictive distribution of future values is given by

$$\begin{aligned}
&p_{X_{n+1},\ldots,X_{n+k}|X_1,\ldots,X_n}(x_{n+1},\ldots,x_{n+k} \mid x_1,\ldots,x_n) \\
&\quad= \int \prod_{i=1}^{k} p_{X_{n+i}}(x_{n+i} \mid \theta)p_{\Theta|X_1,\ldots,X_n}(\theta \mid x_1,\ldots,x_n)d\theta.
\end{aligned}$$

2.2. **Conjugate prior distributions.** In the Bayesian setting it is important to compute posterior distributions. This is not always an easy task. The main difficulty is to compute the normalising constant $p_X(x)$ in the denominator of Bayes' theorem. However, for certain parametric families $\mathcal{P}_0 = \{p_{X|\Theta}(x \mid \theta) : \theta \in \Theta\}$ there are convenient choices of prior distributions. Particularly convenient is when the posterior belongs to the same family of distributions as the prior. Such families are called conjugate families.

Definition 1. Let $\mathcal{P}_0$ denote a class of probability densities $p_{X|\Theta}(x \mid \theta)$. A class Π of prior distributions is a *conjugate family* for $\mathcal{P}_0$ if the posterior distribution is in the class Π for all priors in Π, and all $x \in \mathcal{X}$.

Example 4 (Casella & Berger, Example 7.2.14)**.** Let $X_1, \ldots, X_n$ be IID Ber(θ) given $\Theta = \theta$ and put $Y = \sum_{i=1}^n X_i$. Then $Y \sim \text{Bin}(n, \theta)$. Let the prior distribution be Beta(α, β), where α, β are called hyperparameters. Then the posterior of Θ given $Y = y$ is Beta($y + \alpha, n - y + \beta$).

The joint density is

$$\begin{aligned} p_{Y,\Theta}(y,\theta) &= p_{Y|\Theta}(y \mid \theta) p_\Theta(\theta) \\ &= \binom{n}{y} \theta^y (1-\theta)^{n-y} \frac{\Gamma(\alpha+\beta)}{\Gamma(\alpha)\Gamma(\beta)} \theta^{\alpha-1}(1-\theta)^{\beta-1} \\ &= \binom{n}{y} \frac{\Gamma(\alpha+\beta)}{\Gamma(\alpha)\Gamma(\beta)} \theta^{y+\alpha-1}(1-\theta)^{n-y+\beta-1}. \end{aligned}$$

The marginal density of Y is

$$\begin{aligned} p_Y(y) &= \int_0^1 p_{Y,\Theta}(y,\theta) d\theta \\ &= \binom{n}{y} \frac{\Gamma(\alpha+\beta)}{\Gamma(\alpha)\Gamma(\beta)} \int_0^1 \theta^{y+\alpha-1}(1-\theta)^{n-y+\beta-1} d\theta \\ &= \binom{n}{y} \frac{\Gamma(\alpha+\beta)}{\Gamma(\alpha)\Gamma(\beta)} \frac{\Gamma(y+\alpha)\Gamma(n-y+\beta)}{\Gamma(n+\alpha+\beta)} \end{aligned}$$

(this distribution is known as the beta-binomial distribution). The posterior is then computed as

$$p_{\Theta|Y}(\theta \mid y) = \frac{p_{Y,\Theta}(y,\theta)}{p_\Theta(y)} = \frac{\Gamma(n+\alpha+\beta)}{\Gamma(y+\alpha)\Gamma(n-y+\beta)} \theta^{y+\alpha-1}(1-\theta)^{n-y+\beta-1}.$$

This is the density of the Beta$(y+\alpha, n-y+\beta)$ distribution.

For more examples and exercises, see Exercise 7.22, 7.23, 7.24 in Casella and Berger.

Exercise 1. Show that the following distributions are conjugate priors for the corresponding densities.

(a) The Poisson distribution $p_{X|\Lambda}(k \mid \lambda) = \frac{\lambda^k}{k!} e^{-\lambda}$ has the $\Gamma(\alpha,\beta)$ distribution, $p_\Lambda(\lambda) = \frac{\beta^\alpha}{\Gamma(\alpha)} \lambda^{\alpha-1} e^{-\beta\lambda}$, $\lambda > 0$, as its is conjugate prior. The parameters α, β are hyperparameters.

(b) The multinomial distribution with k categories and

$$p_{X|\theta_1,\dots,\theta_k}(x_1,\dots,x_k) = \frac{n!}{x_1!\cdots x_k!} \prod_{i=1}^k \theta_i^{x_i},$$

where $x_i \geq 0$ are integers and $\sum_{i=1}^k x_i = n$, has the Dirichlet distribution,

$$p_{\Theta_1,\dots,\Theta_k}(\theta_1,\dots,\theta_k) = \frac{\prod_{i=1}^k \Gamma(\alpha_i)}{\Gamma(\sum_{i=1}^k \alpha_i)} \prod_{i=1}^k \theta_i^{\alpha_i - 1},$$

as its conjugate prior. The parameters $\alpha_1,\dots,\alpha_k$ are hyperparameters.

(c) The normal distribution with mean parameter θ and known variance σ^2,

$$p_{X|\Theta}(x \mid \theta) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left\{ -\frac{1}{2\sigma^2}(x-\theta)^2 \right\},$$

has a normal distribution $N(\psi, \tau^2)$,

$$p_\Theta(\theta) = \frac{1}{\sqrt{2\pi\tau^2}} \exp\left\{ -\frac{1}{2\tau^2}(\theta-\psi)^2 \right\},$$

as its conjugate prior for the mean. The parameters ψ, τ^2 are hyperparameters.

(d) The normal distribution with known mean parameter μ and variance θ,

$$p_{X|\Theta}(x \mid \theta) = \frac{1}{\sqrt{2\pi\theta}} \exp\left\{-\frac{1}{2\theta}(x-\mu)^2\right\},$$

has an inverse gamma distribution

$$p_{\Theta}(\theta) = \frac{\beta^{\alpha}}{\Gamma(\alpha)}\theta^{-\alpha-1}e^{-\beta/\theta}$$

as its conjugate prior for the variance. The parameters α, β are hyperparameters.

Lecture 2

Literature:

- Casella and Berger, 2002, Chapter 6.

In this lecture we introduce exponential families. We also introduce the notion of sufficient statistics and explain it use in statistical inference. In addition, we give a brief introduction to statistical decision theory, from a classical and Bayesian perspective.

3. Exponential families

Exponential families of distributions are perhaps the most widely used family of distributions in statistics. It contains most of the common distributions that we know from undergraduate statistics.

Definition 2. A parametric family of distributions $\mathcal{P}_0 = \{p_{X|\Theta}(x \mid \theta) : \theta \in \Theta\}$ is called an exponential family if

$$p_{X|\Theta}(x \mid \theta) = c(\theta)h(x)\exp\Big\{\sum_{i=1}^{k}\pi_i(\theta)t_i(x)\Big\}$$

for some measurable functions $c, h, \pi_1, \ldots, \pi_k, t_1, \ldots, t_k$, and some integer k.

Example 5. If X are conditionally IID $\text{Exp}(\theta)$ given $\Theta = \theta$ then it follows that $p_{X|\Theta}(x \mid \theta) = \theta^n \exp\{-\theta\sum_{i=1}^n x_i\}$ so this is a one-dimensional exponential family with $c(\theta) = \theta^n$, $h(x) = 1$, $\pi(\theta) = -\theta$, and $t(x) = x_1 + \cdots + x_n$.

Example 6. If X are conditionally IID $\text{Ber}(\theta)$ given $\Theta = \theta$, then with $m = x_1 + \cdots + x_n$, we have

$$p(x \mid \theta) = \theta^m(1-\theta)^{n-m} = (1-\theta)^n\Big(\frac{\theta}{1-\theta}\Big)^m = (1-\theta)^n\exp\Big\{\log\Big(\frac{\theta}{1-\theta}\Big)m\Big\}.$$

So, this is also a one-dimensional exponential family with $c(\theta) = (1-\theta)^n$, $h(x) = 1$, $\pi(\theta) = \log(\theta/(1-\theta))$, and $t(x) = x_1 + \cdots + x_n$.

There are many other examples, like the normal, Poisson, gamma, and beta distributions (see Casella & Berger, Exercise 3.28, p. 132).

Note that the function $c(\theta)$ can be thought of as a normalizing function to make $p_{X|\Theta}$ a probability density. It is necessary that

$$c(\theta) = \Big(\int_{\mathcal{X}} h(x)\exp\Big\{\sum_{i=1}^{k}\pi_i(\theta)t_i(x)\Big\}dx\Big)^{-1}$$

so the dependence on θ comes through the vector $(\pi_1(\theta), \ldots, \pi_k(\theta))$ only. It is useful to have a name for this vector; it will be called the *natural parameter*.

Definition 3. For an exponential family the vector $\Pi = (\pi_1(\Theta), \ldots, \pi_k(\Theta))$ is called the *natural parameter* and

$$\Gamma = \Big\{\pi \in \mathbb{R}^k : \int_{\mathcal{X}} h(x)\exp\Big\{\sum_{i=1}^{k}\pi_i t_i(x)\Big\}dx < \infty\Big\}$$

the *natural parameter space*.

When we deal with an exponential family it is convenient to use the notation $\Theta = (\Theta_1, \dots, \Theta_k)$ for the natural parameter and $\varTheta$ for the parameter space. Therefore we will often write

$$p_{X|\Theta}(x \mid \theta) = c(\theta)h(x) \exp\Big\{ \sum_{i=1}^{n} \theta_i t_i(x) \Big\} \tag{1}$$

and $\varTheta$ for the natural parameter space Γ and hope that this does not cause confusion.

For an example on how to write the normal distribution as an exponential family with its natural parametrisation see Examples 3.4.4 and 3.4.6, pp. 112-113 in Casella & Berger.

3.1. **Conjugate priors for exponential families.** Let us take a look at conjugate priors for exponential families. Suppose that the conditional distribution of $X = (X_1, \dots, X_n)$ given $\Theta = \theta$ forms a natural exponential family (1). We will look for a "natural" family of priors that serves as conjugate priors. If $p(\theta)$ is a prior then the posterior has density

$$\begin{aligned} p_{\Theta|X}(\theta \mid x) &= \frac{p(x \mid \theta)p(\theta)}{\int p(x \mid \vartheta)p(\vartheta)d\vartheta} \\ &= \frac{c(\theta)e^{\sum_{i=1}^{k} \theta_i t_i(x)} p(\theta)}{\int c(\vartheta)e^{\sum_{i=1}^{k} \vartheta_i t_i(x)} p(\vartheta)d\vartheta}. \end{aligned}$$

Then a natural choice for the conjugate family is densities of the form

$$p_{\Theta}(\theta) = \frac{c(\theta)^{\alpha} e^{\sum_{i=1}^{k} \theta_i \beta_i}}{\int c(\vartheta)^{\alpha} e^{\sum_{i=1}^{k} \vartheta_i \beta_i} d\vartheta},$$

where $\alpha > 0$ and $\beta = (\beta_1, \dots, \beta_k)$.

Indeed, the posterior is then proportional to

$$c(\theta)^{\alpha+1} \exp\Big\{ \sum_{i=1}^{k} \theta_i (t_i(x) + \beta_i) \Big\}$$

which is of the same form as the prior (after putting in the right normalising constant). Note that the posterior is an exponential family with natural parameter $\xi = t + \beta$ and representation

$$c'(\xi)h'(\theta) \exp\Big\{ \sum_{i=1}^{k} \xi_i \theta_i \Big\},$$

where $h'(\theta) = c(\theta)^{\alpha+1}$ and $c'(\xi)$ is the normalising constant to make it a probability density.

Example 7. Take another look at the family of n IID Ber(ϕ) variables (see Example 6). The natural parameter is $\theta = \log(\phi/1-\phi)$ and then $c(\theta) = (1-\phi)^n = (1+e^{\theta})^{-n}$. Then the proposed conjugate prior is proportional to

$$c(\theta)^{\alpha} e^{\theta\beta} = (1-\phi)^{\alpha n} \phi^{\beta} (1-\phi)^{-\beta} = \phi^{\beta} (1-\phi)^{\alpha n - \beta}$$

which is a Beta($\beta + 1, \alpha n - \beta + 1$) distribution (when you put in the normalisation). So again we see that beta-distributions are conjugate priors for IID Bernoulli random variables (here α and β are not the same as in Example 4, though).

3.2. **Some properties of exponential families.** The random vector $T(X) = (t_1(X), \dots, t_k(X))$ is of great importance for exponential families. In some cases it is possible to compute the distribution of T explicitly.

Example 8. Consider the exponential family of n IID $\text{Exp}(\theta)$ random variables as in Example 5. Then $t(x) = x_1 + \cdots + x_n$ and $T = t(X)$ has $\Gamma(n, \theta)$ distribution. Thus, T has density

$$p(t \mid \theta) = \frac{\theta^n}{\Gamma(n)} t^{n-1} e^{-\theta t}.$$

In general, we can compute the moment generating function of T, $M_T(\xi) = \text{E}[e^{\xi^{\text{T}} T}]$.

Proposition 1. *The moment generating function $M_T(\xi)$ of T for an exponential family is given by*

$$M_T(\xi) = M_T(\xi_1, \dots, \xi_k) = \frac{c(\theta)}{c(\xi + \theta)}.$$

Proof. Since

$$c(\theta) = \Big(\int_{\mathcal{X}} h(x) \exp\Big\{ \sum_{i=1}^k \theta_i t_i(x) \Big\} dx \Big)^{-1},$$

it follows that

$$\begin{aligned}
M_T(\xi) &= \text{E}\Big[\exp\Big\{ \sum_{i=1}^k \xi_i T_i \Big\} \Big] \\
&= \int \exp\Big\{ \sum_{i=1}^k \xi_i t_i(x) \Big\} p(x \mid \theta) dx \\
&= \int \exp\Big\{ \sum_{i=1}^k \xi_i t_i(x) \Big\} c(\theta) h(x) \exp\Big\{ \sum_{i=1}^k \theta_i t_i(x) \Big\} dx \\
&= c(\theta) \int h(x) \exp\Big\{ \sum_{i=1}^k (\xi_i + \theta_i) t_i(x) \Big\} dx \\
&= \frac{c(\theta)}{c(\xi + \theta)}.
\end{aligned}$$

□

Whenever θ is in the interior of the parameter space all moments of T are finite and can be computed. We call the function

$$\kappa(\theta) = -\log c(\theta)$$

the *cumulant function*. The cumulant function is useful to compute moments of $T(X)$.

Proposition 2. *For an exponential family with cumulant function κ we have*

$$\begin{aligned}
\text{E}_\theta[T_i] &= \frac{\partial}{\partial \theta_i} \kappa(\theta), \\
\text{cov}_\theta(T_i, T_j) &= \frac{\partial^2}{\partial \theta_i \partial \theta_j} \kappa(\theta).
\end{aligned}$$

Proof. For the mean we have

$$\begin{aligned}\mathrm{E}_\theta[T_i] &= \frac{\partial}{\partial \xi_i} M_T(\xi)\Big|_{\xi=0} \\ &= \frac{\partial}{\partial u_i} \frac{c(\theta)}{c(\xi+\theta)}\Big|_{u=0} \\ &= -\frac{\frac{\partial}{\partial \theta_i} c(\theta)}{c(\theta)} \\ &= -\frac{\partial}{\partial \theta_i} \log c(\theta).\end{aligned}$$

The proof for the covariance is similar. □

3.3. **Exponential tilting.** Let X be a random variable with density p and finite moment generating function $M(\xi) = \mathrm{E}[e^{\xi X}] < \infty$. Then the probability distribution given by

$$p(x \mid \xi) = \frac{e^{\xi x} p(x)}{M(\xi)},$$

is called an *exponentially tilted distribution*.

Now, if $p(x)$ is the density of a natural exponential family, $p(x \mid \theta) = c(\theta)h(x)\exp\{\theta x\}$, then the density of the exponentially tilted distribution is

$$p(x \mid \xi, \theta) = \frac{e^{ux} p(x \mid \theta)}{M(\xi)} = \frac{c(\theta)h(x)\exp\{(\theta+\xi)x\}}{c(\theta)/c(\theta+\xi)} = c(\theta+\xi)h(x)\exp\{(\theta+\xi)x\}.$$

Hence, for an exponential family, exponential tilting by ξ is identical to shifting the natural parameter by ξ.

This also suggests how to construct exponential families; start with a probability p and consider the family of all exponential tilts. This forms an exponential family. Indeed, if we tilt p by θ the resulting density is

$$p(x \mid \theta) = \frac{1}{M(\theta)} p(x)\exp\{\theta x\},$$

so putting $c(\theta) = 1/M(\theta)$ and $h(x) = p(x)$ yields the representation of a natural exponential family.

4. Elements of Bayesian decision theory

In this section we introduce the notion of sufficient statistics (in the Bayesian sense) and explain its use in Bayesian decision theory.

To gain information about the parameter we study various functions of the data X. For instance, if $X = (X_1, \ldots, X_n)$ are IID $\mathrm{Ber}(\theta)$ given $\Theta = \theta$, then we would use $T(X) = n^{-1}(X_1 + \cdots + X_n)$ to get information about the parameter. A function of the data is called a *statistic*.

Formally a statistic is a measurable mapping from the sample space $\mathcal{X}$ to some space $\mathcal{T}$, but we can also think of the composition $T \circ X : \Omega \to \mathcal{T}$ (recall that Ω is the underlying probability space). This is a random variable taking values in $\mathcal{T}$ and we often write T for this random quantity.

4.1. **Sufficient statistics (Bayesian).** A statistic T is called a sufficient statistic for the parameter Θ (in the Bayesian sense) if, for every prior p_Θ, the posterior distributions with respect to X and T coincide

$$p_{\Theta|T}(\theta \mid T(x)) = p_{\Theta|X}(\theta \mid x).$$

Hence, no matter what prior one uses, one only has to consider the sufficient statistic for making inference, because the posterior distribution given $T = T(x)$ is the same as the posterior given the data $X = x$.

One way to check that T is sufficient in the Bayesian sense is to check that the posterior $p_{\Theta|X}(\theta \mid x)$ is a function of $T(x)$.

We have the following result.

Lemma 1. *Let T be a statistic. T is sufficient in the Bayesian sense if and only if, for every prior p_Θ and every θ, the posterior $x \mapsto p_{\Theta|X}(\theta \mid x)$ is a function of $T(x)$.*

Proof. 'only if' part: If T sufficient in the Bayesian sense then for every prior $p_{\Theta|X}(\theta \mid x) = p_{\Theta|T}(\theta \mid T(x))$, which implies that $x \mapsto p_{\Theta|X}(\theta \mid x)$ is a function of $T(x)$.

'if' part: (We give the proof in the discrete case, i.e. $\mathcal{X} = \{x_1, x_2, \dots, \}$ and $T \in \{t_1, t_2, \dots\}$). Suppose that for every prior and each θ, $p_{\Theta|X}(\theta \mid x)$ is a function of $T(x)$. We want to show that T sufficient in the Bayesian sense. That is,

$$p_{\Theta|X}(\theta \mid x) = p_{\Theta|T}(\theta \mid T(x)).$$

Note that, with $T^{-1}(t) = \{x : T(x) = t\}$, it follows that

$$p_{\Theta|T}(\theta \mid t) = \frac{p_{T,\Theta}(t,\theta)}{p_T(t)} = \frac{\sum_{x\in T^{-1}(t)} p_{X,\Theta}(x,\theta)}{p_T(t)} = \frac{\sum_{x\in T^{-1}(t)} p_{\Theta|X}(\theta \mid x) p_X(x)}{p_T(t)}.$$

Since $p_{\Theta|X}(\theta \mid x)$ is a function of $T(x)$ it is constant over the set $T^{-1}(t)$ and the expression in the last display equals

$$\frac{p_{\Theta|X}(\theta \mid x) \sum_{x\in T^{-1}(t)} p_X(x)}{p_T(t)} = p_{\Theta|X}(\theta \mid x),$$

for any $x \in T^{-1}(t)$. □

Example 9. Let $\{X_n\}$ be conditionally IID $\text{Exp}(\theta)$ given $\Theta = \theta$ and $X = (X_1, \dots, X_n)$. Put $T(x) = x_1 + \cdots + x_n$. Let us show T is sufficient in the Bayesian sense. Let p_Θ be the prior (which is arbitrary). Then the posterior distribution has density (Bayes theorem)

$$p_{\Theta|X}(\theta \mid x) = \frac{\prod_{i=1}^n \theta e^{-\theta x_i} p_\Theta(\theta)}{\int \prod_{i=1}^n \vartheta e^{-\vartheta x_i} p_\Theta(\vartheta) d\vartheta} = \frac{\theta^n e^{-\theta \sum_{i=1}^n x_i} p_\Theta(\theta)}{\int \vartheta^n e^{-\vartheta \sum_{i=1}^n x_i} p_\Theta(\vartheta) d\vartheta}.$$

Since $x \mapsto p_{\Theta|X}(\theta \mid x)$ is a function of $T(x)$ it follows that T is a sufficient statistic in the Bayesian sense. In this case we can even compute explicitly the distribution of $T(X)$. Indeed, $T(X)$ has $\Gamma(n, \theta)$ distribution given $\Theta = \theta$ and it follows that

$$p_{\Theta|T}(\theta \mid t) = \frac{\frac{\theta^n}{\Gamma(n)} t^{n-1} e^{-\theta t} p_\Theta(\theta)}{\int \frac{\vartheta^n}{\Gamma(n)} t^{n-1} e^{-\vartheta t} p_\Theta(\vartheta) d\vartheta} = \frac{\theta^n e^{-\theta t} p_\Theta(\theta)}{\int \vartheta^n e^{-\vartheta t} p_\Theta(\vartheta) d\vartheta}.$$

That is, we have shown by direct calculation that $p_{\Theta|X}(\theta \mid x) = p_{\Theta|T}(\theta \mid T(x))$.

4.2. **How to find a sufficient statistic?** Suppose someone hands you a parametric family $\mathcal{P}_0 = \{p_{X|\Theta}(\cdot \mid \theta), \theta \in \Theta\}$ of densities. How do you come up with a sufficient statistic T? Further, if you have come up with a suggestion of a statistic T, how do you check if it is a sufficient statistic? The next theorem gives the answer.

Theorem 1 (Factorization Theorem, cf. Theorem 6.2.6 in Casella & Berger)**.** *Suppose $\mathcal{P}_0 = \{P_\theta : \theta \in \Theta\}$ is a parametric family. Then $T(X)$ is sufficient for Θ (in the Bayesian sense) if and only if there exist functions h and g such that*

$$p_{X|\Theta} = h(x)g(\theta, T(x)).$$

Proof. It is sufficient to check the equivalence in the Bayesian sense. If $p_{X|\Theta}(x \mid \theta) = h(x)g(\theta, T(x))$, then by Bayes' theorem

$$\begin{aligned} p_{\Theta|X}(\theta \mid x) &= \frac{p_{X|\Theta}(x \mid \theta)p_\Theta(\theta)}{\int p_{X|\Theta}(x \mid \vartheta)p_\Theta(\vartheta)d\vartheta} \\ &= \frac{h(x)g(\theta, T(x))p_\Theta(\theta)}{\int h(x)g(\vartheta, T(x))p_\Theta(\vartheta)d\vartheta} \\ &= \frac{g(\theta, T(x))p_\Theta(\theta)}{\int g(\vartheta, T(x))p_\Theta(\vartheta)d\vartheta}, \end{aligned}$$

which is a function of $T(x)$. Hence, by Lemma 1, it is sufficient in the Bayesian sense. Conversely, suppose $T(X)$ is sufficient in the Bayesian sense so that $p_{\Theta|X}(\theta \mid x) = p_{\Theta|T}(\theta \mid T(x))$. Then

$$p_{X|\Theta}(x \mid \theta) = \frac{p_{\Theta|X}(\theta \mid x)p_X(x)}{p_\Theta(\theta)} = \underbrace{p_X(x)}_{h(x)} \underbrace{\frac{p_{\Theta|T}(\theta \mid T(x))}{p_\Theta(\theta)}}_{g(\theta, T(x))}.$$

□

Example 10 (Exponential families)**.** If we put $T(X) = (t_1(X), \ldots, t_k(X))$ then by the factorization theorem it follows that $T(X)$ is sufficient. Indeed,

$$f_{X|\Theta}(x \mid \theta) = \underbrace{h(x)}_{h(x)} \underbrace{c(\theta) \exp\left\{\sum_{i=1}^k \theta_i t_i(x)\right\}}_{g(\theta, T(x))}.$$

5. Decision theory

Many statistical problems can be phrased in the language of decision theory. Suppose as usual that we have data X whose distribution depends on a parameter Θ. Based on observing $X = x$ we want to take some action. Let $\aleph$ be a set of possible actions. On $\aleph$ we take a σ-field α. The result of our action depends, not only on the chosen action, but also on an unobserved quantity Y taking values in a set $\mathcal{Y}$. For most of the text we will assume that X and Y are conditionally independent given the parameter Θ.

A *loss function* is a function $L : \mathcal{Y} \times \aleph \to \mathbb{R}$. We interpret $L(y, a)$ as the incurred loss if we took action a and the future outcome of Y is y.

A *deterministic decision rule* is a measurable function δ from $\mathcal{X}$ to $\aleph$. We interpret $\delta(x)$ as the action to take if we observe $X = x$. A *randomized decision rule* is a mapping from $\mathcal{X}$ to a probability measure on $(\aleph, \alpha)$ such that $x \mapsto \delta(A; x)$ is measurable for each $A \in \alpha$.

We think of executing a randomized decision rule as follows. Given $X = x$ we "throw a coin" according to $\delta(\cdot\,; x)$. This gives us an action $a \in \aleph$ which is the decision we take. A deterministic decision rule can be thought of as a special case of a randomized decision rule where all the probability mass is placed at a single action. In this case the action $\delta(x)$ is identified with the probability measure on $\aleph$ given by $\delta(A; x) = I_A(\delta(x))$.

If δ is a deterministic rule, then we associate the loss $L(y, \delta(x))$. If δ is a randomized decision rule we associate the loss $L(y, \delta(\cdot; x)) = \int_{\aleph} L(y, a)\delta(da; x)$. That is, the average loss when we draw the action from $\delta(\cdot; x)$.

In the Bayesian case, one introduces the *posterior risk function*

$$r(\delta \mid x) = \int_{\mathcal{Y}} L(y, \delta(x)) p_{Y|X}(y \mid x) dy.$$

That is, the average loss for decision rule δ given the observation $X = x$. If, for example, we have the likelihoods $p_{Y|\Theta}$, $p_{X|\Theta}$, and the prior p_Θ, then the posterior risk is

$$\begin{aligned} r(\delta \mid x) &= \int_{\mathcal{Y}} L(y, \delta(x)) p_{Y|X}(y \mid x) dy \\ &= \int_{\mathcal{Y}} L(y, \delta(x)) \left[\int_\Omega p_{Y|\Theta}(y \mid \theta) p_{\Theta|X}(\theta \mid x) d\theta \right] dy. \end{aligned}$$

One would like to find a decision rule that minimizes the posterior risk simultaneously for all $x \in \mathcal{X}$.

Definition 4. If δ_0 is a decision rule such that for all x, $r(\delta_0 \mid x) < \infty$ and for all x and all decision rules δ, $r(\delta_0 \mid x) \leq r(\delta \mid x)$, then δ_0 is called a *formal Bayes rule*.

Exercise 2 (A coin tossing example)**.** Consider the following situation. You have an amount of m dollars to bet on the outcome of a Bernoulli random variable X_{n+1}. You observe $X = (X_1, \ldots, X_n)$. Suppose $X_1, \ldots, X_{n+1}$ are conditionally IID Ber(θ) random variables given $\Theta = \theta$. Based on the observations in X you have to make a decision whether to bet on $X_{n+1} = 0$ or $X_{n+1} = 1$. If you win, you win the amount m and otherwise you lose m.

Formulate this as a Bayesian decision problem. Write down the sample space $\mathcal{X}$, the parameter space Θ, and the action space $\aleph$. Choose an appropriate prior distribution and an appropriate loss function of your choice. Then find the best decision rule (formal Bayes rule), i.e. the decision rule δ that minimizes the posterior risk simultaneously for all x.

Exercise 3 (Binary classification)**.** Consider a binary classification problem where we have a pair (X, Y) with $X \in \mathcal{X}$ and $Y \in \{-1, 1\}$. The objective is to construct a classifier that, given X, predicts the label Y. The action space is $\aleph = \{-1, 1\}$ and the loss function $L(y, a)$ can be taken to be $\gamma I\{y \neq a\}$ for some constant $\gamma > 0$. The posterior risk of a decision rule δ is

$$r(\delta \mid x) = L(-1, \delta(x)) p_{Y|X}(-1 \mid x) + L(1, \delta(x)) p_{Y|X}(1 \mid x).$$

Show that the decision rule $\delta(x) = 2I\{p_{Y|X}(1 \mid x) \geq 1/2\} - 1$ is a formal Bayes rule and determine its posterior risk.

Exercise 4 (Multi-class classification)**.** In classification problems with multiple classes $\{1, \ldots, C\}$ we have a pair (X, Y), where the class label Y may represented

as a *one-hot-vector*

$$Y = (Y^1, \dots, Y^C),$$

where all but one elements are zero and $Y^c = 1$ for the correct class. The objective is to construct a classifier that, given X, predicts the correct label. The action space is $\aleph = \{a_1, \dots, a_C\}$, where a_c is a one-hot-vector with a 1 in the cth component. The loss function $L(y, a)$ can be taken to be $\gamma I\{y \neq a\}$ for some constant $\gamma > 0$.

(a) Give an expression for the posterior risk of a decision rule δ, in terms of the class probabilities $p_{Y|X}(a_c \mid X)$.
(b) Determine a formal Bayes rule and its posterior risk.

Exercise 5 (Bayesian prediction)**.** Let $X_1, X_2, \dots$ be conditionally IID given $\Theta = \theta$ and represented as

$$X_i = \Theta + \varepsilon_i, \quad i = 1, 2, \dots,$$

where $\mathrm{E}[\varepsilon_i] = 0$ for each i and $\{\varepsilon_i\}_{i \geq 1}$ are IID and independent of Θ. The objective is, given observations $X_1 = x_1, \dots, X_n = x_n$, to predict X_{n+1}. Consider the squared loss

$$L(y, a) = (y - a)^2.$$

(a) Show that the posterior risk can be expressed as

$$r(\delta \mid x_1, \dots, x_n) = \mathrm{E}[(X_{n+1} - \delta(X_1, \dots, X_n))^2 \mid X_1 = x_1, \dots, X_n = x_n].$$

(b) Show that the predictive mean $\mathrm{E}[X_{n+1} \mid X_1 = x_1, \dots, X_n = x_n]$ is a formal Bayes rule.
(c) Show that the predictive mean equals the posterior mean, that is,

$$\mathrm{E}[X_{n+1} \mid X_1 = x_1, \dots, X_n = x_n] = \mathrm{E}[\Theta \mid X_1 = x_1, \dots, X_n = x_n].$$

5.1. **Sufficient statistics in decision theory.** If T is a sufficient statistic we would expect that we can base our decisions on T and do not need all the information in X since T contains all information about the unknown parameter Θ. In the Bayesian setting we have the following theorem that supports this.

Theorem 2. *If there is a formal Bayes rule and T is a sufficient statistic (in the Bayesian sense) then there is a formal Bayes rule which is a function of T.*

Proof. We first show that for any x, y such that $T(x) = T(y)$, it holds that $r(\delta \mid x) = r(\delta \mid y)$, as long as δ is a formal Bayes rule. Assume first that this is not true for two arbitrary $x, y \in \mathcal{X}$ with $T(x) = T(y)$, and a formal Bayes rule δ, and wlog that $r(\delta \mid x) < r(\delta \mid y)$. Then,

$$\begin{aligned}
r(\delta \mid x) &= \int L(\theta, \delta(x)) p_{\Theta|X}(\theta \mid x) d\theta \\
&= \int L(\theta, \delta(x)) p_{\Theta|T}(\theta \mid T(x)) d\theta \\
&= \int L(\theta, \delta(x)) p_{\Theta|T}(\theta \mid T(y)) d\theta \\
&= \int L(\theta, \delta(x)) p_{\Theta|X}(\theta \mid y) d\theta \\
&= r(\tilde{\delta} \mid y),
\end{aligned}$$

where the final equality is obtained by defining a new decision rule $\tilde{\delta}$ at y by $\delta(x)$ (and arbitrarily for all other $x' \in \mathcal{X}$). Thus, we have $r(\tilde{\delta} \mid y) = r(\delta \mid x) < r(\delta \mid y)$, contradicting that δ is a formal Bayes rule.

Assume that there exists a formal Bayes rule δ based on X, and define δ^T, based on T, by $\delta^T(t) = \delta(\tilde{x})$ for some $\tilde{x} \in T^{-1}(t)$, for each $t \in T$. We get from the equation above that

$$\begin{aligned} r(\delta^T \mid t) &= \int L(\theta, \delta^T(t)) p_{\Theta|T}(\theta \mid t) d\theta \\ &= \int L(\theta, \delta(\tilde{x})) p_{\Theta|T}(\theta \mid T(\tilde{x})) d\theta \\ &= r(\delta \mid \tilde{x}). \end{aligned}$$

Since $r(\delta \mid x) = r(\delta \mid \tilde{x})$ for any other x such that $T(x) = T(\tilde{x}) = t$, this means that $r(\delta^T \mid T(x)) = r(\delta \mid x)$. □

Note that (in the proof above) the formal Bayes rule δ that we started with does not have to be a function of T. For instance, it may be the case that $T(x) = T(y)$, $\delta(x) \neq \delta(y)$, and $L(\theta, \delta(x)) = L(\theta, \delta(y))$ for each θ. Then $r(\delta \mid x) = r(\delta \mid y)$ although, $\delta(x) \neq \delta(y)$.

5.2. **Remarks on finding a formal Bayes rule.** In Bayesian decision theory the following is a good way to find a deterministic formal Bayes rule.

(1) Take $x \in \mathcal{X}$.
(2) Find $a \in \aleph$ that minimizes $\int_{\mathcal{Y}} L(y, a) p_{Y|X}(y \mid x) dy$.
(3) Put $\delta(x) = a$.
(4) Repeat for all x.

However, it is not always that a formal Bayes rule exists, for instance the minimum in step (2) may not exist in $\aleph$. Here is a simple example.

Example 11. Let $X \sim N(\theta, 1)$ conditionally on $\Theta = \theta$, and $Y = \Theta \sim N(0, 1)$, where the parameter space is $\Theta = \mathbb{R}$. Then the predictive distribution of Y given $X = x$ is the posterior of Θ given $X = x$, which is $N(x/2, 1/2)$. Let the action space be $\aleph = \mathbb{R}$ and the loss function $L(\theta, a) = 0$ if $a \geq \theta$, $L(\theta, a) = 1$ if $a < \theta$. That is, a loss occurs if our guess of θ is below θ. Then for any x,

$$\int_{\Theta} L(\theta, a) p_{\Theta|X}(\theta \mid x) d\theta = \Pr(\Theta > a \mid X = x) = 1 - \Phi\Big(\frac{a - x/2}{1/\sqrt{2}}\Big).$$

This converges to 0 as $a \to \infty$, so the risk is minimized at $a = \infty$ but this is not in the action space $\aleph$. For this example no formal Bayes rule exists.

Lecture 3

6. Elements of classical decision theory

6.1. **Sufficient statistics (classical).** The idea of sufficiency is to find a function T of the data X that summarizes the information about the parameter Θ. In Section 4.1 we introduced the notion of sufficiency in the Bayesian sense. In this section we introduce the notion of sufficiency in the classical sense. As we will see below, the two notions are equivalent (in many practical situations).

We say that T is a sufficient statistic for Θ (in the classical sense) if $p_{X|T,\Theta}(x \mid t, \theta)$ does not depend on θ.

Note that, with $t = T(x)$,

$$p_{X|T,\Theta}(x \mid t, \theta) = \frac{p_{X,T|\Theta}(x, t \mid \theta)}{p_{T|\Theta}(t \mid \theta)} = \frac{p_{X|\Theta}(x \mid \theta)}{p_{T|\Theta}(t \mid \theta)}.$$

Hence T is sufficient if this ratio does not depend on θ.

To see how T captures the "information" about Θ we can write down the likelihood function as

$$p_{X|\Theta}(x \mid \theta) = p_{X|T,\Theta}(x \mid T(x), \theta) p_{T|\Theta}(T(x) \mid \theta)$$

If T is sufficient, then the first factor on the RHS does not depend on θ and the likelihood when observing $X = x$ is proportional (as a function of θ) to the likelihood when observing $T = T(x)$. That is, information about Θ comes only through the function T. If we, for example, want to maximize the likelihood we could maximize $p_{T|\Theta}(t \mid \theta)$ instead of maximizing $p_{X|\Theta}(x \mid \theta)$. In this sense, there is no need to know x itself, it is sufficient to know $t = T(x)$ to do inference.

The simplest example (but not particularly useful) of a sufficient statistic is the data itself. That is $\mathcal{T} = \mathcal{X}$ and $T(x) = x$. Using this statistic does not help you to summarize information about the parameter. We have already seen several examples of sufficient statistics in the Bayesian sense, but let's look at some simple cases where we may check directly that a statistic is sufficient in the classical sense.

Example 12 (cf. Example 6.2.3 in Casella & Berger)**.** Let $\{X_n\}$ be IID $\mathrm{Ber}(\theta)$ given $\Theta = \theta$ and $X = (X_1, \ldots, X_n)$. Put $T(x) = x_1 + \cdots + x_n$. Let us show T is sufficient. Note that $T(X)$ is $\mathrm{Bin}(n, \theta)$ given $\Theta = \theta$. For each $x = (x_1, \ldots, x_n)$ $x_i \in \{0, 1\}$ such that $t = T(x)$

$$p_{X|\Theta,T}(x \mid \theta, t) = \frac{p_{X,T|\Theta}(x, t)}{p_{T|\Theta}(t \mid \theta)} = \frac{\theta^t (1-\theta)^{n-t}}{\binom{n}{t} \theta^t (1-\theta)^{n-t}} = \binom{n}{t}^{-1}.$$

Since this does not depend on θ, T is a sufficient statistic.

Example 13. Let $\{X_n\}$ be IID $\mathrm{Exp}(\theta)$ given $\Theta = \theta$ and $X = (X_1, \ldots, X_n)$. Put $T(x) = x_1 + \cdots + x_n$. Let us show T is sufficient. Note that $T(X)$ is $\Gamma(n, \theta)$ given $\Theta = \theta$. For each $x_i > 0$ we have with $t = T(x)$

$$\frac{p_{X|\Theta}(x \mid \theta)}{p_{T|\Theta}(t \mid \theta)} = \frac{\prod_{i=1}^n \theta e^{-\theta x_i}}{\frac{\theta^n}{\Gamma(n)} t^{n-1} e^{-\theta t}} = \frac{(n-1)!}{t^{n-1}},$$

which does not depend on θ.

It is reassuring to know that in many practical situations (in fact all situations we will encounter in this course) the classical and Bayesian notion of sufficiency are the same.

Theorem 3. *Suppose all relevant densities exist. T is sufficient in the classical sense if and only if it is sufficient in the Bayesian sense.*

Proof. Let p_Θ be an arbitrary prior. If T sufficient in the classical sense, then $p_{X|\Theta,T}(x \mid \theta, t) = p_{X|T}(x \mid t)$. Hence the posterior density is (with $t = T(x)$)

$$\begin{aligned} p_{\Theta|X}(\theta \mid x) &= \frac{p_{X|\Theta}(x \mid \theta)p_\Theta(\theta)}{\int p_{X|\Theta}(x \mid \vartheta)p_\Theta(\vartheta)(d\vartheta)} \\ &= \frac{p_{X|\Theta,T}(x \mid \theta, t)p_{T|\Theta}(t \mid \theta)p_\Theta(\theta)}{\int p_{X|\Theta,T}(x \mid \vartheta, t)p_{T|\Theta}(t \mid \vartheta)p_\Theta(\vartheta)d\vartheta} \\ &= \frac{p_{X|T}(x \mid t)p_{T|\Theta}(t \mid \theta)p_\Theta(\theta)}{\int p_{X|T}(x \mid t)p_{T|\Theta}(t \mid \vartheta)p_\Theta(\vartheta)d\vartheta} \\ &= \frac{p_{T|\Theta}(t \mid \theta)p_\Theta(\theta)}{\int p_{T|\Theta}(t \mid \vartheta)p_\Theta(\vartheta)d\vartheta} \\ &= p_{\Theta|T}(\theta \mid t). \end{aligned}$$

For the converse suppose that T is sufficient in the Bayesian sense so that $p_{\Theta|X}(\theta \mid x) = p_{\Theta|T}(\theta \mid T(x))$. Then, with $t = T(x)$ we have

$$\begin{aligned} p_{X|T,\Theta}(x \mid t, \theta) &= \frac{p_{X|\Theta}(x \mid \theta)}{p_{T|\Theta}(t \mid \theta)} \\ &= \frac{p_{\Theta|X}(\theta \mid x)\int p_{X|\Theta}(x \mid \vartheta)p_\Theta(\vartheta)d\vartheta}{p_{\Theta|T}(\theta \mid t)\int p_{T|\Theta}(t \mid \vartheta)p_\Theta(\vartheta)d\vartheta} \\ &= \frac{p_X(x)}{p_T(t)} \end{aligned}$$

which does not depend on θ. Hence, T is sufficient in the classical sense. □

By Theorem 3, it follows that we can use the factorization theorem, Theorem 1, to identify a sufficient statistic in the classical sense.

6.2. **Classical decision theory.** *In this section we introduce the basic concepts in classical decision theory. The purpose of this section is to provide some basic insights into classical decision theory. However, in the remaining parts of the course we will use Bayesian decision theory and empirical risk, so we will not put emphasis on classical decision theory,*

In classical decision theory the setup is similar to Bayesian decision theory. Suppose that we have data X whose distribution depend on a parameter Θ. Based on observing $X = x$ we want to take some action. Let $\aleph$ be a set of possible actions, equipped with the sigma-field α. The result of our action depends, not only on the chosen action, but also on an unobserved quantity Y taking values in a set $\mathcal{Y}$. Throughout this lecture we assume that X and Y are conditionally independent given $\Theta = \theta$ for any $\theta \in \Theta$.

A *loss function* is a function $L : \mathcal{Y} \times \aleph \to \mathbb{R}$. We interpret $L(y, a)$ as the incurred loss if we took action a and the future outcome of Y is y.

A *deterministic decision rule* is a measurable function δ from $\mathcal{X}$ to $\aleph$. We interpret $\delta(x)$ as the action to take if we observe $X = x$. A *randomized decision rule* is a mapping from $\mathcal{X}$ to a probability measure on $(\aleph, \alpha)$ such that $x \mapsto \delta(A; x)$ is measurable for each $A \in \alpha$.

If δ is a deterministic rule, then we associate the loss $L(y, \delta(x))$. If δ is a randomized decision rule we associate the loss $L(y, \delta(\cdot; x)) = \int_{\aleph} L(y, a)\delta(da; x)$. That is, the average loss when we draw the action from $\delta(\cdot; x)$.

In the classical case we condition on $\Theta = \theta$ and introduce the *risk function*

$$R(\theta, \delta) = \int_{\mathcal{X}} \int_{\mathcal{Y}} L(y, \delta(x)) p_{Y|X,\Theta}(y \mid x, \theta) p_{X|\Theta}(x \mid \theta) dy dx.$$

Sometimes it is convenient to express the loss function as a function of the parameter θ and the action a as

$$L'(\theta, a) = \int_{\mathcal{Y}} L(y, a) p_{Y|\Theta}(y \mid \theta) dy.$$

Then the risk function can be written as the conditional mean of the loss, given $\Theta = \theta$:

$$R(\theta, \delta) = \int_{\mathcal{X}} L'(\theta, \delta(x)) p_{X|\Theta}(x \mid \theta) dx = \mathrm{E}[L'(\theta, \delta(X)) \mid \Theta = \theta].$$

Ideally, we would like to find a rule δ that minimizes the risk function simultaneously for all values of θ.

The role of sufficient statistics in classical decision theory is explained by the following theorem. It says that decisions only need to be based on sufficient statistics of the data, no additional information is needed.

Theorem 4. *If δ_0 is a (randomized) decision rule and T is sufficient statistic (in classical sense), then there exists a decision rule δ_1 which is a function of T and $R(\theta, \delta_0) = R(\theta, \delta_1)$ for all θ.*

In the theorem, if δ_0 is deterministic we interpret it as the randomized rule δ by $\delta(A; x) = I_A(\delta_0(x))$. That is, the probability measure on $(\aleph, \alpha)$ that puts all its mass on $\delta_0(x)$. Denote the density of $\delta(\cdot; x)$ at a by $\delta(a; x)$, with some abuse of notation.

Proof. Let $A \in \alpha$ and take

$$\delta_1(A; t) = \mathrm{E}[\delta_0(A; X) \mid T = t, \Theta = \theta].$$

Since T is sufficient, the expectation does not depend on θ. We claim that for any δ_0-integrable function $h : \aleph \to \mathbb{R}$

$$\mathrm{E}\Big[\int_{\aleph} h(a)\delta_0(a; X) da \mid T = t, \Theta = \theta\Big] = \int_{\aleph} h(a)\delta_1(a; t) da.$$

Indeed,

$$\begin{aligned}
\mathrm{E}\Big[\int_{\aleph} h(a)\delta_0(a; X) da \mid T = t\Big] &= \int_{\mathcal{X}} \int_{\aleph} h(a)\delta_0(a; x) p_{X|T}(x \mid t) da dx \\
&= \int_{\aleph} \int_{\mathcal{X}} h(a)\delta_0(a; x) p_{X|T,\Theta}(x \mid t, \theta) dx da \\
&= \int_{\aleph} h(a)\delta_1(a; t) da
\end{aligned}$$

Then we see that

$$\begin{aligned}
R(\theta,\delta_1) &= \int_{\mathcal{X}}\int_{\aleph} L(\theta,a)\delta_1(a;T(x))p_{X|\Theta}(x\mid\theta)dadx\\
&= \int_{\mathcal{X}} \mathrm{E}\Big[\int_{\aleph} L(\theta,a)\delta_0(a;X)da \mid T=T(x),\Theta=\theta\Big]p_{X|\Theta}(x\mid\theta)dx\\
&= \mathrm{E}\Big[\mathrm{E}\Big[\int_{\aleph} L(\theta,a)\delta_0(a;X)da \mid T,\Theta\Big] \mid \Theta=\theta\Big]\\
&= \mathrm{E}\Big[\int_{\aleph} L(\theta,a)\delta_0(a;X)da \mid \Theta=\theta\Big]\\
&= \int_{\mathcal{X}}\int_{\aleph} L(\theta,a)\delta_0(a;x)p_{X|\Theta}(x\mid\theta)dadx\\
&= R(\theta,\delta_0).
\end{aligned}$$

□

One should note that even if δ_0 is a deterministic rule, the resulting rule $\delta_1(A;t) = \mathrm{E}[\delta_0(A;X) \mid T=t,\Theta=\theta]$ may be randomized.

6.3. **Neyman-Pearson's lemma.** Let us first introduce the notion of admissible decision rules.

Definition 5. Let δ be a decision rule. If there exists a decision rule δ_1 such that $R(\theta,\delta_1) \le R(\theta,\delta)$ for all θ with strict inequality for some θ, then we say δ that is *inadmissible* and that it is *dominated* by δ_1. Otherwise, δ is *admissible*.

Of course, one should not use inadmissible decision rules. Among the admissible decision rules, there is no obvious choice, which decision rule to take. One may introduce additional constraints/objectives to choose among the admissible decision rules. In the following we will identify all the admissible decision rules in some simple settings. It is surprisingly complicated to identify the admissible decision rules, and beyond the scope of this course to do it in more complicated settings.

Definition 6. A class $\mathcal{C}$ of decision rules is *complete* if for every $\delta \notin \mathcal{C}$ there exists $\delta_0 \in \mathcal{C}$ that dominates δ, i.e. $R(\theta,\delta_0) \le R(\theta,\delta)\ \forall\theta$ with strict inequality for some θ. A class is *minimal complete* if no proper subclass is also complete.

To see the relation to admissible decision rules, we have the following:

Lemma 2. *A minimal complete class consists exactly of the admissible decision rules.*

Proof. First we show that δ admissible implies $\delta \in \mathcal{C}$. Indeed, if $\delta \notin \mathcal{C}$ then there exists $\delta_0 \in \mathcal{C}$ that dominates δ, which contradicts that δ is admissible.

For the other inclusion we need to show that $\delta \in \mathcal{C}$ implies δ is admissible. Suppose it is not admissible. Then there exists a dominating rule δ_1. Either $\delta_1 \in \mathcal{C}$ or $\delta_1 \notin \mathcal{C}$. In the first case put $\delta_2 = \delta_1$. In the second, there is $\delta_2 \in \mathcal{C}$ that dominates δ_1. Thus, in both cases $\delta_2 \in \mathcal{C}$ dominates δ. If δ' is a rule that is dominated by δ, then it is also dominated by δ_2. This implies that $\mathcal{C}\setminus\{\delta\}$ is complete. This is a contradiction because we assumed that $\mathcal{C}$ is minimal complete. Hence, δ is admissible. □

There is one simple case where a minimal complete class can be found. This is called the Neyman-Pearson fundamental lemma. It concerns the situation where $Y = \Theta$ and Θ can take only two values 0 and 1. The objective is, given data X from $p_{X|\Theta}$, to decide whether X was generated from model "0" or model "1". Since Θ can take only two values we will denote the likelihood by p_0 and p_1 for the two possibilities.

Theorem 5. *Let $\Theta = \aleph = \{0, 1\}$, $L(0, 0) = L(1, 1) = 0$, $L(1, 0) = k_1 > 0$, and $L(0, 1) = k_0 > 0$. Let $p_i(x)$ $i = 0, 1$ be the densities of the two models. For δ, a decision rule, let $\phi(x) = \delta(1; x)$ be the probability to select 1 given x (the so called test function of δ). Let $\mathcal{C}$ be the class of rules with test functions of the form below:*

For each $k \in (0, \infty)$ and each function $\gamma : \mathcal{X} \to [0, 1]$,

$$\phi_{k,\gamma}(x) = \begin{cases} 1, & p_1(x) > kp_0(x), \\ \gamma(x), & p_1(x) = kp_0(x), \\ 0, & p_1(x) < kp_0(x). \end{cases}$$

For $k = 0$,

$$\phi_0(x) = \begin{cases} 1, & p_1(x) > 0, \\ 0, & p_1(x) = 0. \end{cases}$$

For $k = \infty$,

$$\phi_\infty(x) = \begin{cases} 1, & p_0(x) = 0, \\ 0, & p_0(x) > 0. \end{cases}$$

Then $\mathcal{C}$ is a minimal complete class.

As we will not pursue classical decision theory, we omit the proof. It can be found in e.g. M. J. Schervish, *Theory of Statistics*, Springer 1995, or other advanced books on statistics. The decision rules are associated with a threshold $k \in [0, \infty]$.

- To $k = 0$ there corresponds one decision rule which says "choose $a = 1$ if $p_1(x) > 0$ and $a = 0$ otherwise".
- To $k = \infty$ there corresponds one decision rule which says "choose $a = 1$ if $p_0(x) = 0$ and $a = 0$ otherwise".
- To each $k \in (0, \infty)$ there are lots of decision rules. They all say that $a = 1$ should be chosen if it is sufficiently likely that $\theta = 1$. That is: "choose $a = 1$ if $p_1(x) > kp_0(x)$, choose $a = 0$ if $p_1(x) < kp_0(x)$, and in the event that we cannot decide $f_1(x) = kf_0(x)$ we choose $a = 1$ with probability $\gamma(x)$ where γ is some function $\gamma : \mathcal{X} \to [0, 1]$".

Example 14. The Neyman-Pearson lemma can be used when deciding between competing models. Suppose we have two competing models for the distribution of X given by continuous densities f_0 and f_1 w.r.t. Lebesgue measure. Based on observing $X = x$ we have to decide which is the more appropriate one. Decisions are $a = 1$ "f_1 is correct" and $a = 0$ "f_0 is correct". The Neyman-Pearson lemma says that the admissible rules (the minimal complete class) are of the form: for $k \in (0, \infty)$ choose $a = 1$ if $p_1(x) > kp_1(x)$ and $a = 0$ if $p_1(x) < kp_0(x)$. There is no need to specify the case $p_1(x) = kp_0(x)$ since this event has probability zero. Also the cases $k = 0$ or ∞ corresponds to "always choose $a = 1$" and "always choose $a = 0$". None of these seem very desirable, but they are admissible.

Example 15. If we continue the above example when $p_0(x) = \lambda_0 e^{-\lambda_0 x}$ and $p_1(x) = \lambda_1 e^{-\lambda_1 x}$ we see that we choose $a = 1$ if

$$\frac{p_1(x)}{p_0(x)} > k \iff x \leq \frac{\log \lambda_0 - \log \lambda_1 + \log k}{\lambda_1 - \lambda_0}.$$

You can think of the case $k = 1$ as the fair case where we choose the model which is most likely. $k > 1$ penalizes choosing $a = 1$ whereas $k < 1$ penalizes choosing $a = 0$.

6.4. **Rao-Blackwell theorem.** Sometimes it is not very nice to use randomized decision rules, so a question is if it can be avoided. Under convexity assumptions it is a good idea to take the mean of a randomized rule.

Proposition 3. *Suppose $\aleph \subset \mathbb{R}^m$ is convex and for each θ, $a \mapsto L(\theta, a)$ is convex. Let δ be a randomized rule, $B = \{x : \int_\aleph |a|\delta(a; x)da < \infty\}$ and put*

$$\delta_0(x) = \int_\aleph a\delta(a; x)da, \quad x \in B,$$

the mean of the randomized rule δ. Then $L(\theta, \delta_0(x)) \leq L(\theta, \delta(x))$ for each θ and $x \in B$.

Proof. By Jensen's inequality

$$L(\theta, \delta_0(x)) = L(\theta, \int_\aleph a\delta(a; x)da) \leq \int_\aleph L(\theta, a)\delta(a; x)da = L(\theta, \delta(x)).$$

□

The Rao-Blackwell theorem tells us that we can possibly improve further by conditioning on a sufficient statistic.

Theorem 6 (Rao-Blackwell theorem)**.** *Suppose $\aleph \subset \mathbb{R}^m$ is convex, for each θ, $a \mapsto L(\theta, a)$ is convex, T is a sufficient statistic, and δ_0 a deterministic rule with $\mathrm{E}[\|\delta_0(X)\| \mid \Theta = \theta] < \infty$. Put*

$$\delta_1(t) = \mathrm{E}[\delta_0(X) \mid T = t],$$

then $R(\theta, \delta_1) \leq R(\theta, \delta_0)$.

Proof. Think of δ_0 as random by putting $\delta_3(A; x) = I_A(\delta_0(x))$. We also put

$$\delta_4(A; t) = \mathrm{E}[\delta_3(A; X) \mid T = t],$$
$$\delta_2(t) = \int_\aleph a\delta_4(a; t)da.$$

Then,

$$R(\theta, \delta_2) \leq \{\text{Proposition 3}\} \leq R(\theta, \delta_4) = \{\text{Theorem 4}\} = R(\theta, \delta_3) = R(\theta, \delta_0).$$

It remains to show $\delta_2 = \delta_1$:

$$\delta_2(t) = \int_\aleph a\delta_4(a; t)da = \mathrm{E}[\int_\aleph a\delta_3(a; X)da \mid T = t] = \mathrm{E}[\delta_0(X) \mid T = t] = \delta_1(t).$$

Since δ_2 is a deterministic decision rule, so is δ_1. □

7. Bayes risk and Empirical risk

There is a weaker concept than a formal Bayes rule. Denote by p_Θ the prior distribution of Θ. Together with the likelihood $p_{X|\Theta}$ it specifies the predictive (marginal) distribution of X, written p_X. We call the function

$$r(p_\Theta, \delta) = \int_{\mathcal{X}} r(\delta \mid x) p_X(x) dx$$

the *Bayes risk* and each δ that minimizes the Bayes risk is called a *Bayes rule* with respect to p_Θ, assuming $r(p_\Theta, \delta) < \infty$. The Bayes risk is the mean of the posterior risk, before observing $X = x$.

Alternatively, we may start from classical decision theory and the risk function $R(\theta, \delta)$. Take a prior distribution p_Θ for Θ and minimize

$$\int_\Theta R(\theta, \delta) p_\Theta(\theta) d\theta.$$

Note that by interchanging the order of integration, and using Bayes' theorem, it follows that

$$\begin{aligned}
\int_\Theta R(\theta, \delta) p_\Theta(\theta) d\theta &= \int_\Theta \int_{\mathcal{X}} L'(\theta, \delta(x)) p_{X|\Theta}(x \mid \theta) p_\Theta(\theta) dx d\theta \\
&= \int_{\mathcal{X}} \int_\Theta L'(\theta, \delta(x)) p_{\Theta|X}(\theta \mid x) p_X(x) d\theta dx \\
&= \int_{\mathcal{X}} \int_\Theta \int_{\mathcal{Y}} L(y, \delta(x)) p_{Y|\Theta}(y \mid \theta) p_{\Theta|X}(\theta \mid x) p_X(x) dy d\theta dx \\
&= \int_{\mathcal{X}} r(\delta \mid x) p_X(x) dx \\
&= r(p_\Theta, \delta),
\end{aligned}$$

which is the Bayes risk with respect to p_Θ.

Note that we can also write the Bayes risk as

$$\int_{\mathcal{X}} r(\delta \mid x) p_X(x) dx = \int_{\mathcal{X}} \left(\int_{\mathcal{Y}} L(y, \delta(x)) p_{Y|X}(y \mid x) dy \right) p_X(x) dx = \mathrm{E}[L(Y, \delta(X))],$$

where the expectation is taken with respect to the joint distribution of (X, Y). If we directly specify the joint distribution of (X, Y), then there is no need to introduce a parameter Θ and the associated prior p_Θ. In this case it is more convenient to write the Bayes risk as a function of the decision rule δ (without reference to p_Θ):

$$R(\delta) = \mathrm{E}[L(Y, \delta(X))].$$

When observations of (X, Y) are available we may approximate the risk by the empirical risk. This is the topic of the next subsection.

7.1. Empirical risk.

Suppose that $\{(X_i, Y_i)\}_{i=1}^{n+1}$ are IID with (unknown) joint density $p_{X,Y}$. Suppose we have observations $(x_1, y_1), \ldots, (x_n, y_n)$ and x_{n+1} of the corresponding random variables. Let $X = X_{n+1}$ and $Y = Y_{n+1}$ and consider a decision rule $\delta : \mathcal{X} \to \aleph$. If n is large, it is reasonable to approximate the Bayes risk

$$R(\delta) = \mathrm{E}[L(Y, \delta(X))],$$

by the empirical risk

$$R_n^{\text{emp}}(\delta) = \frac{1}{n}\sum_{i=1}^{n} L(y_i, \delta(x_i)).$$

By the strong law of large numbers $\lim_{n\to\infty} R_n^{\text{emp}}(\delta) = R(\delta)$ almost surely. It is tempting to look for a decision rule that minimizes the empirical risk. However, this may easily lead to overfitting.

Example 16 (Overfitting)**.** Consider minimizing the empirical risk under squared loss, $L(y,a) = (y-a)^2$, among all (measurable) functions $\delta : \mathcal{X} \to \mathbb{R}$, where we have assumed $\mathcal{Y} = \mathbb{R}$. The expected loss is minimized by taking $\delta(x_i) = y_i$, $i = 1, \ldots, n$, and $\delta(x) = 0$ for all other x. Obviously, this choice of δ is not particularly good, as it will provide action 0 for all new (distinct) observations x.

To avoid overfitting it is useful to consider a restricted class $\mathcal{D}$ of decision rules, which has some regularity. For instance, one could consider decision rules of the form

$$\delta(x) = \sum_{j=1}^{d} \theta_j \phi_j(x),$$

where $\phi_1, \ldots, \phi_d$ are given basis functions and $\theta_1, \ldots, \theta_d$ are coefficients. That is, we might take

$$\mathcal{D} = \{\delta : \delta(x) = \sum_{j=1}^{d} \theta_j \phi_j(x), (\theta_1, \ldots, \theta_d) \in \mathbb{R}^d\}.$$

There are many other possible choices for $\mathcal{D}$.

Denote by δ^* a Bayes rule, which minimizes the Bayes risk, by δ_1^* a decision rule that minimizes the Bayes risk in the class $\mathcal{D}$, and δ_2^* a decision rule that minimizes the empirical risk in the class $\mathcal{D}$. In practice only δ_2^* can be found, by minimizing the empirical risk. The residual risk can be decomposed as

$$R(\delta_2^*) - R(\delta^*) = \underbrace{[R(\delta_2^*) - R(\delta_1^*)]}_{\text{estimation error}} + \underbrace{[R(\delta_1^*) - R(\delta^*)]}_{\text{approximation error}},$$

where the estimation error is the error due to the limitation of using training data and the approximation error is the error by restricting the decision rule to belong to the class $\mathcal{D}$. It may very well be that the Bayes rule δ^* does not belong to $\mathcal{D}$. The more training data we have, the smaller is the estimation error and the more flexible the class $\mathcal{D}$, the smaller the approximation error.

Lecture 4

8. An overview of methods for classification

8.1. **Bayesian approach to binary classification.** Recall Exercise 3 where we considered a pair (X, Y) with $X \in \mathcal{X}$ and $Y \in \{-1, 1\}$. The objective was to construct a classifier that, given X, predicts the label Y. The action space is $\aleph = \{-1, 1\}$ and the loss function $L(y, a)$ can be taken to be $I\{y \neq a\}$. The posterior risk of a decision rule δ is given by

$$r(\delta \mid x) = L(-1, \delta(x))p_{Y|X}(-1 \mid x) + L(1, \delta(x))p_{Y|X}(1 \mid x).$$

A formal Bayes rule is given by $\delta(x) = 1$ if $p_{Y|X}(1 \mid x) \geq 1/2$ and $\delta(x) = -1$ otherwise. Note that in the setting just described, we have not specified what X is. For instance, suppose we have training data in the form of $\{(x_i, y_i)\}_{i=1}^n$ that are outcomes of independent pairs $\{(X_i, Y_i)\}_{i=1}^n$ where X_i represents explanatory variables (say, images), and Y_i the corresponding label (say, 1 if the image contains a cat and -1 otherwise). We also have $X_{n+1} = x_{n+1}$ and want to predict the label Y_{n+1}. In this case we could put $X = (X_1, Y_1, \ldots, X_n, Y_n, X_{n+1})$ and $Y = Y_{n+1}$ and a formal Bayes rule is based on the predictive class probability:

$$p_{Y|X}(1 \mid x) = p_{Y_{n+1}|X_1,Y_1,\ldots,X_n,Y_n,X_{n+1}}(1 \mid x_1, y_1, \ldots, x_n, y_n, x_{n+1}).$$

What remains is to model the predictive distribution $p_{Y|X}$ and in the sequel we will consider a variety of models and methods for computing $p_{Y|X}(1 \mid x)$.

8.1.1. *Bayesian logistic regression.* As a warm-up, consider the following exercise where there are no explanatory variables present. We will need the sigmoid function, denoted sigm, defined by

$$\text{sigm}(t) = \frac{e^t}{1 + e^t} = (1 + e^{-t})^{-1}.$$

Note that $\text{sigm}(t) + \text{sigm}(-t) = 1$ for all $t \in \mathbb{R}$.

Exercise 6. Let $Y_1, Y_2, \ldots$ be conditionally IID random variables given $\Theta = \theta$ where

$$\Pr(Y_i = 1) = \text{sigm}(\theta),$$
$$\Pr(Y_i = -1) = \text{sigm}(-\theta).$$

The parameter space is $\Theta = \mathbb{R}$.

(a) Show that $\{p_{Y_1,\ldots,Y_n|\Theta}(y_1, \ldots, y_n \mid \theta), \theta \in \mathbb{R}\}$ is a natural exponential family and determine the sufficient statistic.
(b) Suppose we have observed $X = (Y_1, \ldots, Y_n)$ and we want to predict $Y = Y_{n+1}$. Derive the predictive distribution $p_{Y|X}(y \mid x) = p_{Y_{n+1}|Y_1,\ldots,Y_n}(y \mid y_1, \ldots, y_n)$.
(c) Take a conjugate prior (consider conjugate prior for exponential families) and derive an explicit expression for the predictive distribution.

Having completed the warm-up exercise we are ready for the next step. Suppose we have training data in the form of $\{(x_i, y_i)\}_{i=1}^n$ that are outcomes of independent pairs $\{(X_i, Y_i)\}_{i=1}^n$ where X_i represents the explanatory variables and Y_i the corresponding label. We also have $X_{n+1} = x_{n+1}$ and want to predict the label Y_{n+1}. In logistic

regression we assume that $Y_1 \mid X_1, Y_2 \mid X_2, \dots$ are conditionally independent given $\Theta = \theta \in \mathbb{R}^d$ with

$$\Pr(Y_i = 1 \mid \Theta = \theta, X_i = x_i) = \text{sigm}\Big(\sum_{j=1}^d \theta_j \phi_j(x_i)\Big),$$

$$\Pr(Y_i = -1 \mid \Theta = \theta, X_i = x_i) = \text{sigm}\Big(-\sum_{j=1}^d \theta_j \phi_j(x_i)\Big),$$

where $\phi_1, \dots, \phi_d$ are given functions, summarizing the necessary information in X_i. With $X = (X_1, Y_1, \dots, X_n, Y_n, X_{n+1})$ and $Y = Y_{n+1}$ we have the likelihood

$$\begin{aligned} p_{X|\Theta}(x \mid \theta) &= p_{Y_1,\dots,Y_n|\Theta,X_1,\dots,X_{n+1}}(y_1, \dots, y_n \mid \theta, x_1, \dots, x_{n+1}) p_{X_1,\dots,X_{n+1}}(x_1, \dots, x_{n+1}) \\ &= \prod_{i=1}^n \text{sigm}\Big(y_i \sum_{j=1}^d \theta_j \phi_j(x_i)\Big) \prod_{i=1}^{n+1} p_{X_i}(x_i). \end{aligned}$$

Note that, by the factorization theorem, a sufficient statistic for Θ is given by $\{(y_i, \phi(x_i))\}_{i=1}^n$, where $\phi(x) = (\phi_1(x), \dots, \phi_d(x))^{\mathrm{T}}$. Given a prior p_Θ, the predictive distribution is given by

$$\begin{aligned} p_{Y|X}(y \mid x) &= \int_\Theta p_{Y|\Theta,X}(y \mid \theta, x) p_{\Theta|X}(\theta \mid x) d\theta \\ &= \int_\Theta p_{Y_{n+1}|\Theta,X_1,Y_1,\dots,X_n,Y_n,X_{n+1}}(y \mid \theta, x_1, y_1, \dots, x_n, y_n, x_{n+1}) \\ &\qquad \times p_{\Theta|X_1,Y_1,\dots,X_n,Y_n,X_{n+1}}(\theta \mid x_1, y_1, \dots, x_n, y_n, x_{n+1}) d\theta \\ &= \int_\Theta p_{Y_{n+1}|\Theta,X_{n+1}}(y \mid \theta, x_{n+1}) p_{\Theta|X_1,Y_1,\dots,X_n,Y_n,X_{n+1}}(\theta \mid x_1, y_1, \dots, x_n, y_n, x_{n+1}) d\theta \\ &= \int_\Theta \text{sigm}\Big(y \sum_{j=1}^d \theta_j \phi_j(x_{n+1})\Big) p_{\Theta|X_1,Y_1,\dots,X_n,Y_n}(\theta \mid x_1, y_1, \dots, x_n, y_n) d\theta \end{aligned}$$

To classify a new observation x_{n+1} we must be able to evaluate this predictive distribution. We will, in subsequent lectures, consider a number of alternatives for evaluating the predictive distribution. These are not limited to logistic regression but can be considered for a variety of different models for Bayesian regression.

- If we are lucky, we can design a prior p_Θ for which the predictive distribution can be evaluated explicitly. This is a rare situation in practice, but it works for instance in the warm-up exercise.
- We may approximate the posterior $p_{\Theta|X}(\theta \mid x)$ by a delta-function at its maximum argument. That is, find the maximizer of the posterior distribution, called MAP (maximum a posteriori),

  $$\theta^{\text{MAP}} = \text{argmax}_\theta p_{\Theta|X}(\theta \mid x),$$

  and approximate

  $$\begin{aligned} p_{Y|X}(y \mid x) &= \int_\Theta p_{Y|\Theta,X}(y \mid \theta, x) p_{\Theta|X}(\theta \mid x) d\theta \\ &\approx p_{Y|\Theta,X}(y \mid \theta^{\text{MAP}}, x). \end{aligned}$$

- Approximate the posterior $p_{\Theta|X}(\theta \mid x)$ by a distribution $q_{\Theta|X}(\theta \mid x)$, where $q_{\Theta|X}$ is chosen so that

$$\int_{\Theta} p_{Y|\Theta,X}(y \mid \theta, x) q_{\Theta|X}(\theta \mid x) d\theta$$

can be evaluated. That is, the predictive distribution is approximated by

$$\begin{aligned} p_{Y|X}(y \mid x) &= \int_{\Theta} p_{Y|\Theta,X}(y \mid \theta, x) p_{\Theta|X}(\theta \mid x) d\theta \\ &\approx \int_{\Theta} p_{Y|\Theta,X}(y \mid \theta, x) q_{\Theta|X}(\theta \mid x) d\theta. \end{aligned}$$

- Approximate the predictive distribution by Monte Carlo simulation by using e.g. Markov chain Monte Carlo (MCMC) to generate samples θ_k, $k = 1, \ldots, N$ from the true posterior $p_{\Theta|X}$ and approximate the predictive distribution by the sample average,

$$p_{Y|X}(y \mid x) \approx \frac{1}{N} \sum_{k=1}^{N} p_{Y|\Theta,X}(y \mid \theta_k, x).$$

Exercise 7 (Segmentation (i))**.** In a machine learning application on image segmentation, images containing d pixels are considered. In each image, there is an "object" that occupies exactly V pixels, where V is known and $V \leq d$ (you may think of V as volume). Nothing is known about the "object" except its volume. For instance, it may be entirely disconnected. Let $Y = (Y_1, \ldots, Y_d)$ be a random segmentation, that is, a binary random vector taking values in $\{0, 1\}^d$, where $Y_i = 1$ indicates that the object occupies the ith pixel. Since the volume is known, it holds that $\sum_{i=1}^{d} Y_i = V$ with probability 1. Consider a Bayesian model for Y, with parameter $\Theta = (\Theta_1, \ldots, \Theta_d)$, and likelihood given by

$$p_{Y|\Theta}(y \mid \theta) = \frac{\prod_{i=1}^{d} \theta_i^{y_i}}{Z(\theta)} I\left\{ \sum_{i=1}^{d} y_i = V \right\},$$

where $Z(\theta)$ is a normalizing constant.
(a) Show that the likelihood forms an exponential family and identify the natural parameter.
(b) Determine a conjugate prior for the natural parameter.

Exercise 8 (Segmentation (ii))**.** In the previous exercise, the segmentation does not depend on the underlying image (which obviously is not a good model) and the volume is known. Let us generalize the model in this exercise to a setting where the segmentation depends on the underlying observed image, and the volume of the object is unknown. Let $(X^{(1)}, Y^{(1)}), \ldots, (X^{(N)}, Y^{(N)})$, $(X^{(N+1)}, Y^{(N+1)})$ be independent pairs of images and their segmentations. Suppose we observe $X = (X^{(1)}, Y^{(1)}, \ldots, X^{(N)}, Y^{(N)}, X^{(N+1)})$ and we are interested in predicting $Y = Y^{(N+1)}$. Suppose the likelihood is given by

$$p_{Y^{(k)}|X^{(k)},\Theta}(y \mid x^{(k)}, \theta) = \prod_{i=1}^{d} \mathrm{sigm}(y_i f_i(x^{(k)}, \theta)),$$

where $f(\cdot, \theta)$ is a function from the image to a vector in $\mathbb{R}^d$, parametrized by θ.
(a) Suppose $f_i(x, \theta) = \sum_{j=1}^{p} \theta_j \Phi_{ij}(x)$, $i = 1, \ldots, d$, for some feature function

$\Phi : \mathbb{R}^d \to \mathbb{R}^{d\times p}$. Identify a sufficient statistic for Θ.
(b) A decision rule is a binary vector $a \in \aleph = \{0,1\}^d$, where 1 means that the object occupies the ith pixel. Consider the accuracy loss function given by $L(y,a) = -\sum_{i=1}^d (2a_i - 1)(2y_i - 1)$. Let p_Θ be an arbitrary prior. Find a formal Bayes rule.

8.2. **Bayesian multi-class classification.** Recall Exercise 4 where we considered a multi-class classification problem, with classes $\{1,\dots,C\}$, and a pair (X,Y), where the class label Y may be represented as a *one-hot-vector*

$$Y = (Y^1,\dots,Y^C).$$

The objective is to construct a classifier that, given X, predicts the correct label. The action space is $\aleph = \{a_1,\dots,a_C\}$, where a_c is a one-hot-vector with a 1 in the cth component and loss function $L(y,a)$ can be taken to be $I\{y \neq a\}$. As a generalization of the binary classification problem, the formal Bayes rule will be determined by the predictive probabilities $p_{Y|X}(a_c \mid x)$, $c = 1,\dots,C$.

Suppose we have training data in the form of $\{(x_i,y_i)\}_{i=1}^n$ that are outcomes of independent pairs $\{(X_i,Y_i)\}_{i=1}^n$ where X_i represents explanatory variables and Y_i the corresponding labels. We also have $X_{n+1} = x_{n+1}$ and want to predict the label Y_{n+1}. In this case we could put $X = (X_1,Y_1,\dots,X_n,Y_n,X_{n+1})$ and $Y = Y_{n+1}$ and a formal Bayes rule is based on the predictive class probability:

$$p_{Y|X}(a_c \mid x) = p_{Y_{n+1}|X_1,Y_1,\dots,X_n,Y_n,X_{n+1}}(a_c \mid x_1,y_1,\dots,x_n,y_n,x_{n+1}).$$

What remains is to model the predictive distribution $p_{Y_i|X_i}$.

8.2.1. *A generalization of logistic regression.* Let us consider a generalization of the logistic regression to multi-class classification. For $t = (t_1,\dots,t_C) \in \mathbb{R}^C$ consider the *softmax* function as a generalization of the sigmoid:

$$\text{softmax}(t) = \left(\frac{e^{t_1}}{\sum_{c=1}^C e^{t_c}},\dots,\frac{e^{t_C}}{\sum_{c=1}^C e^{t_c}}\right)^{\mathrm{T}}.$$

Note that, for any $t \in \mathbb{R}^C$, softmax(t) defines a probability on $\{1,\dots,C\}$.

Consider parameters θ_{cj} and feature functions $\phi_{cj} : \mathcal{X} \to \mathbb{R}$, $c = 1,\dots,C$, $j = 1,\dots,d_c$. Let $\theta = (\theta_{cj})_{c=1,\dots,C,j=1,\dots d_c}$ and introduce $f_\theta(x) = (f_\theta^1(x),\dots,f_\theta^C(x))^{\mathrm{T}}$, where

$$f_\theta^c(x) = \sum_{j=1}^{d_c} \theta_{cj}\phi_{cj}(x).$$

Then we can model the conditional class probabilities as

$$\Pr(Y_i = a_c \mid \Theta = \theta, X_i = x_i) = \text{softmax}(f_\theta(x_i))_c = \frac{e^{f_\theta^c(x_i)}}{\sum_{c'=1}^C e^{f_\theta^{c'}(x_i)}}.$$

Now we can proceed just as in binary classification. With $X = (X_1, Y_1, \dots, X_n, Y_n, X_{n+1})$ and $Y = Y_{n+1}$ we have the likelihood

$$\begin{aligned}
p_{X|\Theta}(x \mid \theta) &= p_{Y_1,\dots,Y_n|\Theta,X_1,\dots,X_{n+1}}(y_1, \dots, y_n \mid \theta, x_1, \dots, x_{n+1}) p_{X_1,\dots,X_{n+1}}(x_1, \dots, x_{n+1}) \\
&= \prod_{i=1}^{n} \Big(\prod_{c=1}^{C} \operatorname{softmax}\big(f_\theta(x_i)\big)_c^{I\{y_i = a_c\}} \Big) \prod_{i=1}^{n+1} p_{X_i}(x_i) \\
&= \prod_{i=1}^{n} \Big(\prod_{c=1}^{C} \Big(\frac{e^{f_\theta^c(x_i)}}{\sum_{c'=1}^{C} e^{f_\theta^{c'}(x_i)}} \Big)^{I\{y_i = a_c\}} \Big) \prod_{i=1}^{n+1} p_{X_i}(x_i).
\end{aligned}$$

Note that, by the factorization theorem, a sufficient statistic for Θ is given by $\{(y_i, \phi(x_i))\}_{i=1}^n$, where $\phi(x) = (\phi_{cj}(x))$. Given a prior p_Θ, the predictive distribution is given by

$$\begin{aligned}
p_{Y|X}(y \mid x) &= \int_\Theta p_{Y|\Theta,X}(y \mid \theta, x) p_{\Theta|X}(\theta \mid x) d\theta \\
&= \int_\Theta p_{Y_{n+1}|\Theta,X_1,Y_1,\dots,X_n,Y_n,X_{n+1}}(y \mid \theta, x_1, y_1, \dots, x_n, y_n, x_{n+1}) \\
&\qquad \times p_{\Theta|X_1,Y_1,\dots,X_n,Y_n,X_{n+1}}(\theta \mid x_1, y_1, \dots, x_n, y_n, x_{n+1}) d\theta \\
&= \int_\Theta p_{Y_{n+1}|\Theta,X_{n+1}}(y \mid \theta, x_{n+1}) p_{\Theta|X_1,Y_1,\dots,X_n,Y_n,X_{n+1}}(\theta \mid x_1, y_1, \dots, x_n, y_n, x_{n+1}) d\theta \\
&= \int_\Theta \Big(\prod_{c=1}^{C} \operatorname{softmax}\big(f_\theta(x_{n+1})\big)_c^{I\{y = a_c\}} \Big) p_{\Theta|X_1,Y_1,\dots,X_n,Y_n}(\theta \mid x_1, y_1, \dots, x_n, y_n) d\theta
\end{aligned}$$

To evaluate the predictive class probabilities, we will resort to the methods mentioned above for binary classification.

8.3. **Gaussian processes for classification.** Here we give a very brief intro to Gaussian processes for classification. A more thorough introduction will be given in subsequent lectures.

We consider the binary classification problem. Suppose we have training data in the form of $\{(x_i, y_i)\}_{i=1}^n$ that are outcomes of independent pairs $\{(X_i, Y_i)\}_{i=1}^n$ where X_i represents the explanatory variables and Y_i the corresponding label. We also have $X_{n+1} mo = x_{n+1}$ and want to predict the label Y_{n+1}. We will represent the parameter Θ as a random function, and call it f. That is, f is a random function drawn from a probability on a space $\mathcal{F}$ of (smooth) functions. The conditional probabilities are modeled as

$$\begin{aligned}
\Pr(Y_i = 1 \mid f, X_i = x_i) &= \operatorname{sigm}\Big(f(x_i)\Big), \\
\Pr(Y_i = -1 \mid f, X_i = x_i) &= \operatorname{sigm}\Big(-f(x_i)\Big).
\end{aligned}$$

A simple example is when $\mathcal{F}$ consists of functions of the form $\sum_{j=1}^d \theta_j \phi_j(x)$ and the distribution on $\mathcal{F}$ is obtained from a prior distribution p_Θ. Then, the situation is reduced to Bayesian logistic regression. In general, we may consider richer classes of functions $\mathcal{F}$ and probabilities on them.

Gaussian processes are random functions f on $\mathcal{X}$ such that, for any finite collection of points $x_1, \dots, x_m$ the random vector $(f(x_1), \dots, f(x_m))^{\mathrm{T}}$ has a multivariate Gaussian distribution. To perform classification based on Gaussian processes we,

again, must evaluate the predictive class probabilities. With $Y = Y_{n+1}$ and $X = (X_1, Y_1, \ldots, X_n, Y_n, X_{n+1})$ we have

$$p_{Y|X}(y \mid x) = \mathrm{E}[p_{Y|f,X_{n+1}}(y \mid f(x_{n+1})) \mid X = x] = \mathrm{E}[\mathrm{sigm}(yf(x_{n+1})) \mid X = x]$$

where the expectation is taken with respect to the posterior distribution of f given $X = x$. Again, the predictive class probabilities cannot be evaluated explicitly, but we have to resort to approximation methods that will be studied in subsequent lectures.

We may also consider the Gaussian process framework for multi-class classification. It follows the same path as the generalization of logistic regression to multi-class classification we studied above.

8.4. **Bayesian model selection and maximum likelihood estimation.** Consider, again, the binary classification problem, where we have training data in the form of $\{(x_i, y_i)\}_{i=1}^n$ that are outcomes of independent pairs $\{(X_i, Y_i)\}_{i=1}^n$, and $X_{n+1} = x_{n+1}$. To predict the label Y_{n+1} we must evaluate the predictive distribution given by

$$p_{Y|X}(y \mid x) = \int_\Theta p_{Y|\Theta,X}(y \mid \theta, x) p_{\Theta|X}(\theta \mid x) d\theta,$$

where $Y = Y_{n+1}$ and $X = (X_1, Y_1, \ldots, X_n, Y_n, X_{n+1})$. How is the predictive distribution affected by the choice of prior, p_Θ? We may consider a range of different prior distributions and different hyperparameters for a given family of prior distributions. The choice of prior will clearly have a large impact on the decision, particularly if the sample size, n, of the training data is not excessively large.

The approach of Bayesian model selection is to optimize the hyperparameter (which may include optimizing over several families of prior distributions) by maximizing the probability of the data, given by the *evidence*

$$p_X(x) = \int_\Theta p_{X|\Theta}(x \mid \theta) p_\Theta(\theta) d\theta,$$

over the hyperparameters of p_Θ.

To be precise, let us denote the hyperparameter by Ψ and write $p_{\Theta|\Psi}(\theta \mid \psi)$ for the prior, given the hyperparameter. The evidence can then be expressed as a function of the hyperparameter,

$$p_{X|\Psi}(x \mid \psi) = \int_\Theta p_{X|\Theta,\Psi}(x \mid \theta, \psi) p_{\Theta|\Psi}(\theta \mid \psi) d\theta,$$

which is to be optimized over ψ.

One could, rather than optimizing over ψ, put a prior on Ψ, denoted p_Ψ, and consider the resulting predictive distribution

$$p_{Y|X}(y \mid x) = \int_\Psi \int_\Theta p_{Y|\Theta,\Psi,X}(y \mid \theta, \psi, x) p_{\Theta|\Psi,X}(\theta \mid \psi, x) p_{\Psi|X}(\psi \mid x) d\theta d\psi.$$

Then, one can optimize the hyperparameters of p_Ψ or consider yet another level, leading to an arbitrary sequence of nested Bayesian models.

In the other direction, we mentioned at the end of Section 8.1 that the predictive distribution can be approximated using the MAP, $\theta^{\mathrm{MAP}} = \mathrm{argmax}\, p_{\Theta|X}(\theta \mid x)$, by

$$p_{Y|X}(y \mid x) \approx p_{Y|X,\Theta,}(y \mid x, \theta^{\mathrm{MAP}}).$$

Going even further, we could ignore the prior completely and use the maximum likelihood estimate $\theta^{\mathrm{ML}} = \mathrm{argmax}_\theta p_{X|\Theta}(x \mid \theta)$ as the "true" value of θ and base classification decisions entirely on

$$p_{Y|X,\Theta}(y \mid x, \theta^{\mathrm{ML}}).$$

8.5. **Neural networks for classification.** Consider first the binary classification problem. Suppose we have training data in the form of $\{(x_i, y_i)\}_{i=1}^n$ that are outcomes of independent pairs $\{(X_i, Y_i)\}_{i=1}^n$ where X_i represents the explanatory variables and Y_i the corresponding label. We also have $X_{n+1} = x_{n+1}$ and want to predict the label Y_{n+1}.

The conditional probabilities are modeled as

$$\Pr(Y_i = 1 \mid \Theta = \theta, X_i = x_i) = \mathrm{sigm}\Big(f_\theta(x_i)\Big),$$
$$\Pr(Y_i = -1 \mid \Theta = \theta, X_i = x_i) = \mathrm{sigm}\Big(-f_\theta(x_i)\Big).$$

where $\mathcal{D} = \{f_\theta, \theta \in \Theta\}$ is a class of functions, parametrized by θ.

The functions f_θ in $\mathcal{D}$ are compositions of functions $f_{\theta_i}^{(i)}$, $i = 1, \ldots, m$, of the form

$$f_\theta(x) = f_{\theta_m}^{(m)} \circ \cdots \circ f_{\theta_1}^{(1)}(x) = f_{\theta_m}^{(m)}(f_{\theta_{m-1}}^{(m-1)}(\ldots f_{\theta_1}^{(1)}(x))). \tag{2}$$

We interpret each $f_{\theta_i}^{(i)}$ as the transition between hidden layers and $f_{\theta_i}^{(i)}$ has a simple form

$$f_{\theta_i}^{(i)}(h^{(i-1)}) = g^{(i)}(W^{(i)}h^{(i-1)} + b^{(i)}).$$

In this notation $h^{(i-1)}$ is a vector, the value at the $(i-1)$th layer, $h^{(i-1)} = f_{\theta_{i-1}}^{(i-1)} \circ \cdots \circ f_{\theta_1}^{(1)}(x)$, and $\theta_i = (W^{(i)}, b^{(i)})$ where $W^{(i)}$ is a matrix, $b^{(i)}$ is a vector, called the bias, and $g^{(i)}$ is an activation function applied to each component of $W^{(i)}h^{(i-1)} + b^{(i)}$.

To perform classification with neural networks we could put a prior p_Θ on the parameters and proceed with the Bayesian approach outlined above to compute the predictive class probabilities. However, often the prior is completely ignored and instead the maximum likelihood estimate $\theta^{\mathrm{ML}} = \mathrm{argmax}_\theta p_{X|\Theta}(x \mid \theta)$ is used as the "true" value of θ and classification decisions are based entirely on

$$p_{Y|X,\Theta}(y \mid x, \theta^{\mathrm{ML}}).$$

With $X = (X_1, Y_1, \ldots, X_n, Y_n, X_{n+1})$ the log-likelihood becomes

$$\begin{aligned}\log p_{X|\Theta}(x \mid \theta) &= \log p_{Y_1,\ldots,Y_n|\Theta,X_1,\ldots,X_{n+1}}(y_1, \ldots, y_n \mid \theta, x_1, \ldots, x_{n+1}) \\ &\quad + \log p_{X_1,\ldots,X_{n+1}}(x_1, \ldots, x_{n+1}) \\ &= \sum_{i=1}^n \log \mathrm{sigm}\Big(y_i f_\theta(x_i)\Big) + \sum_{i=1}^{n+1} \log p_{X_i}(x_i).\end{aligned}$$

Maximizing the log-likelihood is equivalent to minimizing the loss function

$$L(\{x_i, y_i\}_{i=1}^n; \theta\}) = -\sum_{i=1}^n \log \mathrm{sigm}(y_i f_\theta(x_i)),$$

over the parameters $\theta = \{(W^{(i)}, b^{(i)})\}_{i=1}^m$.

Lecture 5

9. An overview of methods for regression

9.1. **Bayesian regression.** Recall Exercise 5 where we considered predicting the next variable X_{n+1} based on observing $X_1, \dots, X_n$. We may consider this problem in a somewhat more general setting by considering a pair (X, Y) of random variables with $X \in \mathcal{X}$ and $Y \in \mathcal{Y}$, and construct a predictor that predicts Y given X.

Exercise 9 (Bayesian prediction II)**.** Suppose $\mathcal{Y} = \mathbb{R}^d$, $\aleph = \mathbb{R}^d$, and Y is square-integrable, and consider the squared loss

$$L(y, a) = |y - a|^2.$$

The objective is to predict Y given X. The posterior risk can be expressed as

$$r(\delta \mid x) = \mathrm{E}[|Y - \delta(X)|^2 \mid X = x].$$

Show that the predictive mean $\mathrm{E}[Y \mid X = x]$ is a formal Bayes rule.

With other loss functions, the formal Bayes rules may be different from the predictive mean. Here is an example where formal Bayes rules are quantiles of the predictive distribution.

Proposition 4. *Let (X, Y) be a pair of random variables with values in $\mathcal{X} \times \mathcal{Y}$, with $\mathcal{Y} \subset \mathbb{R}$. Let $p_{Y|X}$ denote the predictive distribution of Y given X. Suppose that $\mathrm{E}[Y \mid X = x] < \infty$, for each $x \in \mathcal{X}$. For $\aleph = \mathbb{R}$ and the loss function*

$$L(y, a) = \begin{cases} c(a - y), & \text{if } a \geq y, \\ (1 - c)(y - a), & \text{if } a < y, \end{cases}$$

where $c \in (0, 1)$, a formal Bayes rule is the $(1 - c)$-quantile of the predictive distribution of Y given $X = x$. That is,

$$\delta_c(x) = \min\{y : \Pr(Y \leq y \mid X = x) \geq 1 - c\},$$

is a formal Bayes rule.

Proof. Fix $x \in \mathcal{X}$. If $a > \delta_c(x)$, then

$$\begin{aligned} L(y, a) - L(y, \delta_c(x)) &= \begin{cases} c(a - \delta_c(x)), & \text{if } \delta_c(x) \geq y, \\ c(a - \delta_c(x)) - (y - \delta_c(x)), & \text{if } a \geq y > \delta_c(x), \\ (1 - c)(\delta_c(x) - a), & \text{if } y > a, \end{cases} \\ &= c(a - \delta_c(x)) + \begin{cases} 0, & \text{if } \delta_c(x) \geq y, \\ \delta_c(x) - y, & \text{if } a \geq y > \delta_c(x), \\ \delta_c(x) - a, & \text{if } y > a. \end{cases} \end{aligned}$$

The corresponding difference in posterior risk is

$$\begin{aligned} r(a \mid x) - r(\delta_c \mid x) &= c(a - \delta_c(x)) + \int_{\delta_c(x)}^{a} (\delta_c(x) - y) p_{Y|X}(y \mid x) dy \\ &\quad + (\delta_c(x) - a) \Pr(Y > a \mid X = x) \\ &\geq c(a - \delta_c(x)) + (\delta_c(x) - a) \Pr(Y > a \mid X = x) \\ &\quad + (\delta_c(x) - a) \Pr(Y \in (\delta_c(x), a] \mid X = x) \\ &= (a - \delta_c(x)) \left(c - \Pr(Y > \delta_c(x) \mid X = x)\right). \end{aligned}$$

Since $\Pr(Y > \delta_c(x) \mid X = x) \leq c$ it follows that $r(a \mid x) \geq r(\delta_c \mid x)$. A similar argument for $a < \delta_c(x)$ shows that

$$L(y,a) - L(y,\delta_c(x)) = c(a - \delta_c(x)) + \begin{cases} 0, & \text{if } a \geq y, \\ y - a, & \text{if } \delta_c(x) \geq y > a, \\ \delta_c(x) - a, & \text{if } y > \delta_c(x), \end{cases}$$

and consequently,

$$r(a \mid x) - r(\delta_c \mid x) \geq (\delta_c(x) - a)\left(\Pr(Y > a \mid X = x) - c\right).$$

Since $\Pr(Y > a \mid X = x) \geq \Pr(Y \geq \delta_c(x) \mid X = x) \geq c$, it follows that $r(a \mid x) \geq r(\delta_c \mid x)$. Since $x \in \mathcal{X}$ was arbitrary, this completes the proof. □

In the settings just described we have not specified what X is. For instance, suppose we have training data in the form of $\{(x_i, y_i)\}_{i=1}^n$ that are outcomes of independent pairs $\{(X_i, Y_i)\}_{i=1}^n$ where X_i represents explanatory variables and Y_i the corresponding response variables. We also have $X_{n+1} = x_{n+1}$ and want to predict the response Y_{n+1}. In this case we could put $X = (X_1, Y_1, \ldots, X_n, Y_n, X_{n+1})$ and $Y = Y_{n+1}$ and a formal Bayes rule is based on the predictive distribution (predictive mean if we use the squared loss):

$$p_{Y|X}(y \mid x) = p_{Y_{n+1}|X_1,Y_1,\ldots,X_n,Y_n,X_{n+1}}(y \mid x_1, y_1, \ldots, x_n, y_n, x_{n+1}).$$

What remains is to model the predictive distribution $p_{Y|X}$ and in the sequel we will consider a variety of models and methods for computing it.

9.1.1. *Bayesian linear regression.* Consider a linear regression model with additive noise given by

$$Y_i = f_\Theta(X_i) + \varepsilon_i,$$

where $\varepsilon = \{\varepsilon_i\}_{i \geq 1}$ is a sequence of centered IID random variables.

In linear models the function f_θ is given by a linear form

$$f_\theta(x) = \theta_0 + \sum_{j=1}^p \theta_j \phi_j(x) = \theta^{\mathrm{T}} \phi(x),$$

where $\phi = (1, \phi_1, \ldots, \phi_p)^{\mathrm{T}}$ are given basis functions $\phi_j : \mathcal{X} \to \mathbb{R}$, $j = 1, \ldots, p$, and $\theta = (\theta_0, \ldots, \theta_p)^{\mathrm{T}}$ are the regression coefficients.

Suppose that the training data consists of observations $(x_1, y_1), \ldots, (x_n, y_n)$ and the additive noise $\varepsilon = \{\varepsilon_i\}_{i \geq 1}$ is a sequence of IID random variables with probability density p_ε. Let p_Θ be a prior on Θ. With $X = (X_1, Y_1, \ldots, X_n, Y_n, X_{n+1})$ and $Y = Y_{n+1}$ we have the likelihood

$$\begin{aligned} p_{X|\Theta}(x \mid \theta) &= p_{Y_1,\ldots,Y_n|\Theta,X_1,\ldots,X_{n+1}}(y_1, \ldots, y_n \mid \theta, x_1, \ldots, x_{n+1}) p_{X_1,\ldots,X_{n+1}}(x_1, \ldots, x_{n+1}) \\ &= \prod_{i=1}^n p_\varepsilon(y_i - \theta^{\mathrm{T}} \phi(x_i)) \prod_{i=1}^{n+1} p_{X_i}(x_i). \end{aligned}$$

Note that, by the factorization theorem, a sufficient statistic for Θ is given by $\{(y_i, \phi(x_i))\}_{i=1}^n$.

Given a prior p_Θ, the predictive distribution is given by

$$\begin{aligned}
p_{Y|X}(y \mid x) &= \int_\Theta p_{Y|\Theta,X}(y \mid \theta, x) p_{\Theta|X}(\theta \mid x) d\theta \\
&= \int_\Theta p_{Y_{n+1}|\Theta,X_1,Y_1,\ldots,X_n,Y_n,X_{n+1}}(y \mid \theta, x_1, y_1, \ldots, x_n, y_n, x_{n+1}) \\
&\qquad \times p_{\Theta|X_1,Y_1,\ldots,X_n,Y_n,X_{n+1}}(\theta \mid x_1, y_1, \ldots, x_n, y_n, x_{n+1}) d\theta \\
&= \int_\Theta p_{Y_{n+1}|\Theta,X_{n+1}}(y \mid \theta, x_{n+1}) p_{\Theta|X_1,Y_1,\ldots,X_n,Y_n,X_{n+1}}(\theta \mid x_1, y_1, \ldots, x_n, y_n, x_{n+1}) d\theta \\
&= \int_\Theta p_{\varepsilon_{n+1}}(y_i - \theta^{\mathrm{T}} \phi(x_i)) p_{\Theta|X_1,Y_1,\ldots,X_n,Y_n}(\theta \mid x_1, y_1, \ldots, x_n, y_n) d\theta.
\end{aligned}$$

To predict the response associated with a new observation x_{n+1} we must be able to evaluate this predictive distribution (predictive mean if we have the squared loss).

Exercise 10. Consider the Bayesian linear regression model above in the one-dimensional case. That is, $\{(X_i, Y_i)\}_{i \geq 1}$ is an IID sequence of pairs of random variables with

$$Y_i = \Theta \phi(X_i) + \varepsilon_i, \quad i = 1, \ldots, n+1,$$

where $\{\varepsilon_i\}$ are IID with $N(0, \sigma^2)$ distribution and independent of $\{X_i\}$. Let the prior p_Θ be a $N(0, \tau^2)$ distribution and consider the squared loss $L(y, a) = (y - a)^2$. Determine a formal Bayes rule for predicting Y_{n+1} based on observing $X_1, Y_1, \ldots, X_n, Y_n, X_{n+1}$.

9.2. **Gaussian processes for regression.** Here we provide a brief introduction to Gaussian processes for regression. A more thorough introduction will be given in subsequent lectures.

A Gaussian process on $\mathcal{X}$ is a collection of random variables $\{f(x)\}_{x \in \mathcal{X}}$ such that, for any finite collection $(x_1, \ldots, x_d) \in \mathcal{X}^d$ the random vector $(f(x_1), \ldots, f(x_{d)})^{\mathrm{T}}$ has a multivariate normal distribution.

Example 17 (Bayesian linear regression)**.** Consider a Bayesian regression model where f has the form $f(x) = \theta^{\mathrm{T}} \phi(x)$, where $\phi(x) = (1, \phi_1(x), \ldots, \phi_p(x))^{\mathrm{T}}$ and $\theta = (\theta_0, \ldots, \theta_p)^{\mathrm{T}}$ with prior $\Theta \sim N(0, \Sigma)$. Then, f is a Gaussian process with mean function m and covariance function k given by

$$\begin{aligned}
m(x) &= \mathrm{E}[\Theta^{\mathrm{T}} \phi(x)] = \mathrm{E}[\Theta^{\mathrm{T}}] \phi(x) = 0, \\
k(x, x') &= \phi(x)^{\mathrm{T}} \, \mathrm{E}[\Theta \Theta^{\mathrm{T}}] \phi(x') = \phi(x)^{\mathrm{T}} \Sigma \phi(x').
\end{aligned}$$

Consider a regression problem with additive noise given by

$$Y_i = f(X_i) + \varepsilon_i,$$

where f is a Gaussian process and $\{\varepsilon_i\}_{i \geq 1}$ is independent Gaussian noise, independent of f. The Gaussian process is interpreted as a prior, $p_f(f)$, on the regression function f and, similar to the Bayesian regression model above, the predictive distribution is

given by

$$\begin{aligned}
p_{Y|X}(y \mid x) &= \int p_{Y|f,X}(y \mid f, x) p_{f|X}(f \mid x) df \\
&= \int p_{Y_{n+1}|f,X_1,Y_1,\ldots,X_n,Y_n,X_{n+1}}(y \mid f, x_1, y_1, \ldots, x_n, y_n, x_{n+1}) \\
&\qquad \times p_{f|X_1,Y_1,\ldots,X_n,Y_n,X_{n+1}}(f \mid x_1, y_1, \ldots, x_n, y_n, x_{n+1}) df \\
&= \int p_{Y_{n+1}|f,X_{n+1}}(y \mid f, x_{n+1}) p_{f|X_1,Y_1,\ldots,X_n,Y_n,X_{n+1}}(f \mid x_1, y_1, \ldots, x_n, y_n, x_{n+1}) df \\
&= \int p_{\varepsilon_{n+1}}(y_{n+1} - f(x_{n+1})) p_{f|X_1,Y_1,\ldots,X_n,Y_n}(f \mid x_1, y_1, \ldots, x_n, y_n) df,
\end{aligned}$$

where we have to make mathematical sense of the integration in the last display. With a Gaussian prior and a Gaussian likelihood we expect that the posterior $f \mid X$ is also Gaussian, and since p_ε is Gaussian, we further expect that the predictive distribution is Gaussian. The details of these arguments are postponed for now, but we can expect that Gaussian process regression actually leads to explicit formulas for the predictive distribution.

9.3. **Neural networks for regression.** Consider a regression problem with additive noise. Suppose we have training data in the form of $\{(x_i, y_i)\}_{i=1}^n$ that are outcomes of independent pairs $\{(X_i, Y_i)\}_{i=1}^n$ where X_i represents the explanatory variables and Y_i the response variable. We also have $X_{n+1} = x_{n+1}$ and want to predict the response Y_{n+1}.

The conditional probabilities are modeled as

$$Y_i = f_\theta(X_i) + \varepsilon_i.$$

where $\mathcal{D} = \{f_\theta, \theta \in \Theta\}$ is a class of functions, $f_\theta : \mathcal{X} \to \mathcal{Y}$, parametrized by θ.

The functions f_θ in $\mathcal{D}$ are compositions of functions $f_{\theta_i}^{(i)}$, $i = 1, \ldots, m$, of the form

$$f_\theta(x) = f_{\theta_m}^{(m)} \circ \cdots \circ f_{\theta_1}^{(1)}(x) = f_{\theta_m}^{(n)}(f^{(n-1)}(\ldots f^{(1)_{\theta_1}}(x))). \tag{3}$$

We interpret each $f_{\theta_i}^{(i)}$ as the transition between hidden layers and $f_{\theta_i}^{(i)}$ has a simple form

$$f_{\theta_i}^{(i)}(h^{(i-1)}) = g^{(i)}(W^{(i)} h^{(i-1)} + b^{(i)}).$$

In this notation $h^{(i-1)}$ is a vector, the value at the $(i-1)$th layer, $h^{(i-1)} = f_{\theta_{i-1}}^{(i-1)} \circ \cdots \circ f_{\theta_1}^{(1)}(x)$, and $\theta_i = (W^{(i)}, b^{(i)})$ where $W^{(i)}$ is a matrix, $b^{(i)}$ is a vector, called the bias, and $g^{(i)}$ is an activation function applied to each component of $W^{(i)} h^{(i-1)} + b^{(i)}$.

To perform regression with neural networks we could put a prior p_Θ on the parameters and proceed with the Bayesian approach outlined above to compute the predictive distribution (or predictive mean). However, often the prior is completely ignored and instead the maximum likelihood estimate $\theta^{\mathrm{ML}} = \mathrm{argmax}_\theta p_{X|\Theta}(x \mid \theta)$ is used as the "true" value of θ and prediction is based entirely on

$$p_{Y|X,\Theta}(y \mid x, \theta^{\mathrm{ML}}).$$

With $X = (X_1, Y_1, \ldots, X_n, Y_n, X_{n+1})$ the log-likelihood becomes

$$\begin{aligned}\log p_{X|\Theta}(x \mid \theta) &= \log p_{Y_1,\ldots,Y_n|\Theta,X_1,\ldots,X_{n+1}}(y_1, \ldots, y_n \mid \theta, x_1, \ldots, x_{n+1}) \\ &\quad + \log p_{X_1,\ldots,X_{n+1}}(x_1, \ldots, x_{n+1}) \\ &= \sum_{i=1}^{n} \log p_\varepsilon\Big(y_i - f_\theta(x_i)\Big) + \sum_{i=1}^{n+1} \log p_{X_i}(x_i).\end{aligned}$$

Maximizing the log-likelihood is equivalent to minimizing the loss function

$$L(\{x_i, y_i\}_{i=1}^n; \theta\}) = -\sum_{i=1}^{n} \log p_\varepsilon(y_i - f_\theta(x_i)),$$

over the parameters $\theta = \{(W^{(i)}, b^{(i)})\}_{i=1}^m$. For instance, if p_ε is the density of a Gaussian random variable with known variance σ^2, then the loss function becomes

$$L(\{x_i, y_i\}_{i=1}^n; \theta\}) = -\sum_{i=1}^{n} \log p_\varepsilon(y_i - f_\theta(x_i)) = \text{constant} + \frac{1}{\sigma^2} \sum_{i=1}^{n} (y_i - f_\theta(x_i))^2.$$

Minimizing this loss function is equivalent to minimizing the empirical squared loss.

Lecture 6

Literature:

- MacKay, Chapter 20.

10. Decision theory for clustering

Suppose we have observations of IID random variables $X_1, \ldots, X_n$ and we would like to cluster the observations into $C \geq 2$ clusters. That is, each observation will be assigned to one of the C clusters. We may view the clustering problem as a decision problem in the following way. Suppose $Y = (Y_1, \ldots, Y_n)^{\mathrm{T}}$ are unobserved class labels of $X_1, \ldots, X_n$. That is, Y_i takes values in $\{1, \ldots, C\}$. This resembles a standard multi-class classification problem, but here the class labels are unobserved. A decision rule δ is a function that maps $X = (X_1, \ldots, X_n)^{\mathrm{T}}$ to a vector $\delta(X) = (\delta_1(X), \ldots, \delta_n(X))^{\mathrm{T}} \in \aleph = \{1, \ldots, C\}^n$, where $\delta_k(X)$ is the cluster assignment of X_k. Note that δ_i depends on all the observations, not only X_k. The loss function $L(y, a)$ can be taken to be $n^{-1} \sum_{k=1}^n I\{y_k \neq a_k\}$. The posterior risk of a decision rule δ is given by

$$\begin{aligned} r(\delta \mid x) &= \frac{1}{n} \sum_{k=1}^n \mathrm{E}[I\{Y_k \neq \delta_k(X)\} \mid X = x] \\ &= \frac{1}{n} \sum_{k=1}^n \Pr(Y_k \neq \delta_k(X) \mid X = x) \\ &= \frac{1}{n} \sum_{k=1}^n 1 - \Pr(Y_k = \delta_k(X) \mid X = x) \\ &= 1 - \frac{1}{n} \sum_{k=1}^n p_{Y_k \mid X}(\delta_k(X) \mid x). \end{aligned}$$

A formal Bayes rule is given by $\delta_k(x) = \mathrm{argmax}_{c=1,\ldots,C} p_{Y_k \mid X}(c \mid x)$. It remains to introduce a model to be able to compute the predictive class probabilities. Note that, in contrast to a classification problem, the labels are unobserved, so it is insufficient to only specify the likelihood $p_{Y_k \mid X_k, \theta}$. Instead we introduce a model for the joint probabilities $p_{X_k, Y_k \mid \Theta}(x_k, y_k \mid \theta)$.

Consider a mixture model for (X_k, Y_k) of the form

$$p_{X_k, Y_k \mid \Theta}(x_k, y_k \mid \theta) = \psi_{y_k} p_{y_k}(x_k),$$

where p_c denotes the (c-conditional) density of the observations in the cth cluster, and θ denotes the parameters $\{\psi_c\}$ together with all the parameters in p_c, $c = 1, \ldots, C$. With the joint distribution specified it follows that the marginal distribution of X_k is given by

$$p_{X_k \mid \Theta}(x_k \mid \theta) = \sum_{c=1}^C \psi_c p_c(x_k),$$

and the conditional distribution is

$$p_{Y_k \mid X_k, \Theta}(y_k \mid x_k, \theta) = \frac{\psi_{y_k} p_{y_k}(x_k)}{\sum_{c=1}^C \psi_c p_c(x_k)}.$$

To perform clustering we need the predictive distribution $p_{Y \mid X}(y \mid x)$. We could introduce a prior on the parameters and consider a Bayesian approach, but here we

will aim for determining the maximum likelihood estimator θ^{ML} and base decisions on $p_{Y_k|X_k,\Theta}(y_k \mid x_k, \theta^{\mathrm{ML}})$.

Example 18 (Normal mixture)**.** The most popular approach to clustering with mixture models is to consider a normal mixture model for (X_k, Y_k) of the form

$$p_{X_k,Y_k|\Theta}(x_k, y_k \mid \psi, \mu) = \psi_{y_k}\varphi(x_k - \mu_{y_k}),$$

where φ denotes a standard normal density, $\psi = (\psi_1, \ldots, \psi_C)^{\mathrm{T}}$ the unconditional class probabilities, $\mu = (\mu_1, \ldots, \mu_C)^{\mathrm{T}}$ the cluster centers, and $\theta = (\psi, \mu)$. With the joint distribution specified it follows that the marginal distribution of X_k is given by

$$p_{X_k|\Theta}(x_k \mid \theta) = \sum_{c=1}^{C} \psi_c \varphi(x_k - \mu_c),$$

and the conditional distribution is

$$p_{Y_k|X_k,\Theta}(y_k \mid x_k, \theta) = \frac{\psi_{y_k}\varphi(x_k - \mu_{y_k})}{\sum_{c=1}^{C} \psi_c \varphi(x_k - \mu_c)}.$$

11. The EM-algorithm

The EM-algorithm (Expectation-Maximization algorithm) is an iterative procedure for computing the maximum likelihood estimator when only a subset of the data is available. The first proper theoretical study of the algorithm was done by Dempster, Laird, and Rubin (1977). The EM algorithm is extensively used throughout the statistics literature. We will give an overview on how it works and apply it to compute the maximum likelihood estimate in the mixture model for clustering.

Let $Z = (Z_1, \ldots, Z_n)$ be a sample with conditional density $p_{Z|\Theta}(z \mid \theta)$ given $\Theta = \theta$. We will write

$$l(\theta; Z) = \log p_{Z|\Theta}(Z \mid \theta)$$

for the log-likelihood function. The EM-algorithm is used when we do not have a complete data set of observations from Z. We will assume that the data Z consists of observed variables $X = (X_1, \ldots, X_{d_x})$ and unobserved (missing or latent variables) $Y = (Y_1, \ldots, Y_{d_z - d_x})$. We write $Z = (X, Y)$. With this notion the log-likelihood function for the observed data X is

$$l_{obs}(\theta; X) = \log \int p_{Z|\Theta}(X, y \mid \theta) dy.$$

The problem here is that when maximizing the likelihood we have to compute the integral and this can be difficult. We might not be able to find a closed-form expression for $l_{obs}(\theta; X)$. To maximize $l_{obs}(\theta; X)$ with respect to θ the idea is to do an iterative procedure where each iteration has two steps, called the *E-step* and the *M-step*. Let $\theta^{(i)}$ denote the estimate of Θ after the ith step. Then the two steps in the $(i+1)$th iteration are

E-step: Compute $Q(\theta \mid \theta^{(i)}) = \mathrm{E}_{\theta^{(i)}}[l(\theta; Z) \mid X]$.
M-step: Maximize $Q(\theta \mid \theta^{(i)})$ with respect to θ and put $\theta^{(i+1)} = \mathrm{argmax}_\theta Q(\theta \mid \theta^{(i)})$.

This procedure is iterated until it converges.

11.1. **The EM-algorithm for mixtures.** Let $p_c(\cdot)$, $c = 1, \dots, C$ be probability densities and suppose the random variable X_k is a mixture with weights $\psi_1, \dots, \psi_C$. That is, X_k has density

$$p_{X_k|\Psi}(x \mid \psi) = \sum_{c=1}^{C} \psi_c p_c(x_k),$$

where $\psi = (\psi_1, \dots, \psi_C) \in [0,1]^C$ and $\sum_{c=1}^{C} \psi_c = 1$. One can sample from X_k by first generating a variable $\tilde{Y}_k$ in $\{1, \dots, C\}$ according to the probabilities ψ_c, and if $\tilde{Y}_k = c$ you sample X_k from density p_c. Let Y_k be the one-hot-vector representation of $\tilde{Y}_k$. That is, $Y_k = (Y_{k1}, \dots, Y_{kC})^{\mathrm{T}}$ where $Y_{kc} = 1$ if $\tilde{Y}_k = c$ and $Y_{kc'} = 0$ for $c' \neq \tilde{Y}_k$.

Suppose we have observed a random sample of IID random variables, $X = (X_1, \dots, X_n)$ from the mixture. We want to estimate the parameters $(\psi_1, \dots, \psi_C)$. One can view this as a missing data problem where the missing data is taken to be $Y = (Y_1, \dots, Y_n)$. We denote the complete data by $Z = (X, Y)$.

The complete likelihood is then

$$p_{Z|\Psi}((x,y) \mid \psi) = \prod_{k=1}^{n} \prod_{c=1}^{C} \Big(\psi_c p_c(x_k)\Big)^{y_{kc}}$$

and the complete log-likelihood

$$\log p_{Z|\Psi}((x,y) \mid \psi) = \sum_{k=1}^{n} \sum_{c=1}^{C} y_{kc}[\log \psi_c + \log p_c(x_k)].$$

E-step: In the E-step we must compute

$$\begin{aligned} Q(\psi \mid \psi^{(i)}) &= \mathrm{E}_{\psi^{(i)}} \Big[\sum_{k=1}^{n} \sum_{c=1}^{C} Y_{kc}[\log \psi_c + \log p_c(X_k)] \;\Big|\; X = x \Big] \\ &= \sum_{k=1}^{n} \sum_{c=1}^{C} \mathrm{E}_{\psi^{(i)}}[Y_{kc} \mid X = x] \left(\log \psi_c + \log p_c(x_k)\right) \\ &\overset{c}{=} \sum_{k=1}^{n} \sum_{c=1}^{C} y_{kc}^{(i)} \log \psi_c, \end{aligned}$$

with $y_{kc}^{(i)} = \mathrm{E}_{\psi^{(i)}}[Y_{kc} \mid X = x]$. This can actually be computed as

$$y_{kc}^{(i)} = \mathrm{E}_{\psi^{(i)}}[Y_{kc} \mid X = x] = \frac{\psi_c^{(i)} p_c(x_k)}{\sum_{c'=1}^{C} \psi_{c'}^{(i)} p_{c'}(x_k)}. \tag{4}$$

$y_{kc}^{(i)}$ has interpretation as the current (at iteration i) posterior probability that the kth observation arises from the cth component in the mixture.

M-step: To execute the M-step, we maximize $Q(\psi \mid \psi^{(i)})$ w.r.t. ψ and find an expression for the maximizer. The maximizer is just as if Y was observed, but with y_{kc} replaced by $y_{kc}^{(i)}$. Hence, we get

$$\psi_c^{(i+1)} = \frac{1}{n} \sum_{k=1}^{n} y_{kc}^{(i)}.$$

By iterating the E-step and the M-step, the EM-algorithm, with given $p_1, \dots, p_C$, can be summarized by the iterative updates: initiate $\psi^{(0)}$, for $i \geq 0$,

(i) put $y_{kc}^{(i)} = \psi_c^{(i)} p_c(x_k) / \Big(\sum_{c=1}^C \psi_c^{(i)} p_c(x_k) \Big)$,

(ii) update $\psi_c^{(i+1)} = \frac{1}{n} \sum_{k=1}^n y_{kc}^{(i)}$.

11.2. **EM-algorithm for normal mixtures.** Suppose that the densities p_c are $N(\mu_c, \sigma_c^2)$ but the parameters are unknown. Then the density for the data X_k is given by

$$p_{X_k|\Theta}(x_k \mid \theta) = \sum_{c=1}^C \psi_c p_c(x_k \mid \mu_c, \sigma_c),$$

where $\theta = (\psi, \mu, \sigma) = (\psi_1, \dots, \psi_C, \mu_1, \dots, \mu_C, \sigma_1, \dots, \sigma_C)^{\mathrm{T}}$ and $p_c(x_k \mid \mu_c, \sigma_c)$ is a $N(\mu_c, \sigma_c^2)$-density.

The complete likelihood is then

$$p_{Z|\Theta}((x, y) \mid \theta) = \prod_{k=1}^n \prod_{c=1}^C \Big(\psi_c p_c(x_k \mid \mu_c, \sigma_c) \Big)^{y_{kc}},$$

and the complete log-likelihood

$$\log p_{Z|\Theta}((x, y) \mid \theta) = \sum_{k=1}^n \sum_{c=1}^C y_{kc} [\log \psi_c + \log p_c(x_k \mid \mu_c, \sigma_c)].$$

E-step: The function $Q(\theta \mid \theta^{(i)})$ used in the E-step of the EM algorithm is given by

$$\begin{aligned} Q(\theta \mid \theta^{(i)}) &= \mathrm{E}_{\theta^{(i)}} \Big[\sum_{k=1}^n \sum_{c=1}^C Y_{kc} [\log \psi_c + \log p_c(X_k \mid \mu_c, \sigma_c)] \;\Big|\; X = x \Big] \\ &= \sum_{k=1}^n \sum_{c=1}^C \mathrm{E}_{\theta^{(i)}} [Y_{kc} \mid X = x] [\log \psi_c + \log p_c(x_k \mid \mu_c, \sigma_c)] \\ &= \sum_{k=1}^n \sum_{c=1}^C y_{kc}^{(i)} [\log \psi_c + \log p_c(x_k \mid \mu_c, \sigma_c)], \end{aligned}$$

with $y_{kc}^{(i)} = \mathrm{E}_{\theta^{(i)}} [Y_{kc} \mid X = x]$, which can be computed as

$$y_{kc}^{(i)} = \mathrm{E}_{\theta^{(i)}} [Y_{kc} \mid X = x] = \frac{\psi_c^{(i)} p_c(x_k \mid \mu_c^{(i)}, \sigma_c^{(i)})}{\sum_{c'=1}^C \psi_{c'}^{(i)} p_{c'}(x_k \mid \mu_{c'}^{(i)}, \sigma_{c'}^{(i)})}. \tag{5}$$

$y_{kc}^{(i)}$ has interpretation as the current (at iteration i) posterior probability that the kth observation arises from the cth component in the mixture.

M-step: To execute the M-step, we maximize $Q(\theta \mid \theta^{(i)})$ w.r.t. θ and find an expression for the maximizer. The maximizer is just as if Y is observed, but y_{kc}

replaced by $y_{kc}^{(i)}$. Performing the optimization, we get

$$\psi_c^{(i+1)} = \frac{1}{n}\sum_{k=1}^{n} y_{kc}^{(i)},$$
$$\mu_c^{(i+1)} = \frac{\sum_{k=1}^n y_{kc}^{(i)} x_k}{\sum_{k=1}^n y_{kc}^{(i)}},$$
$$(\sigma_c^2)^{(i+1)} = \frac{\sum_{k=1}^n y_{kc}^{(i)} (x_k - \mu_c^{(i+1)})^2}{\sum_{k=1}^n y_{kc}^{(i)}}.$$

Remark 1. If p_c is the density of a normal distribution with known (fixed) variance parameter $\sigma_c^2 = 1/\beta$, $c = 1, \dots, C$, $\beta > 0$, then the EM-algorithm for the normal mixture corresponds to the soft K-means algorithm (see MacKay, Sec. 20.2).

Exercise 11. Redo the computations of the above two EM-algorithms without the one-hot representation for Y_k, i.e. with $Y_k = \tilde{Y}_k \in \{1, \dots, C\}$.

11.3. **Clustering with normal mixtures and feature functions.** Recall the clustering algorithm based on the EM-algorithm for normal mixtures in the previous section. Let us assume that the clustering is based on a feature vector representation of the data, $\phi(x) = (\phi_1(x), \dots, \phi_m(x))^{\mathrm{T}}$ for given feature functions $\phi_1, \dots, \phi_m$. Then, the algorithm proceeds similarly, with x_k replaced by $\phi(x_k)$.

(i) *Assignment step.* Assign each data point $\phi(x_k)$ to class c using

$$y_{kc}^{(i)} = \frac{\psi_c^{(i)} p_c(\phi(x_k) \mid \mu_c^{(i)}, \sigma_c^{(i)})}{\sum_{c'=1}^C \psi_{c'}^{(i)} p_{c'}(\phi(x_k) \mid \mu_{c'}^{(i)}, \sigma_{c'}^{(i)})}. \tag{6}$$

(ii) *Update step.* Update the parameters by

$$\psi_c^{(i+1)} = \frac{1}{n}\sum_{k=1}^{n} y_{kc}^{(i)},$$
$$\mu_c^{(i+1)} = \frac{\sum_{k=1}^n y_{kc}^{(i)} \phi(x_k)}{\sum_{k=1}^n y_{kc}^{(i)}}$$
$$(\sigma_c^2)^{(i+1)} = \frac{\sum_{k=1}^n y_{kc}^{(i)} (\phi(x_k) - \mu_c^{(i+1)})^2}{\sum_{k=1}^n y_{kc}^{(i)}}.$$

11.4. **Convergence of the EM-algorithm.** To give a heuristic explanation of why the EM-algorithm converges, one can argue as follows. First note that what we want originally is to maximize the log-likelihood $l_{obs}(\theta; X)$ of the observations. For this we want to find a $\hat{\theta}$ which solves $\partial/\partial_\theta l_{obs}(\theta; X)|_{\theta=\hat{\theta}} = 0$. We will show that if $\theta^{(i)}$ coming from the algorithm converges to some θ^* then this θ^* satisfies $\partial/\partial_\theta l_{obs}(\theta; X)|_{\theta=\theta^*} = 0$, which is exactly what we want.

To see this, note that the complete likelihood is, letting $z = (x, y)$,

$$l(\theta; z) = \log p_{Z|\Theta}(z \mid \theta) = \log p_{Y|X,\Theta}(y \mid x, \theta) + \log p_{X|\Theta}(x \mid \theta).$$

Then

$$Q(\theta \mid \theta^{(i)}) = \int \log p_{Y|X,\Theta}(y \mid x, \theta) p_{Y|X,\Theta}(y \mid x, \theta^{(i)}) dy + l_{obs}(\theta; X).$$

Differentiating w.r.t. θ and putting equal to 0 in order to maximize Q gives

$$0 = (\partial/\partial_\theta) Q(\theta \mid \theta^{(i)}) = \int \frac{\partial_{\theta_j} p_{Y|X,\Theta}(y \mid x, \theta)}{p_{Y|X,\Theta}(y \mid x, \theta)} p_{Y|X,\Theta}(y \mid x, \theta^{(i)}) dy + (\partial/\partial_\theta) l_{obs}(\theta; X).$$

If $\theta^{(i)} \to \theta^*$ then we have for θ^* that (with $j = 1, \ldots, k$)

$$\begin{aligned}
0 &= \partial_{\theta_j} Q(\theta \mid \theta^*)\big|_{\theta=\theta^*} \\
&= \int \frac{\partial_{\theta_j} p_{Y|X,\Theta}(y \mid x, \theta^*)}{p_{Y|X,\Theta}(y \mid x, \theta^*)} p_{Y|X,\Theta}(y \mid x, \theta^*) dy + \partial_{\theta_j} l_{obs}(\theta^*; X) \\
&= \partial_{\theta_j} \int p_{Y|X,\Theta}(y \mid x, \theta^*) dy + \partial_{\theta_j} l_{obs}(\theta^*; X) \\
&= \partial_{\theta_j} l_{obs}(\theta^*; X).
\end{aligned}$$

Hence, θ^* satisfies the desired equation. To actually prove that the EM-algorithm works, we need to show that the sequence $\theta^{(i)}$ actually converges to some θ^*. This is usually done in two steps.

In the first step, we show that the sequence $l_{obs}(\theta^{(i)}; X)$ is non-decreasing. If it is also bounded, then it converges. It should be noted that it may converge to a local maximum, and not necessarily to a global maximum. In practice one would typically choose different initial values $\theta^{(0)}$ to see if there seems to be several local maximima. In the second step we show that $l_{obs}(\theta^{(i)}; X) \to l^*$ implies $\theta^{(i)} \to \theta^*$. This second step requires stronger assumptions than the first step.

Proposition 5. *The sequence $l_{obs}(\theta^{(i)}; X)$ in the EM-algorithm is non-decreasing.*

Proof. We write $Z = (X, Y)$ for the complete data, with X the observed data and Y the unobserved data. Then

$$p_{Y|X,\Theta}(Y \mid X, \theta) = \frac{p_{Z|\Theta}((X, Y) \mid \theta)}{p_{X|\Theta}(X \mid \theta)}.$$

Hence,

$$l_{obs}(\theta; X) = \log p_{X|\Theta}(X \mid \theta) = \log p_{Z|\Theta}((X, Y) \mid \theta) - \log p_{Y|X,\Theta}(Y \mid X, \theta).$$

Taking conditional expectation given Y and $\Theta = \theta^{(i)}$ on both sides yields

$$\begin{aligned}
l_{obs}(\theta; X) &= \mathrm{E}_{\theta^{(i)}}[l_{obs}(\theta; X) \mid X] \\
&= \mathrm{E}_{\theta^{(i)}}[\log p_{Z|\Theta}((X, Y) \mid \theta) \mid X] - \mathrm{E}_{\theta^{(i)}}[\log p_{Y|X,\Theta}(Y \mid X, \theta) \mid X] \\
&= Q(\theta \mid \theta^{(i)}) - H(\theta \mid \theta^{(i)}),
\end{aligned}$$

with

$$H(\theta \mid \theta^{(i)}) = \mathrm{E}_{\theta^{(i)}}[\log p_{Y|X,\Theta}(Y \mid X, \theta) \mid X].$$

Then we have

$$l_{obs}(\theta^{(i+1)}; X) - l_{obs}(\theta^{(i)}; X) = [Q(\theta^{(i+1)} \mid \theta^{(i)}) - Q(\theta^{(i)} \mid \theta^{(i)})] - [H(\theta^{(i+1)} \mid \theta^{(i)}) - H(\theta^{(i)} \mid \theta^{(i)})].$$

Since $\theta^{(i+1)}$ maximizes $Q(\theta \mid \theta^{(i)})$ the first term is nonnegative. For the second term we have, for any θ, by Jensen's inequality for concave functions (log is concave)

$$\begin{aligned} H(\theta \mid \theta^{(i)}) - H(\theta^{(i)} \mid \theta^{(i)}) &= \mathrm{E}_{\theta^{(i)}}\left[\log\left(\frac{p_{Y|X,\Theta}(Y \mid X, \theta)}{p_{Y|X,\Theta}(Y \mid X, \theta^{(i)})}\right) \,\middle|\, X\right] \\ &\leq \log \mathrm{E}_{\theta^{(i)}}\left[\frac{p_{Y|X,\Theta}(Y \mid X, \theta)}{p_{Y|X,\Theta}(Y \mid X, \theta^{(i)})} \,\middle|\, X\right] \\ &= \log \int \frac{p_{Y|X,\Theta}(y \mid X, \theta)}{p_{Y|X,\Theta}(y \mid X, \theta^{(i)})} p_{Y|X,\Theta}(y \mid X, \theta^{(i)}) dy \\ &= \log 1 = 0. \end{aligned}$$

This shows $l_{obs}(\theta^{(i+1)}; X) - l_{obs}(\theta^{(i)}; X) \geq 0$. □

This proposition is the main ingredient in the following theorem due to Wu (1983). We also need the following assumptions:

- Ω is a subset of $\mathbb{R}^k$.
- $\Omega_{\theta_o} = \{\theta \in \Omega : l(\theta; x) \geq l(\theta_o; x)\}$ is compact for any $l(\theta_o; x) > -\infty$.
- $l(\theta_o; x)$ is continuous and differentiable in the interior of Ω.

Theorem 7. *Suppose that $Q(\theta \mid \phi)$ is continuous in both θ and ϕ. Then all limit points of any instance $\{\theta^{(i)}\}$ of the EM algorithm are stationary points, i.e. $\theta^* = \mathrm{argmax}_\theta\, Q(\theta \mid \theta^*)$, and $l(\theta^{(i)}; y)$ converges monotonically to some value $l^* = l(\theta^*; y)$ for some stationary point θ^*.*

To show that $\theta^{(i)}$ converges to some θ^* we can use the next theorem.

Theorem 8. *Assume the hypothesis of Theorem 7. Suppose in addition that $\partial_\theta Q(\theta \mid \phi)$ is continuous in θ and ϕ. Then $\theta^{(i)}$ converges to a stationary point θ^* with $l(\theta^*; x) = l^*$, the limit of $l(\theta^{(i)}; x)$ if either*

$$\{\theta : l(\theta; x) = l^*\} = \{\theta^*\}$$

or $|\theta^{(i+1)} - \theta^{(i)}| \to 0$ and $\{\theta : l(\theta; x) = l^*\}$ is discrete.

In the case when the likelihood is unimodal everything simplifies.

Corollary 1. *Suppose $l(\theta; x)$ is unimodal in Ω with θ^* being the only stationary point and $\partial_\theta Q(\theta \mid \phi)$ is continuous in θ and ϕ. Then $\theta^{(i)}$ converges to the unique maximizer θ^* of $l(\theta; x)$ which is the MLE.*

Lecture 7

Literature:

- Kushner and Yin, *Stochastic Approximation and Recursive Algorithms and Applications*, Springer, Chapter 1.

12. Banach's fixed-point theorem

Let Θ be a complete separable metric space, equipped with a metric d. If you are unfamiliar with such spaces you may consider $\Theta = \mathbb{R}^n$ and the Euclidean metric $d(\theta, \psi) = |\theta - \psi|$. A mapping $T : \Theta \to \Theta$ is a *contraction* if there exists $\gamma \in [0, 1)$ such that

$$d(T\theta, T\psi) \leq \gamma d(\theta, \psi), \qquad \text{for every } \theta, \psi \in \Theta.$$

Note that a contraction T is continuous. Indeed, if $\theta_n \to \bar{\theta}$, then

$$d(T\theta_n, T\bar{\theta}) \leq \gamma d(\theta_n, \bar{\theta}) \to 0.$$

A sequence $\{\theta_n\}$ is a *Cauchy sequence* if, for every $\varepsilon > 0$, there exists n_0 such that $n, m \geq n_0$ implies $d(\theta_n, \theta_m) < \varepsilon$. In a complete separable metric space, Θ, every Cauchy sequence converges to a limit in Θ.

Theorem 9 (Banach's fixed-point theorem)**.** *Let $T : \Theta \to \Theta$ be a contraction. Take $\theta_0 \in \Theta$ and $\theta_{n+1} = T\theta_n$, $n \geq 0$. Then θ_n converges to some $\bar{\theta} \in \Theta$, which is the unique solution to the fixed point equation $\theta = T\theta$.*

Proof. First we prove that the fixed point is unique. Suppose $\bar{\theta}, \hat{\theta} \in \Theta$ both satisfy $\theta = T\theta$. Then

$$d(\bar{\theta}, \hat{\theta}) = d(T\bar{\theta}, T\hat{\theta}) \leq \gamma d(\bar{\theta}, \hat{\theta}), \quad \gamma \in (0, 1).$$

Hence, $d(\bar{\theta}, \hat{\theta}) = 0$.

Next we show that $\{\theta_n\}$ is a Cauchy sequence. Note that

$$\begin{aligned} d(\theta_{n+1}, \theta_n) &= d(T\theta_n, T\theta_{n-1}) \\ &\leq \gamma d(\theta_n, \theta_{n-1}) \\ &\leq \cdots \leq \gamma^n d(\theta_1, \theta_0). \end{aligned}$$

It follows from the triangle inequality that, for any $m \geq 0$,

$$\begin{aligned} d(\theta_{n+m}, \theta_n) &\leq d(\theta_{n+m}, \theta_{n+m-1}) + \cdots + d(\theta_{n+1}, \theta_n) \\ &\leq \left(\gamma^{n+m-1} + \cdots + \gamma^n\right) d(\theta_1, \theta_0) \\ &\leq \gamma^n(\gamma^{m-1} + \cdots + 1) d(\theta_1, \theta_0) \\ &\leq \frac{\gamma^n}{1-\gamma} d(\theta_1, \theta_0) \to 0, \end{aligned}$$

as $n \to \infty$.

To complete the proof we must show that $\bar{\theta} = \lim_{n\to\infty} \theta_n$ is a solution to the fixed point equation. Indeed,

$$\bar{\theta} = \lim_{n\to\infty} \theta_{n+1} = \lim_{n\to\infty} T\theta_n = T\bar{\theta},$$

where the last equality follows from continuity of T. □

Example 19 (Finding the root of a function)**.** Consider a strictly decreasing function $g : \mathbb{R} \to \mathbb{R}$ with a unique root at $\bar{\theta}$. That is, $g(\bar{\theta}) = 0$. To numerically find the root, Newton's method can be applied (if g is differentiable). It starts with an initial value θ_0 and proceeds by updating

$$\theta_{n+1} = \theta_n - \frac{1}{\partial_\theta g(\theta_n)} g(\theta_n).$$

A simpler algorithm is to take a fixed step-size $\epsilon > 0$ and put

$$\theta_{n+1} = \theta_n + \epsilon g(\theta_n).$$

The latter avoids calculating the gradient, but generally converges slower. By defining $T_\epsilon : \mathbb{R} \to \mathbb{R}$ as

$$T_\epsilon \theta = \theta + \epsilon g(\theta),$$

we see that if g is differentiable and if the derivative is bounded away from zero and negative infinity, i.e. $\partial_\theta g(\theta) \in [-M, -m]$ for all θ, where $0 < m \leq M < \infty$, then T_ϵ is a contraction for sufficiently small ϵ. Indeed, for any θ, ψ, assume $\theta > \psi$ which implies that $g(\psi) > g(\theta)$. We get:

$$\begin{aligned} T_\epsilon \theta - T_\epsilon \psi &= \theta - \psi - \epsilon(g(\psi) - g(\theta)) \leq (1 - \epsilon m)(\theta - \psi), \quad \text{and,} \\ T_\epsilon \theta - T_\epsilon \psi &= \theta - \psi - \epsilon(g(\psi) - g(\theta)) \geq (1 - \epsilon M)(\theta - \psi), \end{aligned}$$

where we have used the mean value theorem to get that $m \leq \frac{g(\psi)-g(\theta)}{\theta-\psi} \leq M$. Taking $\epsilon \in (0, \frac{1}{M}]$, we get $|T_\epsilon \theta - T_\epsilon \psi| \leq (1 - \epsilon m)|\theta - \psi|$. This is a contraction with $\gamma = 1 - \epsilon m < 1$.

Consequently, θ_n converges to the fixed-point, which is the solution to the equation $\theta = T_\epsilon \theta = \theta + \epsilon g(\theta)$. That is, θ_n converges to the root $\bar{\theta}$, satisfying $g(\bar{\theta}) = 0$.

13. Robbins-Monro algorithm and stochastic approximation

Let us consider a generalization of Example 19 where $g : \mathbb{R}^n \to \mathbb{R}^n$ cannot be directly evaluated. At any θ_n we may only observe a random variable Y_n whose expectation is $g(\theta_n)$. It is tempting to take the sample average of many independent copies of such Y_n to get a precise estimate of $g(\theta_n)$, but Robbins-Monro realized that it is sufficient to take a single observation Y_n at each update. As long as the stepsize decreases appropriately fast the noise will average out and the algorithm will converge. The Robbins-Monro updates are given by

$$\theta_{n+1} = \theta_n + \epsilon_n Y_n, \quad n \geq 0,$$

where the stepsize ϵ_n satisfies

$$\epsilon_n > 0, \quad \epsilon_n \to 0, \quad \sum_n \epsilon_n = \infty, \quad \sum_n \epsilon_n^2 < \infty.$$

Example 20 (The sample mean)**.** Let $X_1, X_2, \ldots$ be IID random variables with $\mathrm{E}[X_n] = \bar{\theta}$. The sample mean $\theta_n = \frac{1}{n}\sum_{i=1}^n X_n$ can be viewed as Robbins-Monro algorithm. Indeed, write

$$\theta_{n+1} = \frac{1}{n+1}\sum_{i=1}^{n+1} X_i = \frac{n}{n+1}\theta_n + \frac{1}{n+1} X_{n+1} = \theta_n + \frac{1}{n+1}(X_{n+1} - \theta_n).$$

This is a Robbins-Monro algorithm with $\epsilon_n = (n+1)^{-1}$ and $g(\theta_n) = \bar{\theta} - \theta_n$.

13.1. **Finding zeros/solutions of an unknown function.** Let us again consider a generalization of Example 19 where we can sample from an unknown probability density $p_{X|\Theta}(y \mid \theta)$. Let us put $m(\theta) = \mathrm{E}[X \mid \Theta = \theta] = \int x p_{X|\Theta}(x \mid \theta) dx$. We are interested in finding $\bar{\theta}$ such that $m(\bar{\theta}) = \bar{m}$, for a given $\bar{m}$. Let us assume that $m(\theta)$ is strictly decreasing. At each θ_n a sample X_n is taken from $p_{X|\Theta}(\cdot \mid \theta_n)$ and the algorithm updates

$$\theta_{n+1} = \theta_n + \epsilon_n(X_n - \bar{m}).$$

Let us rewrite the update as

$$\theta_{n+1} = \underbrace{\theta_n + \epsilon_n(m(\theta_n) - \bar{m})}_{\text{noise free update}} + \underbrace{\epsilon_n(X_n - m(\theta_n))}_{\text{noise}}.$$

If there were no noise, we would be back in the situation considered in Example 19. Now, suppose (in a slightly more general setting) that X_n are such that

$$\mathrm{E}[X_n - m(\theta_n) \mid X_0, \dots, X_{n-1}, \theta_0, \dots, \theta_n] = 0.$$

We will call $\delta M_n = X_n - m(\theta_n)$ a *martingale difference sequence*. Note that the elements of such a sequence are uncorrelated. Indeed,

$$\begin{aligned}\mathrm{E}[\delta M_n \delta M_{n-1}] &= \mathrm{E}[\mathrm{E}[\delta M_n \delta M_{n-1} \mid X_0, \dots, X_{n-1}, \theta_0, \dots, \theta_n]] \\ &= \mathrm{E}[\delta M_{n-1} \, \mathrm{E}[\delta M_n \mid X_0, \dots, X_{n-1}, \theta_0, \dots, \theta_n]] = 0.\end{aligned}$$

Then, we can argue that the noise over many updates aggregates to approximately 0. To see this, take a small $\Delta > 0$ and let m_n^Δ be such that

$$\sum_{i=n}^{n+m_n^\Delta - 1} \epsilon_i \approx \Delta.$$

Then,

$$\theta_{n+m_n^\Delta} - \theta_n \approx \Delta(m(\theta_n) - \bar{m}) + \sum_{i=n}^{n+m_n^\Delta - 1} \epsilon_i \delta M_i.$$

The last term can be viewed as an error term and has zero mean and variance:

$$\mathrm{Var}\left(\sum_{i=n}^{n+m_n^\Delta - 1} \epsilon_i \delta M_i \right) = \sum_{i=n}^{n+m_n^\Delta - 1} \mathrm{Var}(\epsilon_i \delta M_i) = \sum_{i=n}^{n+m_n^\Delta - 1} \epsilon_i^2 \mathrm{Var}(\delta M_i).$$

Assuming $\mathrm{Var}(\delta M_i) \le \sigma^2 < \infty$ for each i, it follows that the term in the last display can be bounded from above by

$$\sigma^2 \sum_{i=n}^{n+m_n^\Delta - 1} \epsilon_i^2 \le \sigma^2 \epsilon_n \sum_{i=n}^{n+m_n^\Delta - 1} \epsilon_i \approx \sigma^2 \epsilon_n \Delta.$$

We can conclude that the noise term vanishes asymptotically, as $n \to \infty$.

Lecture 8

Literature:

- Kushner and Yin, *Stochastic Approximation and Recursive Algorithms and Applications*, Springer, 2003, Chapter 4 and 5.

14. Limit ODEs and Lyapunov functions

Let us consider the Robbins-Monro updates, and rewrite them as follows

$$\theta_{n+1} = \theta_n + \epsilon_n X_n = \theta_n + \epsilon_n(g(\theta_n) + \delta M_n),$$

where $g(\theta_n) = \mathrm{E}[X_n \mid \theta_n]$ (taking $\bar{\theta} = 0$) and $\delta M_n = X_n - \mathrm{E}[X_n \mid \theta_n]$. Here it is assumed that the distribution of X_n depends only on θ_n and not on other variables observable at time n, but it is possible to have more general settings where, for example, X_n is sampled from a Markov chain whose transition probability depends on θ_n.

As in Section 13.1, one can argue that the noise will vanish in the limit. Therefore we proceed by assuming that the noise $\delta M_n = 0$ and consider the deterministic iteration

$$\theta_{n+1} = \theta_n + \epsilon_n g(\theta_n). \tag{7}$$

We will let $n \to \infty$ and investigate what happens with θ_n. Let us first rewrite the iteration in the form

$$\frac{\theta_{n+1} - \theta_n}{\epsilon_n} = g(\theta_n).$$

When ϵ_n is small, the left-hand side may be interpreted as a discrete approximation to a derivative. By letting $n \to \infty$ the iteration behaves as the solution, at large t, of the ordinary differential equation (ODE) given by

$$\frac{d\theta(t)}{dt} = g(\theta(t)). \tag{8}$$

We call (8) the limit ODE of the Robbins-Monro algorithm. For large values of t one can expect that $\theta(t)$ stabilizes at values of θ where $g(\theta) = 0$. Indeed, at such θ, $d\theta(t)/dt = 0$ so $\theta(t)$ remains constant. Consequently, we can expect that the iteration (7) eventually converges to some point where $g(\theta) = 0$.

To study the convergence to the limit ODE for the noise-free updates, define $t_0 = 0$, $t_n = \sum_{i=0}^{n-1} \epsilon_i$, and $m(t)$ as the unique value of n such that $t_n \le t < t_{n+1}$. A continuous time interpolation of $\{\theta_n\}$ is given by $\theta^0(t) = \theta_0$ for $t \le 0$ and for $t > 0$,

$$\theta^0(t) = \theta_n, \quad \text{for } t_n \le t < t_{n+1},$$

and a shifted process $\theta^n(t)$ is given by

$$\theta^n(t) = \theta^0(t_n + t), \quad t \in (-\infty, \infty).$$

Note that θ^n is piecewise constant, but one could just as well consider a piecewise linear interpolation.

The convergence of the sequence of functions $\{\theta^n(t)\}$ relies on showing that the the sequence is equicontinuous in the extended sense, which means that the set $\{\theta^n(0)\}$ is bounded and for each $T > 0$ and $\epsilon > 0$ there is a $\delta > 0$ such that

$$\limsup_n \sup_{0 \le t-s \le \delta, |t| \le T} |\theta^n(t) - \theta^n(s)| \le \epsilon.$$

Then, by an extension of the Arzela-Ascoli theorem (see Theorem 2.2 in Ch. 4 in Kushner and Yin (2003)), there is a subsequence that converges to a continuous limit, uniformly on each bounded interval. This limit then satisfies the limit ODE.

A useful method to study the stability of solutions to ODEs is by means of Lyapunov functions. Roughly speaking, Lyapunov functions are functions that decrease under the dynamics of the ODE and eventually reach a minimum. Consider the ODE (8), where we may assume $\theta(t) \in \mathbb{R}^d$ and $g : \mathbb{R}^d \to \mathbb{R}^d$. Let $V : \mathbb{R}^d \to [0, \infty)$, be continuously differentiable and such that $V(\bar{\theta}) = 0$, for some $\bar{\theta}$ and $V(\theta) > 0$ for $\theta \neq \bar{\theta}$, $V(\theta) \to \infty$ as $|\theta| \to \infty$. For $\lambda > 0$, we call $Q_\lambda = \{\theta : V(\theta) \leq \lambda\}$ the sub-level set of V at λ. Then,

$$\frac{dV(\theta(t))}{dt} = \nabla_\theta V(\theta(t))^T \frac{d\theta(t)}{dt} = \nabla_\theta V(\theta(t))^T g(\theta(t)) = -k(\theta(t)),$$

where $k(\theta) \coloneqq -\nabla_\theta V(\theta)^T g(\theta)$. Suppose that $k(\theta) \geq 0$ for each $\theta \in Q_\lambda$ and $\theta_0 \in Q_\lambda$. Then $V(\theta(t))$ is non-increasing so $\theta(t)$ remains in Q_λ for all $t \geq 0$. Moreover, since $V(\theta) \geq 0$ it follows that

$$V(\theta(0)) \geq V(\theta(0)) - V(\theta(t)) = -\int_0^t \frac{d}{ds} V(\theta(s)) ds = \int_0^t k(\theta(s)) ds.$$

By letting $t \to \infty$ we find that

$$0 \leq \int_0^\infty k(\theta(s)) ds < \infty,$$

and we conclude that $k(\theta(t)) \to 0$ as $t \to \infty$, which implies that $\theta(t)$ converges to the set $\{\theta \in Q_\lambda : k(\theta) = 0\}$.

Example 21 (Minimization of a convex function)**.** If G is strictly convex and continuously differentiable we may take $g(\theta) = -\nabla G(\theta)$ and iteration

$$\theta_{n+1} = \theta_n - \epsilon_n \nabla G(\theta_n),$$

has an associated limiting ODE

$$\frac{d\theta(t)}{dt} = -\nabla G(\theta(t)).$$

In this case, a Lyapunov function is given by $V(\theta) = G(\theta) + c$, where the constant $c = -G(\bar{\theta})$ and $\bar{\theta}$ is the global minimizer of G, so that the minimum of V is equal to 0. In this case

$$\frac{dV(\theta(t))}{dt} = \nabla_\theta V(\theta(t))^T g(\theta(t)) = -|\nabla G(\theta(t))|^2,$$

which implies that $k(\theta) = |\nabla G(\theta)|^2 \geq 0$, and the set where $k(\theta) = 0$ is the set of stationary points where $\nabla G(\theta) = 0$, of which there is only one: $\bar{\theta}$.

Example 22 (Solving a linear system)**.** Suppose that $g(\theta) = -B\theta + b$, where B is a $d \times d$ matrix and $b \in \mathbb{R}^d$. We may try to find solutions to $g(\theta) = 0$ by the iterative scheme

$$\theta_{n+1} = \theta_n + \epsilon_n g(\theta_n).$$

If B is positive definite (all eigenvalues are positive), then the iteration above is a special case of the previous example and corresponds to minimizing the convex function (quadratic form) $G(\theta) = \frac{1}{2}\theta^{\mathrm{T}} B\theta - \theta^{\mathrm{T}} b$.

Example 23 (Finding zeros of an unknown function)**.** Consider the example in Section 13.1 where $m(\theta) = \mathrm{E}[X \mid \Theta = \theta]$ is strictly decreasing and we want to find $\bar{\theta}$ such that $m(\bar{\theta}) = \bar{m}$. By ignoring the noise the noise-free update can be written as

$$\theta_{n+1} = \theta_n + \epsilon_n g(\theta_n),$$

with $g(\theta) = m(\theta) - \bar{m}$. A Lyapunov function can be constructed as $V(\theta) = \frac{1}{2}(m(\theta) - \bar{m})^2$. Indeed, $V(\bar{\theta}) = 0$, $V(\theta) > 0$ for $\theta \neq \bar{\theta}$, $V(\theta) \to \infty$ as $|\theta| \to \infty$. Then,

$$k(\theta) = -\nabla V(\theta) g(\theta) = -\left(\frac{dm(\theta)}{d\theta}\right)(m(\theta) - \bar{m})^2 \geq 0,$$

since $m(\theta)$ is strictly decreasing. Since m is strictly decreasing (so that $\frac{dm(\theta)}{d\theta} \neq 0$), the limit set, where $k(\theta) = 0$ is the set where $m(\theta) = \bar{m}$. The Lyapunov function is not unique; for instance, we could alternatively take $V(\theta) = \int_\theta^{\bar{\theta}} m(\vartheta) d\vartheta$ as a Lyapunov function.

Example 24 (Contractions)**.** In this example, we relate the limit ODE to Banach's fixed-point theorem. This example also relates to the convergence of reinforcement learning algorithms considered in future lectures. Consider, for simplicity, the state space $\Theta = \mathbb{R}^d$, and a contraction T. Recall that in Banach's fixed-point theorem, we consider the iteration $\theta_{n+1} = T\theta_n = \theta_n + (T - I)\theta_n$, which can be interpreted as a noise-free Robbins-Monro algorithm with step-size $\epsilon_n = 1$. In Robbins-Monro, we usually consider smaller steps of size $\epsilon_n \to 0$. The noise-free Robbins-Monro analogue is then

$$\theta_{n+1} = \theta_n + \epsilon_n [T\theta_n - \theta_n],$$

where the step-size $\epsilon_n \to 0$. Let us write $g(\theta) = T\theta - \theta$ and consider the limit ODE (8). A Lyapunov function in this case is given by $V(\theta) = \frac{1}{2}|\theta - \bar{\theta}|^2$. The intuition for this choice is based on the following. Writing $\theta_{n+1} = (1 - \epsilon_n)\theta_n + \epsilon_n T\theta_n$ we can interpret the update as a convex combination of remaining at θ_n and moving to $T\theta_n$. Recall that, since T is a contraction $|T\theta - \bar{\theta}| \leq |\theta - \bar{\theta}|$, so applying T takes θ closer to $\bar{\theta}$. Therefore, the Robbins-Monro update also takes θ_{n+1} closer to $\bar{\theta}$ than θ_n. To see that V is a Lyapunov function, the first properties are obvious, so we only need to show $k(\theta) \geq 0$. Indeed,

$$\begin{aligned}
k(\theta) &= -\nabla V(\theta)^{\mathrm{T}} g(\theta) \\
&= -(\theta - \bar{\theta})^{\mathrm{T}} (T\theta - \theta) \\
&= -\frac{1}{2}|(\theta - \bar{\theta}) + (T\theta - \theta)|^2 + \frac{1}{2}|\theta - \bar{\theta}|^2 + \frac{1}{2}|T\theta - \theta|^2 \\
&= -\frac{1}{2}|T\theta - \bar{\theta}|^2 + \frac{1}{2}|\theta - \bar{\theta}|^2 + \frac{1}{2}|T\theta - \theta|^2 \\
&\geq 0,
\end{aligned}$$

since $|T\theta - \bar{\theta}| \leq |\theta - \bar{\theta}|$.

Lecture 9

Literature:

- Introductory lectures on stochastic optimization, by John Duchi, available at `https://web.stanford.edu/~jduchi/PCMIConvex/Duchi16.pdf`

15. Stochastic subgradient algorithms in convex optimization

In this section, we consider the problem of minimizing a convex function. Such problems appear frequently in decision problems, as we find the maximum likelihood estimator, optimize hyperparameters in a Bayesian model, or minimize the empirical risk over a class of decision rules parametrized by θ. We will return to these applications later in the course.

Let G be a convex function on $\Theta = \mathbb{R}^d$. The *subgradient* set at θ is defined by

$$\partial_G(\theta) = \{g \in \mathbb{R}^d : G(\psi) \geq G(\theta) + g^{\mathrm{T}}(\psi - \theta), \text{ for all } \psi \in \Theta\}.$$

One can interpret $g \in \partial_G(\theta)$ as the normal of a hyperplane that contains the point $(\theta, G(\theta))$ and lies below the epigraph of G. If G is differentiable at θ, then $\partial_G(\theta)$ consists of the gradient $(\partial/\partial_\theta)G(\theta)$, whereas if G is non-differentiable at θ, then $\partial_G(\theta)$ may consist of many points. For instance, if $G(\theta) = |\theta|$, $\theta \in \mathbb{R}$, then $\partial_G(0) = [-1, 1]$.

15.1. **The subgradient algorithm.** Recall Example 19 from the previous lecture. Suppose that $G(\theta)$ is convex and differentiable everywhere. Then, the optimum $\bar{\theta}$ is the root of the gradient, that is, $g(\bar{\theta}) = (\partial/\partial_\theta)G(\theta) = 0$ and $g(\theta)$ is increasing. The root finding algorithm in Example 19 is then simply (here g is increasing instead of decreasing)

$$\theta_{n+1} = \theta_n - \epsilon g(\theta_n).$$

The subgradient algorithm uses a subgradient, as a replacement of $g(\theta_n)$, to define the updates.

Subgradient algorithm. Take $\theta_1 \in \Theta$ and let $\{\epsilon_n\}_{n\geq 1}$ be the stepsizes. For each $n \geq 1$,

(1) take $g_n \in \partial_G(\theta_n)$,
(2) update: $\theta_{n+1} = \theta_n - \epsilon_n g_n$.

One may observe that the subgradient algorithm is not, in general, a descent method. That is, it may happen that the function value increases after an update. Consider, as an example, the function $G(\theta) = \sum_{i=1}^d |\theta_i|$, $\theta \in \mathbb{R}^d$ and take $\theta = (1, 0, \ldots, 0)^{\mathrm{T}}$. Then

$$\partial_G(\theta) = \{g \in \mathbb{R}^d : g_1 = 1, |g_i| \leq 1, i = 2, \ldots, d\},$$

and

$$G(\theta - \epsilon g) = \sum_{i=1}^d |\theta_i - \epsilon g_i| = (1 - \epsilon) + \epsilon \sum_{i=2}^d |g_i| > 1 = G(\theta),$$

for all $g \in \partial_G(\theta)$ with $\sum_{i=2}^d |g_i| > 1$. Thus, the function value increases after such an update.

Nevertheless, the distance to an optimal point decreases after an update, for sufficiently small step-size. To see this, expand the square to see that, for any $\theta \in \Theta$ and subgradient $g \in \partial_G(\theta)$,

$$|\theta - \epsilon g - \bar{\theta}|^2 = |\theta - \bar{\theta}|^2 - 2\epsilon g^{\mathrm{T}}(\theta - \bar{\theta}) + \epsilon^2 |g|^2, \tag{9}$$

where $\bar{\theta} \in \operatorname{argmin} G(\theta)$. Since g is a subgradient at θ,

$$G(\bar{\theta}) \geq G(\theta) + g^{\mathrm{T}}(\bar{\theta} - \theta) = G(\theta) - g^{\mathrm{T}}(\theta - \bar{\theta}),$$

which implies

$$-g^{\mathrm{T}}(\theta - \bar{\theta}) \leq G(\bar{\theta}) - G(\theta).$$

Inserting this in (9) leads to

$$|\theta - \epsilon g - \bar{\theta}|^2 \leq |\theta - \bar{\theta}|^2 + 2\epsilon[G(\bar{\theta}) - G(\theta)] + \epsilon^2 |g|^2. \tag{10}$$

Therefore, for ϵ sufficiently small that

$$0 < \epsilon < \frac{2[G(\theta) - G(\bar{\theta})]}{|g|^2},$$

it follows that

$$|\theta - \epsilon g - \bar{\theta}|^2 < |\theta - \bar{\theta}|^2.$$

However, since we don't know $G(\bar{\theta})$, we don't know, a priori, how small we have to take ϵ.

The following result gives a simple bound for proving the convergence of the subgradient algorithm.

Theorem 10. *Suppose that there exists at least one minimizing point $\bar{\theta} \in \operatorname{argmin} G(\theta)$ with $G(\bar{\theta}) > -\infty$ and that there is a constant $M < \infty$ such that $|g| \leq M$ for every subgradient $g \in \partial_G(\theta)$ and every $\theta \in \Theta$. Let $\{\epsilon_n\}_{n \geq 1}$ be a non-increasing sequence of stepsizes and $\{\theta_n\}_{n \geq 1}$ be generated by the subgradient algorithm. Then, for $N \geq 1$,*

$$\sum_{n=1}^{N} \epsilon_n [G(\theta_n) - G(\bar{\theta})] \leq \frac{1}{2} \left(R^2 + M^2 \sum_{n=1}^{N} \epsilon_n^2 \right),$$

where $R = |\theta_1 - \bar{\theta}|$.

Proof. The proof relies on the inequality (10). Using that inequality it follows that

$$\begin{aligned} \frac{1}{2}|\theta_{n+1} - \bar{\theta}|^2 &= \frac{1}{2}|\theta_n - \epsilon_n g_n - \bar{\theta}|^2 \\ &\leq \frac{1}{2}|\theta_n - \bar{\theta}|^2 + \epsilon_n[G(\bar{\theta}) - G(\theta_n)] + \frac{1}{2}\epsilon_n^2 |g_n|^2. \end{aligned}$$

Rearranging this inequality and using $|g_n|^2 \leq M^2$ leads to

$$\epsilon_n[G(\theta_n) - G(\bar{\theta})] \leq \frac{1}{2}|\theta_n - \bar{\theta}|^2 - \frac{1}{2}|\theta_{n+1} - \bar{\theta}|^2 + \frac{\epsilon_n^2}{2} M^2.$$

Summation from 1 to N and using the telescoping sum

$$\begin{aligned} \sum_{n=1}^{N} \left(\frac{1}{2}|\theta_n - \bar{\theta}|^2 - \frac{1}{2}|\theta_{n+1} - \bar{\theta}|^2 \right) &= \frac{1}{2}|\theta_1 - \bar{\theta}|^2 - \frac{1}{2}|\theta_{N+1} - \bar{\theta}|^2 \\ &\leq \frac{1}{2} R^2, \end{aligned}$$

completes the proof. □

For the weighted average $\bar{\theta}_N = \sum_{n=1}^N \epsilon_n \theta_n / \sum_{n=1}^N \epsilon_n$ we may use convexity and Theorem 10 and observe that

$$G(\bar{\theta}_N) - G(\bar{\theta}) \leq \frac{\sum_{n=1}^N \epsilon_n [G(\theta_n) - G(\bar{\theta})]}{\sum_{n=1}^N \epsilon_n} \leq \frac{R^2 + M^2 \sum_{n=1}^N \epsilon_n^2}{2 \sum_{n=1}^N \epsilon_n}.$$

Therefore, if $\epsilon_n \to 0$ and $\sum_{n=1}^\infty \epsilon_n = \infty$, it follows that $\sum_{n=1}^N \epsilon_n^2 / \sum_{n=1}^N \epsilon_n \to 0$ so $G(\bar{\theta}_N) - G(\bar{\theta}) \to 0$.

A fixed stepsize, $\epsilon > 0$ in calculation above leads to $\bar{\theta}_N = (1/N) \sum_{n=1}^N \theta_n$ and

$$G(\bar{\theta}_N) - G(\bar{\theta}) \leq \frac{R^2 + M^2 N \epsilon^2}{2N\epsilon}.$$

Minimizing the right-hand-side with respect to ϵ, for a given N, gives the optimal $\epsilon = R/(M\sqrt{N})$ and the bound

$$G(\bar{\theta}_N) - G(\bar{\theta}) \leq \frac{RM}{\sqrt{N}}.$$

Exercise 12 (Robust regression)**.** Consider a linear predictor that predicts a scalar random variable Y given a d-dimensional random vector X. The predictor (decision rule) is $\delta(x) = \theta^{\mathrm{T}} x$ for some $\theta \in \mathbb{R}^d$. We have observations $\{(x_i, y_i)\}_{i=1}^m$ of the independent random pairs $\{(X_i, Y_i)\}_{i=1}^m$ and aim to minimize the empirical risk

$$R^{\mathrm{emp}}(\theta) = \frac{1}{m} \sum_{i=1}^m L(y_i, \theta^{\mathrm{T}} x_i),$$

where the loss function is the ℓ_1-loss: $L(y, a) = |y - a|$, $y, a \in \mathbb{R}$. Determine a subgradient of the convex function R^{emp}at an arbitrary point $\theta \in \mathbb{R}^d$.

15.2. **The projected subgradient algorithm.** Let G be a convex function and suppose we want to minimize G over some constrained set C. We can still use the subgradient method as long as we guarantee that the updates remain in C. A simple way to do this is to project θ_{n+1} onto C, every time the usual subgradient update escapes from C. For a closed convex set $C \subset \mathbb{R}^d$, let

$$\mathrm{proj}_C(\theta) = \mathrm{argmin}_{\psi \in C} |\psi - \theta|,$$

be the projection of θ onto C.

Projected subgradient algorithm. Take $\theta_1 \in C$ and let $\{\epsilon_n\}_{n \geq 1}$ be the stepsizes. For each $n \geq 1$,

(1) take $g_n \in \partial_G(\theta_n)$,
(2) update: $\theta_{n+1} = \mathrm{proj}_C(\theta_n - \epsilon_n g_n)$.

For the projected subgradient algorithm to work well it is necessary that the projection onto C may be efficiently computed. If this is not the case, that is, if the projection is computationally costly, then the algorithm is practically useless.

Assume that C is compact and convex. Since the projection cannot increase the distance to the minimizer $\bar{\theta}$, see Exercise 13 below, it follows that

$$\frac{1}{2}|\theta_{n+1} - \bar{\theta}|^2 = \frac{1}{2}|\mathrm{proj}_C(\theta_n - \epsilon_n g_n) - \bar{\theta}|^2 \leq \frac{1}{2}|\theta_n - \epsilon_n g_n - \bar{\theta}|^2 \tag{11}$$

Then, one can proceed exactly as in the proof of Theorem 10 to prove the same upper bound.

If we again assume that the constraint set C is compact and convex, we can use a slightly different argument to prove the following related result. Let $\bar{\theta} \in \operatorname{argmin}_{\theta \in C} G(\theta)$.

Theorem 11. *Suppose that C is compact and convex and $|\theta - \bar{\theta}| \leq R < \infty$ for all $\theta \in C$. Suppose also that there is a constant $M < \infty$ such that $|g| \leq M$ for every subgradient $g \in \partial_G(\theta)$ and every $\theta \in C$. Let $\{\epsilon_n\}_{n \geq 1}$ be a non-increasing sequence of stepsizes and $\{\theta_n\}$ be generated by the projected subgradient algorithm. Then, for $N \geq 1$,*

$$\sum_{n=1}^{N} [G(\theta_n) - G(\bar{\theta})] \leq \frac{R^2}{2\epsilon_N} + \frac{M^2}{2} \sum_{n=1}^{N} \epsilon_n.$$

Proof. By (11)

$$\frac{1}{2}|\theta_{n+1} - \bar{\theta}|^2 = \frac{1}{2}|\text{proj}_C(\theta_n - \epsilon_n g_n) - \bar{\theta}|^2 \leq \frac{1}{2}|\theta_n - \epsilon_n g_n - \bar{\theta}|^2,$$

and we may apply (10) to obtain

$$\frac{1}{2}|\theta_{n+1} - \bar{\theta}|^2 \leq \frac{1}{2}|\theta_n - \bar{\theta}|^2 + \epsilon_n [G(\bar{\theta}) - G(\theta_n)] + \frac{1}{2}\epsilon_n^2 |g|^2.$$

Rearranging this inequality, using $|g|^2 \leq M^2$, and dividing by ϵ_n leads to

$$G(\theta_n) - G(\bar{\theta}) \leq \frac{1}{2\epsilon_n} \left(|\theta_n - \bar{\theta}|^2 - |\theta_{n+1} - \bar{\theta}|^2 \right) + \frac{M^2}{2} \epsilon_n.$$

Summation from 1 to N yields

$$\sum_{n=1}^{N} [G(\theta_n) - G(\bar{\theta})] \leq \sum_{n=1}^{N} \frac{1}{2\epsilon_n} \left(|\theta_n - \bar{\theta}|^2 - |\theta_{n+1} - \bar{\theta}|^2 \right) + \frac{M^2}{2} \sum_{n=1}^{N} \epsilon_n.$$

The first sum on the right-hand side can be bounded above by

$$\begin{aligned}
&\sum_{n=1}^{N} \frac{1}{2\epsilon_n} \left(|\theta_n - \bar{\theta}|^2 - |\theta_{n+1} - \bar{\theta}|^2 \right) \\
&\quad = \sum_{n=2}^{N} \left(\frac{1}{2\epsilon_n} - \frac{1}{2\epsilon_{n-1}} \right) |\theta_n - \bar{\theta}|^2 + \frac{1}{2\epsilon_1} |\theta_1 - \bar{\theta}|^2 - \frac{1}{2\epsilon_{N+1}} |\theta_{N+1} - \bar{\theta}|^2 \\
&\quad \leq \sum_{n=2}^{N} \left(\frac{1}{2\epsilon_n} - \frac{1}{2\epsilon_{n-1}} \right) R^2 + \frac{R^2}{2\epsilon_1},
\end{aligned}$$

since $\epsilon_n \leq \epsilon_{n-1}$. The last sum is a telescoping sum, which leads to

$$\sum_{n=1}^{N} [G(\theta_n) - G(\bar{\theta})] \leq \frac{R^2}{2\epsilon_N} + \frac{M^2}{2} \sum_{n=1}^{N} \epsilon_n.$$

This completes the proof. □

For the average $\bar{\theta}_N = (1/N) \sum_{n=1}^{N} \theta_n$ we may use convexity and Theorem 11 and the step-size $\epsilon_n = \epsilon/\sqrt{n}$ to obtain

$$G(\bar{\theta}_N) - G(\bar{\theta}) \leq \frac{1}{N} \sum_{n=1}^{N} G(\theta_n) - G(\bar{\theta}) \leq \frac{R^2}{2\epsilon\sqrt{N}} + \frac{M^2 \epsilon}{2N} \sum_{n=1}^{N} \frac{1}{\sqrt{n}} \leq \frac{R^2}{2\epsilon\sqrt{N}} + \frac{M^2 \epsilon}{\sqrt{N}},$$

since

$$\sum_{n=1}^{N} \frac{1}{\sqrt{n}} \leq \int_0^N \frac{1}{\sqrt{x}} dx = 2\sqrt{N}.$$

Exercise 13 (Projections on convex sets)**.** Let $C \subset \mathbb{R}^d$ be a closed convex set and $\bar{\theta} \in C$. For any $\theta \in \mathbb{R}^d$, let $\theta_p = \text{proj}_C(\theta)$ be the projection of θ onto C. Show that $|\theta_p - \bar{\theta}| \leq |\theta - \bar{\theta}|$ for every $\theta \in \mathbb{R}^d$.

15.3. **The stochastic subgradient algorithm.** Let G be a convex function and $\partial_G(\theta)$ is subgradient set at θ. Again we will be interested in minimizing G over a closed convex set C (which could be taken to be $\mathbb{R}^d$ if no constraints are present). This time we will use random updates, in the form of stochastic subgradients.

A random vector g is a *stochastic subgradient* of G at θ if $\mathrm{E}[g] \in \partial_G(\theta)$. Note that the outcome of g does not have to be a subgradient, only its expectation. This is similar to the Robbins-Monro algorithm in the previous lecture.

Stochastic subgradient algorithm. Take $\theta_1 \in C$ and let $\{\epsilon_n\}_{n\geq 1}$ be the stepsizes. For each $n \geq 1$,

(1) take g_n to be a stochastic subgradient such that $\mathrm{E}[g_n \mid \theta_n] \in \partial_G(\theta_n)$,
(2) update: $\theta_{n+1} = \text{proj}_C(\theta_n - \epsilon_n g_n)$.

Example 25 (Robust regression)**.** Consider the robust regression problem in Exercise 12 where we aim to minimize the empirical risk

$$R^{\text{emp}}(\theta) = \frac{1}{m} \sum_{i=1}^{m} L(y_i, \theta^{\mathrm{T}} x_i),$$

where the loss function is the ℓ_1-loss: $L(y, a) = |y - a|$, $y, a \in \mathbb{R}$. If g_i is a subgradient of $L(y_i, \theta^{\mathrm{T}} x_i)$, then $g = (1/m) \sum_{i=1}^{m} g_i$ is a subgradient of R^{emp}. This implies that the evaluation of the subgradient is costly if m is large. A simple stochastic subgradient is obtained by sampling an index i uniformly among $\{1, \ldots, m\}$ and using the resulting g_i. Since

$$\mathrm{E}[g_i \mid \theta_n] = \frac{1}{m} \sum_{i=1}^{m} g_i,$$

is a subgradient of R^{emp} at θ_n, it is a stochastic subgradient. If the loss function $L(y, a)$ is convex and everywhere differentiable in a, then $g_i = (\partial/\partial_\theta) L(y_i, \theta^{\mathrm{T}} x_i)$ and the algorithm just described is called a *stochastic gradient descent algorithm*.

For the stochastic subgradient algorithm, with a compact and convex constraint set C, we have a similar convergence result as in the projected subgradient algorithm. Let $\bar{\theta} \in \text{argmin}_{\theta \in C} G(\theta)$.

Theorem 12. *Suppose that C is compact and convex and $|\theta - \bar{\theta}| \leq R < \infty$ for all $\theta \in C$. Suppose also that there is a constant $M < \infty$ such that $\mathrm{E}[|g|^2] \leq M^2$ for every stochastic subgradient g at every $\theta \in C$. Let $\{\epsilon_n\}_{n\geq 1}$ be a non-increasing sequence of stepsizes and $\{\theta_n\}$ be generated by the stochastic subgradient algorithm. Then, for $N \geq 1$, with $\bar{\theta}_N = (1/N) \sum_{n=1}^{N} \theta_n$,*

$$\mathrm{E}[G(\bar{\theta}_N) - G(\bar{\theta})] \leq \frac{R^2}{2N\epsilon_N} + \frac{M^2}{2N} \sum_{n=1}^{N} \epsilon_n.$$

Proof. As in the proof of the projected subgradient algorithm, we expand $\frac{1}{2}|\theta_{n+1}-\bar{\theta}|^2$ as

$$\begin{aligned}\frac{1}{2}|\theta_{n+1}-\bar{\theta}|^2 &= \frac{1}{2}|\mathrm{proj}_C(\theta_n-\epsilon_n g_n)-\bar{\theta}|^2 \le \frac{1}{2}|\theta_n-\epsilon_n g_n-\bar{\theta}|^2\\ &= \frac{1}{2}|\theta_n-\bar{\theta}|^2-\epsilon_n\langle g_n,\theta_n-\bar{\theta}\rangle+\frac{\epsilon_n^2}{2}|g_n|^2,\end{aligned}$$

where g_n is the stochastic subgradient. We add and subtract $\epsilon_n\langle \mathrm{E}[g_n \mid \theta_n],\theta_n-\bar{\theta}\rangle$, which leads to

$$\begin{aligned}\frac{1}{2}|\theta_{n+1}-\bar{\theta}|^2 &\le \frac{1}{2}|\theta_n-\bar{\theta}|^2-\epsilon_n\langle \mathrm{E}[g_n \mid \theta_n],\theta_n-\bar{\theta}\rangle+\frac{\epsilon_n^2}{2}|g_n|^2-\epsilon_n\langle g_n-\mathrm{E}[g_n \mid \theta_n],\theta_n-\bar{\theta}\rangle\\ &\le \frac{1}{2}|\theta_n-\bar{\theta}|^2-\epsilon_n[G(\theta_n)-G(\bar{\theta})]+\frac{\epsilon_n^2}{2}|g_n|^2-\epsilon_n\langle g_n-\mathrm{E}[g_n \mid \theta_n],\theta_n-\bar{\theta}\rangle.\end{aligned}$$

Rearranging this inequality and dividing by ϵ_n leads to

$$G(\theta_n)-G(\bar{\theta}) \le \frac{1}{2\epsilon_n}\left(|\theta_n-\bar{\theta}|^2-|\theta_{n+1}-\bar{\theta}|^2\right)+\frac{\epsilon_n}{2}|g_n|^2-\langle g_n-\mathrm{E}[g_n \mid \theta_n],\theta_n-\bar{\theta}\rangle.$$

Summation from 1 to N yields, as in the previous proof,

$$\sum_{n=1}^N[G(\theta_n)-G(\bar{\theta})] \le \frac{R^2}{2\epsilon_N}+\frac{1}{2}\sum_{n=1}^N\epsilon_n|g_n|^2-\sum_{n=1}^N\langle g_n-\mathrm{E}[g_n \mid \theta_n],\theta_n-\bar{\theta}\rangle.$$

Taking expectation, it follows that

$$\mathrm{E}[\langle g_n-\mathrm{E}[g_n \mid \theta_n],\theta_n-\bar{\theta}\rangle]=\mathrm{E}[\langle \mathrm{E}[g_n \mid \theta_n]-\mathrm{E}[g_n \mid \theta_n],\theta_n-\bar{\theta}\rangle]=0,$$

and with $\mathrm{E}[|g_n|^2] \le M^2$ we obtain

$$\mathrm{E}\Big[\sum_{n=1}^N G(\theta_n)-G(\bar{\theta})\Big] \le \frac{R^2}{2\epsilon_N}+\frac{M^2}{2}\sum_{n=1}^N\epsilon_n.$$

The result for the average $\bar{\theta}_N$ follows by convexity (as shown above). □

15.4. **Maximum likelihood in logistic regression.** Consider the logistic regression model encountered in Section 8.1.1. Suppose we have training data in the form of $\{(x_i,y_i)\}_{i=1}^n$ that are outcomes of independent pairs $\{(X_i,Y_i)\}_{i=1}^n$ where X_i represents the explanatory variables and Y_i the corresponding label. We also have $X_{n+1}=x_{n+1}$ and want to predict the label Y_{n+1}. In logistic regression we assume that $Y_1 \mid X_1, Y_2 \mid X_2, \dots$ are conditionally independent given $\Theta=\theta\in\mathbb{R}^d$ with

$$\Pr(Y_i=1 \mid \Theta=\theta, X_i=x_i)=\mathrm{sigm}\Big(\sum_{j=1}^d\theta_j\phi_j(x_i)\Big),$$

$$\Pr(Y_i=-1 \mid \Theta=\theta, X_i=x_i)=\mathrm{sigm}\Big(-\sum_{j=1}^d\theta_j\phi_j(x_i)\Big),$$

where $\phi_1,\dots,\phi_d$ are given functions, summarizing the necessary information in X_i. With $X=(X_1,Y_1,\dots,X_n,Y_n,X_{n+1})$ and $Y=Y_{n+1}$ we have the likelihood

$$\begin{aligned}p_{X|\Theta}(x \mid \theta) &= p_{Y_1,\dots,Y_n|\Theta,X_1,\dots,X_{n+1}}(y_1,\dots,y_n \mid \theta,x_1,\dots,x_{n+1})p_{X_1,\dots,X_{n+1}}(x_1,\dots,x_{n+1})\\ &= \prod_{i=1}^n\mathrm{sigm}\Big(y_i\sum_{j=1}^d\theta_j\phi_j(x_i)\Big)\prod_{i=1}^{n+1}p_{X_i}(x_i).\end{aligned}$$

Suppose that the predictive distribution of Y given X is approximated using the maximum likelihood estimate $\theta^{\mathrm{ML}} = \operatorname{argmax}_{\theta\in\mathbb{R}^d} p_{X|\Theta}(x \mid \theta)$. Then, classification of the new observation x_{n+1} is based on

$$p_{Y|X,\Theta}(1 \mid x, \theta) = p_{Y_{n+1}|\Theta,X_{n+1}}(1 \mid x_{n+1,\theta^{\mathrm{ML}}}) = \operatorname{sigm}\left(\sum_{j=1}^{d} \theta_j^{\mathrm{ML}} \phi_j(x_{n+1})\right).$$

It remains to determine the maximum likelihood estimator θ^{ML}.

To this end, we minimize the negative log-likelihood:

$$\begin{aligned} G(\theta) &= -\log p_{X|\Theta}(x \mid \theta) \\ &\propto \sum_{i=1}^{n} -\log \operatorname{sigm}\Big(y_i \sum_{j=1}^{d} \theta_j \phi_j(x_i)\Big) \\ &= \sum_{i=1}^{n} \log\left(1 + \exp\left\{-y_i \sum_{j=1}^{d} \theta_j \phi_j(x_i)\right\}\right) \\ &= \sum_{i=1}^{n} \log\left(1 + \exp\left\{-y_i \theta^{\mathrm{T}} \phi(x_i)\right\}\right). \end{aligned}$$

which is convex in θ (you may prove yourself that $\log(1 + e^{-t})$ is convex in t by differentiating twice).

Note that G is differentiable at every θ. We compute its gradient. Note first that

$$\frac{\partial}{\partial t} \log(1 + e^{-t}) = -\operatorname{sigm}(-t).$$

It follows that

$$\begin{aligned} \frac{\partial}{\partial \theta_j} G(\theta) &= -\sum_{i=1}^{n} y_i \phi_j(x_i) \operatorname{sigm}(-y_i \theta^{\mathrm{T}} \phi(x_i)) \\ &= \sum_{i=1}^{n} y_i \phi_j(x_i) \left(\operatorname{sigm}(y_i \theta^{\mathrm{T}} \phi(x_i)) - 1\right). \end{aligned}$$

In vector notation, the gradient of G at θ is

$$\nabla G(\theta) = \sum_{i=1}^{n} y_i \phi(x_i) \left(\operatorname{sigm}(y_i \theta^{\mathrm{T}} \phi(x_i)) - 1\right).$$

Solving $\nabla G(\theta) = 0$ is non-trivial and numerical methods are needed. If a Newton method is to be used, it is necessary to compute the inverse of the Hessian of G. Although the Hessian can be computed by differentiating again, each component consists of a sum of n elements, and if d is large, computing its inverse will be time-consuming. Although slower in moderate dimensions, a (sub-)gradient method can be used. If n is large, the evaluation of the gradient is also time-consuming and it is often more effective to use a stochastic gradient descent method by sampling an index i uniformly from $\{1, \ldots, n\}$ and use the stochastic (sub-)gradient

$$g = n y_i \phi(x_i) \left(\operatorname{sigm}(y_i \theta^{\mathrm{T}} \phi(x_i)) - 1\right).$$

The n is needed for the expected value to be equal to the (sub-)gradient. Remember that the random variable in g is i, which is sampled uniformly from 0 to n. So we

have that

$$\mathrm{E}[g] = \mathrm{E}[ny_i\phi(x_i)\left(\mathrm{sigm}(y_i\theta^{\mathrm{T}}\phi(x_i)) - 1\right)] = \frac{1}{n}\sum_{i=1}^{n} ny_i\phi(x_i)\left(\mathrm{sigm}(y_i\theta^{\mathrm{T}}\phi(x_i)) - 1\right)$$
$$= \sum_{i=1}^{n} y_i\phi(x_i)\left(\mathrm{sigm}(y_i\theta^{\mathrm{T}}\phi(x_i)) - 1\right) = \nabla G(\theta).$$

Exercise 14. Let $\theta = (\psi, \Theta, \xi)$ where $\psi = (\psi_1, \ldots, \psi_{d'})^{\mathrm{T}}$ is a vector, $\Theta = (\Theta_{kl}; k = 1, \ldots, d, l = 1, \ldots, d')$ is a matrix and $\xi = (\xi_1, \ldots, \xi_{d'})^{\mathrm{T}}$ is a vector. They are parameters of the function $f_\theta : \mathbb{R}^d \to \mathbb{R}$ given by

$$f_\theta(x) = \sum_{l=1}^{d'} \psi_l \left(\sum_{k=1}^{d} x_k \Theta_{kl} + \xi_l \right)_+, \quad x \in \mathbb{R}^d,$$

where, for a vector $t \in \mathbb{R}^{d'}$, $(t)_+ = (\max(t_1, 0), \ldots, \max(t_{d'}, 0))^{\mathrm{T}}$ is the component-wise positive part. The function f_θ can be interpreted as the output of a one-layer neural network with the ReLU activation function. In a binary classification problem, we have observed data $x_1, \ldots, x_n$ where $x_i = (x_{i1}, \ldots, x_{id})^{\mathrm{T}}$ and we are interested in minimizing the negative log-likelihood given by

$$G(\theta) = \sum_{i=1}^{n} -\log \mathrm{sigm}(y_i f_\theta(x_i)).$$

(a) Show that $\psi \mapsto G(\psi, \Theta, \xi)$ is convex.
(b) Show that $\theta \mapsto G(\theta)$ is not convex.

Lecture 10

Literature:

- MacKay, Chapter 45
- Rasmussen and Williams, Chapter 1, 2, and 4

16. Gaussian processes

Let us first recall that a random vector $X = (X_1, \dots, X_n)^{\mathrm{T}}$ has a multivariate normal distribution if, for any vector $a = (a_1, \dots, a_n)^{\mathrm{T}}$, the linear combination $a^{\mathrm{T}}X$ has a (one-dimensional) normal distribution. We often write $N(m, K)$ for a multivariate normal distribution, where m is the mean and K, the covariance matrix. We have the following useful properties of multivariate normal distributions. Let $X \sim N(m, K)$, then

(1) for a $k \times n$ matrix B, we have
$$BX \sim N(Bm, BKB^{\mathrm{T}}),$$
(2) for a decomposition of X as $X = (X_1, X_2)^{\mathrm{T}}$, where X_1 consists of the first n_1 components and X_2 the remaining $n - n_1$ components, and corresponding decomposition of the mean and covariance as
$$m = \begin{pmatrix} m_1 \\ m_2 \end{pmatrix}, \qquad K = \begin{pmatrix} K_{11} & K_{12} \\ K_{21} & K_{22} \end{pmatrix},$$
we have the conditional distribution $X_1 \mid X_2 = x_2 \sim N(m_{1|2}, K_{1|2})$, where
$$m_{1|2} = m_1 + K_{12}K_{22}^{-1}(x_2 - m_2), \qquad K_{1|2} = K_{11} - K_{12}K_{22}^{-1}K_{21}. \tag{12}$$

A general Gaussian process can be thought of as a random function $f : \mathcal{X} \to \mathbb{R}$, where $\mathcal{X}$ is a general space. At each coordinate $x \in \mathcal{X}$, the function value $f(x)$ is a Gaussian random variable.

A Gaussian process on $\mathcal{X}$ is a collection of random variables $\{f(x)\}_{x\in\mathcal{X}}$ such that, for any finite collection $(x_1, \dots, x_n) \in \mathcal{X}^n$ the random vector $(f(x_1), \dots, f(x_n))^{\mathrm{T}}$ has a multivariate normal distribution. Existence of Gaussian processes can be proved using Kolmogorov's extension theorem.

Example 26 (Multivariate normal distribution)**.** A multivariate normal distribution can be interpreted as a Gaussian process on $\mathcal{X} = \{1, \dots, n\}$, with the identification $X = (X_1, \dots, X_n)^{\mathrm{T}} = (f(1), \dots, f(n))^{\mathrm{T}}$.

Example 27 (Brownian motion)**.** A Brownian motion $\{f(t)\}_{t\geq 0}$ is defined by its properties

- $f(0) = 0$,
- for any $0 \leq s < t$, $f(t) - f(s) \sim N(0, t - s)$,
- for any $0 \leq s < t \leq u < v$, the increments $f(t) - f(s)$ and $f(v) - f(u)$ are independent.

The Brownian motion can be viewed as a Gaussian process on $\mathcal{X} = [0, \infty)$, according to the definition above, by noting that for any $0 \leq t_1 < \cdots < t_n$ the random vector $(f(t_1), \dots, f(t_n))^{\mathrm{T}}$ has a multivariate normal distribution. Indeed, by the definition of the Brownian motion
$$(f(t_1), f(t_2) - f(t_1), \dots, f(t_n) - f(t_{n-1}))^{\mathrm{T}} \sim N(0, D),$$

where

$$D = \operatorname{diag}(t_1, t_2 - t_1, \ldots, t_n - t_{n-1}),$$

and

$$(f(t_1), \ldots, f(t_n))^{\mathrm{T}} = \begin{pmatrix} 1 & 0 & \cdots & \cdots & 0 \\ 1 & 1 & 0 & \cdots & 0 \\ \vdots & \vdots & \vdots & \vdots & \vdots \\ 1 & 1 & 1 & 1 & 1 \end{pmatrix} \begin{pmatrix} f(t_1) \\ f(t_2) - f(t_1) \\ \vdots \\ f(t_n) - f(t_{n-1}) \end{pmatrix},$$

which implies that $(f(t_1), \ldots, f(t_n))^{\mathrm{T}}$ has a multivariate normal distribution.

Example 28 (Bayesian linear regression)**.** Consider a Bayesian regression model where f has the form $f(x) = \Theta^{\mathrm{T}}\phi(x)$, where $\phi(x) = (1, \phi_1(x), \ldots, \phi_d(x))^{\mathrm{T}}$ and $\Theta = (\Theta_0, \ldots, \Theta_d)^{\mathrm{T}} \sim N(0, \Sigma)$. Then, f is a Gaussian process with mean function m and covariance function k given by

$$\begin{aligned} m(x) &= \mathrm{E}[\Theta^{\mathrm{T}}\phi(x)] = \mathrm{E}[\Theta^{\mathrm{T}}]\phi(x) = 0, \\ k(x, x') &= \phi(x)^{\mathrm{T}}\,\mathrm{E}[\Theta\Theta^{\mathrm{T}}]\phi(x') = \phi(x)^{\mathrm{T}}\Sigma\phi(x'). \end{aligned}$$

16.1. **Covariance functions.** The properties of a Gaussian process are completely determined by its mean and covariance functions. Let us first look at some examples of covariance functions.

Squared exponential covariance. The squared exponential covariance function is given by

$$k(x, x') = k_{SE}(|x - x'|), \text{ where } k_{SE}(r) = \exp\left(-\frac{r^2}{2\ell^2}\right).$$

The parameter ℓ is called the characteristic length scale.

Matérn class of covariance functions. The Matérn class of covariance functions is given by

$$k(x, x') = k_{\text{Matérn}}(|x - x'|), \text{ where } k_{\text{Matérn}}(r) = \frac{2^{1-\nu}}{\Gamma(\nu)}\left(\frac{\sqrt{2\nu}\,r}{\ell}\right) K_\nu\left(\frac{\sqrt{2\nu}\,r}{\ell}\right).$$

K_ν above is a *modified Bessel function of the second kind*. A special case is $\nu = 1/2$, when the Matérn function reduces to an exponential

$$k(r) = e^{-r/\ell}.$$

Anisotropic versions. Given an isotropic covariance $k(|x - x'|)$ for some function $k(r)$ we can create an anisotropic version by replacing the argument $r = |x - x'|$ by $r = \sqrt{(x - x')^{\mathrm{T}} M (x - x')}$ where M is a positive definite matrix. The introduction of the matrix M corresponds to a change of coordinates $x \to Ax$, where A is the Cholesky decomposition of M, $AA^{\mathrm{T}} = M$. For instance, a diagonal matrix M corresponds to using different length scales along the different dimensions.

Dot-product covariances. Dot-product covariances are those where $k(x, x')$ is a function of the dot-product (scalar product) $x^{\mathrm{T}}x' = x \cdot x'$. A particular case is the Bayesian linear regression considered in Example 28 with $\phi(x) = x$ and $\Theta \sim N(0, \mathrm{diag}(\sigma_0^2, \sigma^2, \ldots, \sigma^2))$. Then, the covariance function is given by

$$k(x, x') = \phi(x)^{\mathrm{T}}\Sigma\phi(x') = \sigma_0^2 + \sigma^2 x^{\mathrm{T}}x'.$$

Related covariances take the form $k(x, x') = (\sigma_0^2 + \sigma^2 x^{\mathrm{T}}x')^p$, for some positive integer p. These correspond to higher-order polynomial regressions.

For more examples of covariance functions popular in machine learning, see Rasmussen and Williams, Chapter 4.

16.2. **Gaussian processes as limits of one-hidden-layer neural networks.** Consider a one-hidden-layer neural network with input variables $x = (x_1, \ldots, x_n)^{\mathrm{T}}$. We write the output of the network as

$$f_H(x) = \frac{1}{\sqrt{H}} \sum_{j=1}^{H} v_j h(x, u_j),$$

where H is the number of hidden variables, v_j are the hidden-to-output weights, h is a bounded activation function, e.g. $h(x, u_j) = \tanh(x^{\mathrm{T}}u_j)$ or $\mathrm{sigm}(x^{\mathrm{T}}u_j)$, etc., and $u_j = (u_{j1}, \ldots, u_{jn})$ are the input-to-hidden weights. Let us take a prior distribution on the parameters as follows. $\{v_j\}, \{u_j\}$ are independent with zero mean, all the v_j's are independent with variance σ_v^2 and the vectors $\{u_j\}$ are independent and identically distributed.

Let f be a composition of functions $f^{(i)}$, $i = 1, \ldots, n$, of the form

$$f(x) = f^{(n)} \circ \cdots \circ f^{(1)}(x) = f^{(n)}(f^{(n-1)}(\ldots f^{(1)}(x))). \tag{13}$$

We interpret each $f^{(i)}$ as the transition between hidden layers and $f^{(i)}$ has a simple form

$$f^{(i)}(h^{(i-1)}) = g^{(i)}(W^{(i)}h^{(i-1)} + b^{(i)}).$$

In this notation $h^{(i-1)}$ is a vector, the value at the $(i-1)$th layer, $h^{(i-1)} = f^{(i-1)} \circ \cdots \circ f^{(1)}(x)$, W^i is a matrix, $b^{(i)}$ is a vector, called the bias, and $g^{(i)}$ is an activation function applied to each component of $W^{(i)}h^{(i-1)} + b^{(i)}$.

We may compute the mean function and covariance function of f as

$$\begin{aligned}
\mathrm{E}[f_H(x)] &= 0, \\
\mathrm{E}[f_H(x)f_H(x')] &= \frac{1}{H}\sum_{j=1}^{H} \sigma_v^2 \,\mathrm{E}[h(x, u_j)h(x', u_j)] \\
&= \sigma_v^2 \,\mathrm{E}[h(x, u_1)h(x', u_1)] \\
&=: k_{\mathrm{NN}}(x, x')
\end{aligned}$$

where we have used the independence and the fact that u_j's are identically distributed. Since the activity function h is bounded $\mathrm{E}[h(x, u_1)h(x', u_1)] < \infty$ and we can view $f_H(x)$ as a normalized sum of independent random variables with finite variance. By the central limit theorem $f_H(x)$ converges in distribution, as $H \to \infty$, to a normal with mean zero and variance $\sigma_v^2 \,\mathrm{E}[h(x, u_1)h(x, u_1)]$. Similarly, for any finite collection of points $x_1, \ldots, x_n$, the multivariate central limit theorem applies, which implies that the limit of $(f_H(x_1), \ldots, f(x_n))$ converges to a multivariate normal

distribution with the corresponding covariance function. We conclude that the limit $f = \lim_{H\to\infty} f_H$ is a Gaussian process with covariance function k_{NN}. In the particular case when $u_j \sim N(0,\Sigma)$ and $h(x,u) = \mathrm{erf}(u_0 + \sum_{j=1}^d u_j x_j)$, where

$$\mathrm{erf}(z) = \frac{2}{\sqrt{\pi}} \int_{-\infty}^{z} e^{-t^2/2} dt,$$

the covariance function can be explicitly computed as

$$k_{\mathrm{NN}}(x,x') = \frac{2}{\pi} \sin^{-1}\left(\frac{2(1,x^{\mathrm{T}})\Sigma(1,(x')^{\mathrm{T}})^{\mathrm{T}}}{\sqrt{(1+2(1,x^{\mathrm{T}})\Sigma(1,x^{\mathrm{T}})^{\mathrm{T}})(2(1,(x')^{\mathrm{T}})\Sigma(1,(x')^{\mathrm{T}})^{\mathrm{T}})}} \right).$$

16.3. **Regularity of Gaussian processes.** In this section, we will give some results on the regularity of Gaussian processes. For simplicity, we will throughout this section assume that the mean function vanishes, $m(x) = 0$, for all x. A stochastic process f is continuous almost surely at x_0 if

$$\Pr\left(\lim_{x\to x_0} f(x) = f(x_0) \right) = 1.$$

If it is continuous almost surely at every x_0, we say that it is continuous almost surely. A weaker condition, which is easier to check, is mean-square continuity. We say that f is mean-square continuous (or L^2-continuous) at x_0 if

$$\lim_{x\to x_0} \mathrm{E}[|f(x) - f(x_0)|^2] = 0.$$

If it is mean-square continuous at every x_0, we say that it is mean-squared continuous.

Since, with $k(x,x') = \mathrm{E}[f(x)f(x')]$,

$$\mathrm{E}[|f(x) - f(x_0)|^2] = k(x,x) - 2k(x,x_0) + k(x_0,x_0),$$

we see that f is mean-squared continuous if k is continuous on the diagonal $\{(x,x)\}_{x\in\mathcal{X}}$. A weakly stationary process, that is, a process for which $k(x,x')$ is a function of $x - x'$, is mean-squared continuous if $k(x-x')$ is continuous at the origin. One can also show (from the Kolmogorov continuity theorem, but no proof in these notes) that an isotropic Gaussian process in $\mathbb{R}^d$ is almost surely continuous if $k(r)$ satisfies

$$k(0) - k(r) \leq C r^{d+\alpha},$$

for some constants $C, \alpha > 0$.

17. Gaussian processes for regression

Consider a regression problem with additive noise. Suppose we have training data in the form of $\{(x_i,y_i)\}_{i=1}^n$ that are outcomes of independent pairs $\{(X_i,Y_i)\}_{i=1}^n$ where X_i represents the explanatory variables and Y_i the response variable. We also have $X_{n+1} = x_{n+1}, \ldots, X_{n+k} = x_{n+k}$ and want to predict the responses $Y_{n+1}, \ldots, Y_{n+k}$.

The conditional probabilities are modeled as

$$Y_i = f(X_i) + \varepsilon_i.$$

It is assumed that the prior distribution of f is a Gaussian process with zero mean and covariance function k. We will write $X = (X_1,\ldots,X_n)^{\mathrm{T}}$, $Y = (Y_1,\ldots,Y_n)^{\mathrm{T}}$ and $X' = (X_{n+1},\ldots,X_{n+k})^{\mathrm{T}}$, $Y' = (Y_{n+1},\ldots,Y_{n+k})^{\mathrm{T}}$ and similarly for the corresponding observations $x = (x_1,\ldots,x_n)^{\mathrm{T}}$ etc.

Conditional on $X = x$ and $X' = x'$ we have, by independence of f and $\{\varepsilon_i\}_{i\geq 1}$,

$$\begin{pmatrix} f(x') \\ Y \\ Y' \end{pmatrix} \sim N\left(\begin{pmatrix} 0 \\ 0 \\ 0 \end{pmatrix}, \begin{pmatrix} k(x',x') & k(x',x) & k(x',x') \\ k(x,x') & k(x,x)+\sigma^2 I_n & k(x,x') \\ k(x',x') & k(x',x) & k(x',x')+\sigma^2 I_k \end{pmatrix}\right), \tag{14}$$

where I_n denotes the $n \times n$ identity matrix. Essentially everything we need to know about Gaussian process regression follows from (14). For instance, taking

$$B = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \end{pmatrix}$$

we find that, conditional on $X = x$ and $X' = x'$ we have

$$\begin{pmatrix} f(x') \\ Y \end{pmatrix} \sim N\left(\begin{pmatrix} 0 \\ 0 \end{pmatrix}, \begin{pmatrix} k(x',x') & k(x',x) \\ k(x,x') & k(x,x)+\sigma^2 I_n \end{pmatrix}\right). \tag{15}$$

The posterior distribution of $f(x')$ given $X = x$ and $Y = y$ is then, by (12), normal with mean and covariance given by

$$\begin{aligned} m_{f(X')|Y,X,X'}(y,x,x') &= k(x',x)[k(x,x)+\sigma^2 I_n]^{-1}y, \\ k_{f(X')|Y,X,X'}(y,x,x') &= k(x',x') - k(x',x)[k(x,x)+\sigma^2 I_n]^{-1}k(x,x'). \end{aligned}$$

Similarly, the predictive distribution of Y' given $X = x, Y = y, X' = x'$, is obtained as follows. Taking

$$B = \begin{pmatrix} 0 & 0 & 1 \\ 0 & 1 & 0 \end{pmatrix}$$

we find that, conditional on $X = x$ and $X' = x'$ we have

$$\begin{pmatrix} Y' \\ Y \end{pmatrix} \sim N\left(\begin{pmatrix} 0 \\ 0 \end{pmatrix}, \begin{pmatrix} k(x',x')+\sigma^2 I_k & k(x',x) \\ k(x,x') & k(x,x)+\sigma^2 I_n \end{pmatrix}\right). \tag{16}$$

The predictive distribution of Y' given $X = x, Y = y, X' = x'$, is then, by (12), normal with mean and covariance given by

$$\begin{aligned} m_{Y'|Y,X,X'}(y,x,x') &= k(x',x)[k(x,x)+\sigma^2 I_n]^{-1}y, \\ k_{Y'|Y,X,X'}(y,x,x') &= k(x',x') + \sigma^2 I_k - k(x',x)[k(x,x)+\sigma^2 I_n]^{-1}k(x,x'). \end{aligned}$$

Recall Exercise 9 where we considered a Bayesian regression problem. In the setting of Gaussian processes, we are interested in predicting Y' given Y, X, X'. Consequently, if each Y_i is d-dimensional and Y' is $d \times k$-dimensional, then using the squared loss

$$L(y', a) = |y' - a|^2,$$

the posterior risk can be expressed as

$$r(\delta \mid y, x, x') = \mathrm{E}[|Y' - \delta(Y, X, X')|^2 \mid Y = y, X = x, X' = x'].$$

The predictive mean $\mathrm{E}[Y' \mid Y = y, X = x, X' = x']$ is a formal Bayes rule. By our calculation above

$$\mathrm{E}[Y' \mid Y = y, X = x, X' = x'] = m_{Y'|Y,X,X'}(y,x,x') = k(x',x)[k(x,x)+\sigma^2 I_n]^{-1}y.$$

Moreover, since we have the entire predictive distribution, we can easily obtain prediction intervals for the new observations Y'.

Exercise 15. Consider the Gaussian process regression $Y_i = f(X_i) + \varepsilon_i$, where f is a Gaussian process with zero mean and covariance function k and $\{\varepsilon_i\}_{i\geq 1}$ are IID $N(0,\sigma^2)$ random variables, independent of $\{X_i\}_{i\geq 1}$ and f. Put $X = (X_1,\dots,X_n)^{\mathrm{T}}$, $Y = (Y_1,\dots,Y_n)^{\mathrm{T}}$. Consider the posterior process $f^{\mathrm{post}} = \{f^{\mathrm{post}}(x')\}_{x'\in\mathcal{X}}$, which is the conditional process f given $Y = y, X = x$. Show that f^{post} is a Gaussian process.

Exercise 16. Consider the Gaussian process regression, as described in this section, and put $X' = X_{n+1}$ and $Y' = Y_{n+1}$. Suppose we have observed $Y = y, X = x, X' = x'$. Determine a 95%-prediction interval for Y'. That is, determine an interval (a,b) such that

$$\Pr(a \leq Y' \leq b \mid Y = y, X = x, X' = x') = 0.95.$$

17.0.1. *Marginal likelihood.* The marginal likelihood is the density of $Y = (Y_1,\dots,Y_n)^{\mathrm{T}}$, conditional on $X = x$, where $X = (X_1,\dots,X_n)^{\mathrm{T}}$. From (14) it follows, after multiplying by $B = (0,1,0)$, that the marginal distribution of Y is Gaussian with zero mean and covariance $k(x,x) + \sigma^2 I_n$, so the log of the marginal likelihood is

$$\log p_{Y|X}(y \mid x) = -\frac{1}{2} y^{\mathrm{T}} (k(x,x) + \sigma^2 I_n)^{-1} y - \frac{1}{2}\log\det(k(x,x) + \sigma^2 I_n) - \frac{n}{2}\log 2\pi.$$

17.1. **Optimizing hyperparameters.** This section will include some of the theory on optimizing hyperparameters by maximizing the evidence (the marginal likelihood) as described in Chapter 5 in Rasmussen and Williams.

Recall the discussion on Bayesian model selection in Section 8.4. Here we repeat the discussion in the context of Gaussian process regression. As we have seen above, to predict new observations $Y' = (Y_{n+1},\dots,Y_{n+k})$ we must evaluate the predictive distribution $p_{Y'|Y,X,X'}(y' \mid y, x, x')$, which is normal with mean $m_{Y'|Y,X,X'}$ and covariance $k_{Y'|Y,X,X'}$. Clearly, the predictive distribution is heavily influenced by the choice of prior, in this case, the choice of the hyperparameters of the Gaussian process f.

The approach of Bayesian model selection is to optimize the hyperparameter by maximizing the probability of the data, given by the *evidence*, which is the marginal likelihood of the data. In the context of Gaussian process regression, this is $p_{Y|X}(y \mid x)$.

To maximize the evidence with respect to the hyperparameters, let's call them ψ, we may equivalently minimize the negative log-evidence

$$-\log p_{Y|X}(y \mid x) = \frac{1}{2} y^{\mathrm{T}} (k(x,x) + \sigma^2 I_n)^{-1} y + \frac{1}{2}\log\det(k(x,x) + \sigma^2 I_n) + \frac{n}{2}\log 2\pi.$$

A gradient descent algorithm is usually employed for this task, although there is in general no guarantee that the negative log-evidence is convex in its parameters. With K denoting the $n \times n$ matrix $K = k(x,x) + \sigma^2 I_n$, ψ the parameters of K, and using the properties that,

$$\frac{\partial K^{-1}}{\partial \psi} = -K^{-1}\frac{\partial K}{\partial \psi}K^{-1},$$

and, if K is a symmetric matrix,

$$\frac{\partial}{\partial \psi}\log\det K = \operatorname{tr}\left(K^{-1}\frac{\partial K}{\partial \psi}\right),$$

it follows that the gradient of the negative log-evidence is given by

$$-\frac{1}{2}y^{\mathrm{T}}K^{-1}\frac{\partial K}{\partial \psi}K^{-1}y+\frac{1}{2}\mathrm{tr}\left(K^{-1}\frac{\partial K}{\partial \psi}\right)=\frac{1}{2}\mathrm{tr}\left((\alpha\alpha^{\mathrm{T}}-K^{-1})\frac{\partial K}{\partial \psi}\right),$$

where $\alpha = K^{-1}y$.

Lecture 11

Literature:

- Rasmussen and Williams, Chapter 3

18. Gaussian processes for classification

Before going into the details of Gaussian process classification, let us recall the general setup of the Bayesian approach to binary classification as in Section 8.1. Suppose we have training data in the form of $\{(x_i, y_i)\}_{i=1}^n$ that are outcomes of independent pairs $\{(X_i, Y_i)\}_{i=1}^n$ where X_i represents the explanatory variables and Y_i the corresponding label. We also have $X_{n+1} = x_{n+1}$ and want to predict the label Y_{n+1}. We have a model of the form

$$\Pr(Y_i = y_i \mid \Theta = \theta, X_i = x_i) = \operatorname{sigm}\Big(y_i f_\theta(x_i)\Big),$$

where $y_i \in \{-1, 1\}$. With $X = (X_1, Y_1, \ldots, X_n, Y_n, X_{n+1})$ and $Y = Y_{n+1}$ we have the likelihood

$$\begin{aligned} p_{X|\Theta}(x \mid \theta) &= p_{Y_1,\ldots,Y_n|\Theta,X_1,\ldots,X_{n+1}}(y_1, \ldots, y_n \mid \theta, x_1, \ldots, x_{n+1}) p_{X_1,\ldots,X_{n+1}}(x_1, \ldots, x_{n+1}) \\ &= \prod_{i=1}^n \operatorname{sigm}\Big(y_i f_\theta(x_i)\Big) \prod_{i=1}^{n+1} p_{X_i}(x_i). \end{aligned}$$

Given a prior p_Θ, the predictive distribution is given by

$$\begin{aligned} p_{Y|X}(y \mid x) &= \int_\Theta p_{Y|\Theta,X}(y \mid \theta, x) p_{\Theta|X}(\theta \mid x) d\theta = \ldots \\ &= \int_\Theta \operatorname{sigm}\Big(y f_\theta(x_{n+1})\Big) p_{\Theta|X_1,Y_1,\ldots,X_n,Y_n}(\theta \mid x_1, y_1, \ldots, x_n, y_n) d\theta \end{aligned}$$

For Gaussian processes we will consider the following alternatives for evaluating the predictive distribution.

- We may approximate the posterior $p_{\Theta|X}(\theta \mid x)$ by a delta-function at its maximum argument. That is, find the maximizer of the posterior distribution, called MAP,

  $$\theta^{\mathrm{MAP}} = \operatorname{argmax}_\theta p_{\Theta|X}(\theta \mid x),$$

  and approximate

  $$\begin{aligned} p_{Y|X}(y \mid x) &= \int_\Theta p_{Y|\Theta,X}(y \mid \theta, x) p_{\Theta|X}(\theta \mid x) d\theta \\ &\approx p_{Y|\Theta,X}(y \mid \theta^{\mathrm{MAP}}, x). \end{aligned}$$

- Approximate the posterior $p_{\Theta|X}(\theta \mid x)$ by a distribution $q_{\Theta|X}(\theta \mid x)$, where $q_{\Theta|X}$ is chosen so that

  $$\int_\Theta p_{Y|\Theta,X}(y \mid \theta, x) q_{\Theta|X}(\theta \mid x) d\theta$$

  can be evaluated. That is, the predictive distribution is approximated by

  $$\begin{aligned} p_{Y|X}(y \mid x) &= \int_\Theta p_{Y|\Theta,X}(y \mid \theta, x) p_{\Theta|X}(\theta \mid x) d\theta \\ &\approx \int_\Theta p_{Y|\Theta,X}(y \mid \theta, x) q_{\Theta|X}(\theta \mid x) d\theta. \end{aligned}$$

- Approximate the predictive distribution by Monte-Carlo simulation by using e.g. Markov chain Monte Carlo (MCMC) to generate samples θ_k, $k = 1, \dots, N$ from the true posterior $p_{\Theta|X}$ and approximate the predictive distribution by the sample average,

$$p_{Y|X}(y \mid x) \approx \frac{1}{N} \sum_{k=1}^{N} p_{Y|\Theta,X}(y \mid \theta_k, x).$$

18.0.1. *Laplace approximation of the posterior.* The general idea of Laplace approximation is to approximate the posterior $p_{\Theta|X}(\theta \mid x)$ by a distribution $q_{\Theta|X}(\theta \mid x)$ where $q_{\Theta|X}$ is a normal distribution with mean θ^{MAP} and covariance matrix A^{-1}. This is achieved by centering $q_{\Theta|X}$ at the maximum of the posterior

$$\theta^{\text{MAP}} = \text{argmax } p_{\Theta|X}(\theta \mid x),$$

and taking the covariance matrix to be A^{-1} where

$$A = -\frac{\partial^2}{\partial \theta^2} \log p_{\Theta|X}(\theta \mid x)\Big|_{\theta=\theta^{\text{MAP}}},$$

which is minus the Hessian of the log-posterior.

Note that, if $p_{\Theta|X}(\theta \mid x)$ has precisely a normal distribution, $N(m, \Sigma)$, then

$$-\log p_{\Theta|X}(\theta \mid x) = \frac{1}{2}(\theta - m)^{\mathrm{T}} \Sigma^{-1} (\theta - m) - \frac{1}{2} \log \det \Sigma - \frac{n}{2} \log 2\pi,$$

which implies that θ^{MAP} solves

$$0 = -\frac{\partial}{\partial \theta} \log p_{\Theta|X}(\theta \mid x) = \Sigma^{-1}(\theta - m),$$

which implies $\theta^{\text{MAP}} = m$ and

$$\begin{aligned} A &= -\frac{\partial^2}{\partial \theta^2} \log p_{\Theta|X}(\theta \mid x)\Big|_{\theta=\theta^{\text{MAP}}} \\ &= \Sigma^{-1}. \end{aligned}$$

Exercise 17. Let $X = \{X_i\}_{i=1}^n$ be conditionally independent $\text{Ber}(\theta)$ distributed given $\Theta = \theta$. Take a prior on Θ, with density proportional to e^θ, i.e. $p_\Theta(\theta) = (e-1)^{-1} e^\theta I\{0 \le \theta \le 1\}$. Let $n_1 \coloneqq \sum_{i=1}^n X_i$, and $n_0 \coloneqq n - n_1$. Provide the Laplace approximation of the posterior $p_{\Theta|X}(\theta \mid x)$ in terms of n_1, n_0. Plot it together with the true (perhaps unnormalized) posterior and comment on its adequacy for $(n_1, n_0) = (3, 7), (1, 1), (1, 0)$ and $(0, 1)$.

18.1. **Binary classification with Gaussian processes.** We consider the binary classification problem. Suppose we have training data in the form of $\{(x_i, y_i)\}_{i=1}^n$ that are outcomes of independent pairs $\{(X_i, Y_i)\}_{i=1}^n$ where X_i represents the explanatory variables and Y_i the corresponding label. We also have $X_{n+1} = x_{n+1}, \dots, X_{n+k} = x_{n+k}$ and want to predict the labels $Y_{n+1}, \dots, Y_{n+k}$. The conditional probabilities are modeled as

$$\Pr(Y_i = y_i \mid f(X_i) = f(x_i)) = \text{sigm}\Big(y_i f(x_i)\Big),$$

where f is a Gaussian process with zero mean and covariance function k.

As in the previous section, on Gaussian process regression, we will write $X = (X_1, \dots, X_n)^{\mathrm{T}}$, $Y = (Y_1, \dots, Y_n)^{\mathrm{T}}$, $X' = (X_{n+1}, \dots, X_{n+k})^{\mathrm{T}}$ and for the unobserved labels $Y' = (Y_{n+1}, \dots, Y_{n+k})^{\mathrm{T}}$. Moreover we write $f(X) = (f(X_1), \dots, f(X_n))^{\mathrm{T}}$,

$f(X') = (f(X'_1), \ldots, f(X'_k))^{\mathrm{T}} = (f(X_{n+1}), \ldots, f(X_{n+k}))^{\mathrm{T}}$ and similarly for the corresponding observations $x = (x_1, \ldots, x_n)^{\mathrm{T}}$ etc. We treat X, Y, X' as the observed variables, that the decision rule (classification) is based on, and Y' as the unobserved. The action space is $\{-1, 1\}^k$. Consider a loss function of the form

$$L(y', a) = \sum_{i=1}^{k} I\{y'_i \neq a_i\}.$$

Exercise 18. Show, similar to Exercise 3, that a formal Bayes rule is given by taking $\delta(y, x, x') = (\delta_1, \ldots, \delta_k)$, where

$$\delta_i = 2I\{p_{Y'_i|Y,X,X'}(1 \mid y, x, x') \geq 1/2\} - 1.$$

From Exercise 18 it follows that it is sufficient to evaluate the predictive class probabilities of Y' conditional on X, Y, X', which will be denoted by $p_{Y'|Y,X,X'}(y' \mid y, x, x')$.

An important distribution for Gaussian process classification is the joint distribution of $Y', f(X'), Y, f(X)$ conditional on X, X'. It plays a similar role as in GP regression. It can be represented as follows: letting $a = (a_i)_{i=1}^n \in \mathbb{R}^n, a' = (a_i)_{i=n+1}^{n+k} \in \mathbb{R}^k$ denote the density variables of $f(x)$ and $f(x')$ respectively,

$$\begin{aligned}
&p_{Y',Y,f(X'),f(X)|X,X'}(y', y, a', a \mid x, x') \\
&\quad = p_{Y',Y|f(X'),f(X)}(y', y \mid a', a) p_{f(X'),f(X)|X,X'}(a', a \mid x, x') \\
&\quad = p_{Y'|f(X'),f(X)}(y' \mid a', a) p_{Y|f(X'),f(X)}(y \mid a', a) \\
&\qquad \times p_{f(X'),f(X)|X,X'}(a', a \mid x, x') \\
&\quad = p_{Y'|f(X')}(y' \mid a') p_{Y|f(X)}(y \mid a) p_{f(X'),f(X)|X,X'}(a', a \mid x, x') \\
&\quad = \prod_{i=n+1}^{n+k} p_{Y_i|f(X_i)}(y_i \mid a_i) \prod_{i=1}^{n} p_{Y_i|f(X_i)}(y_i \mid a_i) \\
&\qquad \times p_{f(X'),f(X)|X,X'}(a', a \mid x, x') \\
&\quad = \prod_{i=1}^{n+k} \mathrm{sigm}(y_i a_i) \times p_{f(X'),f(X)|X,X'}(a', a \mid x, x')
\end{aligned}$$

The predictive distribution can be written as

$$\begin{aligned}
&p_{Y'|Y,X,X'}(y' \mid y, x, x') \\
&\quad = \int p_{Y'|Y,X,f(X')}(y' \mid y, x, a') p_{f(X')|Y,X,X'}(a' \mid y, x, x') da' \\
&\quad = \int \prod_{i=1}^{k} \mathrm{sigm}(y'_i a') p_{f(X')|Y,X,X'}(a' \mid y, x, x') da'.
\end{aligned}$$

To simplify the notation, we will in the sequel not write out the conditioning on X and X', but simply write the predictive density as

$$p_{Y'|Y}(y' \mid y) = \int \prod_{i=1}^{k} \mathrm{sigm}(y'_i a') p_{f(x')|Y}(a' \mid y) da'.$$

In contrast to Gaussian process regression, the conditional class probabilities

$$p_{Y_i|f(X_i)}(y_i \mid a_i) = \mathrm{sigm}(y_i a_i),$$

are not Gaussian. Therefore, neither $p_{f(X')|Y}$ nor the predictive $p_{Y'|Y}$ is Gaussian. Therefore, we will consider approximations of the predictive distribution. In the next section, we discuss the maximum a posteriori and Laplace approximations.

18.1.1. *The MAP and Laplace approximation.* To evaluate the predictive class probabilities, we need the density $p_{f(x')|Y}(f(x') \mid y)$. By integrating out a we find that

$$\begin{aligned} p_{f(x')|Y}(a' \mid y) &= \int p_{f(x')|Y,f(x)}(a' \mid y, a) p_{f(x)|Y}(a \mid y) da \\ &= \int p_{f(x')|f(x)}(a' \mid a) p_{f(x)|Y}(a \mid y) da. \end{aligned}$$

In the following, we will begin by approximating the posterior $p_{f(x)|Y}(a \mid y)$. The maximum of the posterior is given by

$$a^{\mathrm{MAP}} = \mathrm{argmax}_a \, p_{f(x)|Y}(a \mid y).$$

We might as well consider the log-posterior and

$$a^{\mathrm{MAP}} = \mathrm{argmax}_a \log p_{f(x)|Y}(a \mid y). \tag{17}$$

By Bayes' rule,

$$p_{f(x)|Y}(a \mid y) = \frac{p_{Y|f(x)}(y \mid a) p_{f(x)}(a)}{p_Y(y)},$$

and since the marginal distribution $p_Y(y)$ does not depend on a, a^{MAP} is the argmax of

$$\begin{aligned} \Psi(a) &= \log p_{Y|f(x)}(y \mid a) + \log p_{f(x)}(a) \\ &\stackrel{c}{=} \sum_{i=1}^n \log \mathrm{sigm}(y_i a_i) - \frac{1}{2} a^{\mathrm{T}} k(x,x)^{-1} a. \end{aligned}$$

We may compute the gradient and Hessian (w.r.t. $f(x)$) of Ψ as

$$\begin{aligned} G(a) &= \frac{\partial}{\partial a} \Psi(a) = \frac{\partial}{\partial a} \sum_{i=1}^n \log \mathrm{sigm}(y_i a) - k(x,x)^{-1} a, \\ H(a) &= \frac{\partial^2}{\partial a^2} \Psi(a) = \frac{\partial^2}{\partial a^2} \sum_{i=1}^n \log \mathrm{sigm}(y_i a) - k(x,x)^{-1}. \end{aligned}$$

Using that

$$\begin{aligned} \frac{\partial}{\partial t} \log \mathrm{sigm}(t) &= -\frac{\partial}{\partial f(x)} \log(1 + e^{-t}) = \mathrm{sigm}(-t), \\ \frac{\partial^2}{\partial t^2} \log \mathrm{sigm}(t) &= \frac{\partial}{\partial t} \mathrm{sigm}(-t) = -(1 + e^t)^{-2} e^t, \end{aligned}$$

we find that

$$\frac{\partial}{\partial a} \sum_{i=1}^n \log \mathrm{sigm}(y_i a_i)$$

is a vector whose ith element is $y_i \, \mathrm{sigm}(-y_i a_i)$ and

$$\frac{\partial^2}{\partial a^2} \sum_{i=1}^n \log \mathrm{sigm}(y_i a_i)$$

is a diagonal matrix, let us denote it by $-W$ whose diagonal elements are given by

$$W_{ii} = y_i^2(1+e^{y_i a_i})^{-2}e^{y_i a_i} = (1+e^{y_i a_i})^{-2}e^{y_i a_i}.$$

We conclude that

$$\begin{aligned} G(a) &= [y_i \operatorname{sigm}(y_i a_i)]_{i=1}^n - k(x,x)^{-1}a, \qquad \text{[gradient]} \\ H(a) &= -W - k(x,x)^{-1}, \qquad \text{[Hessian]}. \end{aligned}$$

Note that $H(a)$ is (strictly) negative definite, meaning that Ψ is strictly concave. Hence the maximization problem (17) possesses a unique maximum f^{MAP}. To find the a^{MAP} we need to solve $G(a) = 0$, which must be done numerically, either by Newton's method or Robbins-Monro. In Newtons method the iterative scheme for finding f^{MAP} is to start at $a^{(0)}$ and update

$$a^{(k+1)} = a^{(k)} - H(a^{(k)})^{-1}G(a^{(k)}). \tag{18}$$

By approximating $p_{f(x)|Y}(a \mid y)$ by a delta-mass at a^{MAP} it follows that $p_{f(x')|Y}(a' \mid y)$ is approximated by

$$\begin{aligned} p_{f(x')|Y}(a' \mid y) &= \int p_{f(x')|Y,f(x)}(a' \mid y, a)p_{f(x)|Y}(a \mid y)da \\ &\approx p_{f(x')|f(x)}(a' \mid a^{\text{MAP}}), \end{aligned} \tag{19}$$

which is a Gaussian distribution with mean $k(x',x)k(x,x)^{-1}a^{\text{MAP}}$ and covariance $k(x',x') - k(x',x)k(x,x)^{-1}k(x,x')$.

Similarly, by approximating $p_{f(x)|Y}(a \mid y)$ with at Gaussian distribution, denoted $q_{f(x)|Y}(a \mid y)$, centered at a^{MAP} with covariance $-H(a^{\text{MAP}})^{-1}$ it follows that $p_{f(x')|Y}(a' \mid y)$ is approximated by

$$p_{f(x')|Y}(a' \mid y) \approx \int p_{f(x')|Y,f(x)}(a' \mid y, a)q_{f(x)|Y}(a \mid y)da. \tag{20}$$

Using the approximation (20) the expectation $\mathrm{E}[f(x') \mid Y = y]$ is approximated by

$$\begin{aligned} \mathrm{E}[f(x') \mid Y = y] &= \int a' p_{f(x')|Y}(a' \mid y)da' \\ &\approx \int a' \left[\int p_{f(x')|Y,f(x)}(a' \mid y, a)q_{f(x)|Y}(a \mid y)da \right] da' \\ &= \int \left[\int a' p_{f(x')|Y,f(x)}(a' \mid y, a)da' \right] q_{f(x)|Y}(a \mid y)da \\ &= \int k(x',x)k(x,x)^{-1}a q_{f(x)|Y}(a \mid y)da \\ &= k(x',x)k(x,x)^{-1}a^{\text{MAP}}. \end{aligned}$$

Moreover, since both $p_{f(x')|Y,f(x)}(a' \mid y, a)$ and $q_{f(x)|Y}(a \mid y)$ are Gaussian it follows that the joint distribution of $(f(x'), f(x)) \mid Y = y$ under the approximate density $p_{f(x')|Y,f(x)}(a' \mid y, a)q_{f(x)|Y}(a \mid y)$ is Gaussian with mean m and covariance Σ given by (see Exercise 19 below)

$$\begin{pmatrix} m_1 \\ m_2 \end{pmatrix} = \begin{pmatrix} k(x',x)k(x,x)^{-1}a^{\text{MAP}} \\ a^{\text{MAP}} \end{pmatrix}, \qquad \Sigma = \begin{pmatrix} \Sigma_{11} & \Sigma_{12} \\ \Sigma_{21} & \Sigma_{22} \end{pmatrix},$$

where

$$\begin{aligned}
\Sigma_{11} &= k(x',x') - k(x',x)k(x,x)^{-1}k(x,x') \\
&\quad + k(x',x)k(x,x)^{-1}\underbrace{[-H(a^{\text{MAP}})]^{-1}}_{\substack{=[W+k(x,x)^{-1}]^{-1} \\ =k(x,x)-k(x,x)(W^{-1}+k(x,x))^{-1}k(x,x)}}k(x,x)^{-1}k(x,x') \\
&= k(x',x') - k(x',x)(W^{-1}+k(x,x))^{-1}k(x,x') \\
\Sigma_{12} &= k(x',x)k(x,x)^{-1}[-H(a^{\text{MAP}})]^{-1}, \\
\Sigma_{21} &= [-H(a^{\text{MAP}})]^{-1}k(x,x)^{-1}k(x,x'), \\
\Sigma_{22} &= [-H(a^{\text{MAP}})]^{-1}.
\end{aligned}$$

Here, for Σ_{11}, we used the (Woodbury) matrix identity

$$(A+C)^{-1} = A^{-1} - A^{-1}(C^{-1}+A^{-1})^{-1}A^{-1}.$$

The approximate distribution for $f(x') \mid Y = y$, is found by taking the marginal distribution of $f(x')$ in this Gaussian distribution which implies that $p_{f(x')|Y}(a' \mid y)$, under the Laplace approximation, is approximately Gaussian with mean m_1 and covariance Σ_{11} as above. Let us denote this Gaussian density by $q_{f(x')|Y}(a' \mid y)$.

To perform classification based on Gaussian processes, we must now evaluate the predictive class probabilities

$$p_{Y'|Y}(y' \mid y) = \int \prod_{i=1}^{k} \text{sigm}(y_i' a') p_{f(x')|Y}(a' \mid y) da'.$$

These are evaluated, either by approximating $p_{f(x')|Y}(a' \mid y)$ by a delta-mass at $k(x',x)k(x,x)^{-1}a^{\text{MAP}}$, or by computing the averaged probabilities using the Gaussian approximation for $p_{f(x')|Y}(a' \mid y)$.

To summarize, we have two methods for computing the predictive class probabilities.

(1) [MAP] The predictive class probabilities are approximated by

$$\prod_{i=1}^{k} \text{sigm}(y_i'[k(x',x)k(x,x)^{-1}a^{\text{MAP}}]_i),$$

where a^{MAP} is found numerically by iterating (18).

(2) [Laplace approximation] The predictive class probabilities are approximated by

$$p_{Y'|Y}(y' \mid y) = \int \prod_{i=1}^{k} \text{sigm}(y_i' a_i') q_{f(x')|Y}(a' \mid y) da',$$

where $q_{f(x')|Y}(a' \mid y)$ is Gaussian with mean $k(x',x)k(x,x)^{-1}a^{\text{MAP}}$ and covariance $k(x',x') - k(x',x)(W^{-1}+k(x,x))k(x,x')$. This integral may be computed by numerical integration, or, if $k=1$, it may be analytically approximated using that $\text{sigm}(t) \approx \Phi(\sqrt{\frac{\pi}{8}}t)$, where Φ is the cumulative distribution function of the standard normal distribution.

Exercise 19. Let X_1 and X_2 be random vectors of length d_1 and d_2, respectively and such that $p_{X_1|X_2}(x_1 \mid x_2)$ is a normal density with mean Bx_2 and covariance matrix C, and $p_{X_2}(x_2)$ is a normal density with mean μ and covariance matrix A.

It is assumed that B, C, μ, and A are known. Show that the joint distribution of (X_1, X_2) is normal with mean, m, and covariance Σ given by

$$\begin{pmatrix} m_1 \\ m_2 \end{pmatrix} = \begin{pmatrix} B\mu \\ \mu \end{pmatrix}, \qquad \Sigma = \begin{pmatrix} C + BAB^{\mathrm{T}} & BA \\ B^{\mathrm{T}}A^{\mathrm{T}} & A \end{pmatrix}.$$

Exercise 20. Use Exercise 19 to apply the same program for GP **regression**, i.e. get $f(x)|Y = y$ first (analytically), then $f(x')|Y = y$, and finally $Y'|Y = y$, for $Y_i = f(x_i) + \varepsilon_i$ as in Section 17. What do you expect to see?

Lecture 12

Literature:

- Sutton and Barto, Chapters 3 and 4. Available at: `http://incompleteideas.net/book/RLbook2020.pdf`

19. An introduction to Markov decision processes and reinforcement learning

19.1. **Markov decision processes.** Let us first describe the setup of a Markov decision process (MDP). We consider a finite state-space $\mathcal{S} = \{s_1, \dots, s_n\}$. In each state $s \in \mathcal{S}$ the agent can take actions in a finite set $\mathcal{A}(s)$, where the available actions depend on the state s. If, at time $t \geq 0$, the agent is in state $S_t = s \in \mathcal{S}$ and picks action $A_t = a \in \mathcal{A}(s)$ the state S_{t+1} and reward R_{t+1} are updated according to the probability distribution

$$p(s', r \mid s, a) = \Pr(S_{t+1} = s', R_{t+1} = r \mid S_t = s, A_t = a).$$

From this probability distribution, we can express the expected reward for an action $A_t = a$ as

$$r(s, a) = \mathrm{E}[R_{t+1} \mid S_t = s, A_t = a] = \sum_{s', r} r p(s', r \mid s, a),$$

and the transition probability

$$p(s' \mid s, a) = \Pr(S_{t+1} = s' \mid S_t = s, A_t = a) = \sum_{r} p(s', r \mid s, a).$$

A sequence $\pi = (\pi_t)_{t \geq 0}$ where, for each $t \geq 0$, $\pi_t(a \mid s)$ is the probability to take action a, when in state s at time t, is called a *policy*. An agent starting at time t in state s will select its action at time t according to the probability $\pi_t(\cdot \mid s)$. If the outcome is a, then the reward, r, and new state at time $t+1$, s', is sampled from $p(s', r \mid s, a)$ and from there the agent continues to select the action at $t+1$ according to $\pi_{t+1}(\cdot \mid s')$ etc. A policy which does not depend on time $\pi_0 = \pi_1 = \dots$ is called a *stationary policy*. Throughout these lecture notes we will only consider stationary policies.

The *state value* of a policy π is given by the expected future discounted rewards

$$v_\pi(s) = \mathrm{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \;\middle|\; S_t = s \right],$$

where $0 < \gamma < 1$ is the discount factor. The corresponding *action value* is given by

$$q_\pi(s, a) = \mathrm{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \;\middle|\; S_t = s, A_t = a \right].$$

The random sum $G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$ is known as the *return* at time t.

19.1.1. *One-step-ahead conditioning and the Bellman equation.* A useful strategy when doing computations with Markov chains is to condition on the next state, as

this gives rise to useful equations. Consider the state value of a stationary policy π:

$$\begin{aligned}
v_\pi(s) &= \mathrm{E}_\pi\left[\sum_{k=0}^{\infty}\gamma^k R_{t+k+1} \;\middle|\; S_t = s\right] \\
&= \mathrm{E}_\pi\left[R_{t+1} + \gamma\sum_{k=0}^{\infty}\gamma^k R_{t+1+k+1} \;\middle|\; S_t = s\right] \\
&= \mathrm{E}_\pi\left[\mathrm{E}_\pi\left[R_{t+1} + \gamma\sum_{k=0}^{\infty}\gamma^k R_{t+1+k+1} \;\middle|\; R_{t+1}, S_{t+1}\right] \;\middle|\; S_t = s\right] \\
&= \mathrm{E}_\pi\left[R_{t+1} + \gamma\,\mathrm{E}_\pi\left[\sum_{k=0}^{\infty}\gamma^k R_{t+1+k+1} \;\middle|\; S_{t+1}\right] \;\middle|\; S_t = s\right] \\
&= \mathrm{E}_\pi\left[R_{t+1} + \gamma v_\pi(S_{t+1}) \mid S_t = s\right].
\end{aligned}$$

We conclude that the state value of a stationary policy π is the solution to the equation

$$v_\pi(s) = \mathrm{E}_\pi\left[R_{t+1} + \gamma v_\pi(S_{t+1}) \mid S_t = s\right]. \tag{21}$$

This is the *Bellman equation* (it also has a version for the action values q).

For a policy π, let $T_\pi : \mathbb{R}^{|\mathcal{S}|} \to \mathbb{R}^{|\mathcal{S}|}$ be the linear operator defined by

$$(T_\pi v)(s) = \sum_a \Big(\sum_{s',r}(r + \gamma v(s'))p(s', r \mid s, a)\Big)\pi(a \mid s).$$

The Bellman equation (21) can be written as the fixed point equation $v_\pi = T_\pi v_\pi$. Moreover, we will see later that T_π is a contractive operator (called the Bellman operator), with fixed point v_π. Hence, evaluating the value function for a policy can be approached by solving a fixed-point equation.

19.1.2. *Bellman's optimality equation.* The optimal state value is $v_*(s) = \sup_\pi v_\pi(s)$, where the supremum is taken over all stationary policies. A policy π_* is called optimal if $v_*(s) = v_{\pi_*}(s)$ for every $s \in \mathcal{S}$. The optimal action value is $q_*(s, a) = \sup_\pi q_\pi(s, a)$. Similar to the Bellman equation (21), we can derive an equation for the optimal value function. The *Bellman optimality equation* is

$$v_*(s) = \sup_{a\in\mathcal{A}(s)} \mathrm{E}\left[R_{t+1} + \gamma v_*(S_{t+1}) \mid S_t = s, A_t = a\right]. \tag{22}$$

Note that the right-hand side can also be written as

$$\sup_{a\in\mathcal{A}(s)}\left[r(s, a) + \gamma\sum_{s'} v_*(s')p(s' \mid s, a)\right].$$

To prove it, let us first show that

$$v_*(s) = \sup_{a\in\mathcal{A}(s)} q_*(s, a). \tag{23}$$

For any fixed action $a \in \mathcal{A}(s)$ we have

$$v_*(s) = \sup_\pi v_\pi(s) \geq \sup_\pi q_\pi(s, a) = q_*(s, a).$$

Taking supremum over $a \in \mathcal{A}(s)$ gives the inequality

$$v_*(s) \geq \sup_{a \in \mathcal{A}(s)} q_*(s,a).$$

The reverse inequality follows from,

$$v_\pi(s) = \sum_a q_\pi(s,a)\pi(a \mid s) \leq \sum_a q_*(s,a)\pi(a \mid s) \leq \sup_{a \in \mathcal{A}(s)} q_*(s,a),$$

for any policy π. Taking supremum over π yields

$$v_*(s) \leq \sup_{a \in \mathcal{A}(s)} q_*(s,a).$$

Armed with (23) we proceed to derive Bellman's optimality equation, using the one-step-ahead conditioning:

$$\begin{aligned}
v_*(s) &= \sup_{a \in \mathcal{A}(s)} q_*(s,a) \\
&= \sup_{a \in \mathcal{A}(s)} \sup_\pi \mathrm{E}\left[\sum_{k=0}^\infty \gamma^k R_{t+k+1} \mid S_t = s, A_t = a\right] \\
&= \sup_{a \in \mathcal{A}(s)} \sup_\pi \mathrm{E}\left[R_{t+1} + \gamma v_\pi(S_{t+1}) \mid S_t = s, A_t = a\right] \\
&= \sup_{a \in \mathcal{A}(s)} \sup_\pi \sum_{s',r} (r + \gamma v_\pi(s'))p(s', r \mid s, a) \\
&= \sup_{a \in \mathcal{A}(s)} \sum_{s',r} (r + \gamma \sup_\pi v_\pi(s'))p(s', r \mid s, a) \\
&= \sup_{a \in \mathcal{A}(s)} r(s,a) + \gamma \sum_{s'} v_*(s')p(s' \mid s, a).
\end{aligned}$$

In the fifth equality we have used

$$\sup_\pi \sum_{s',r} (r + \gamma v_\pi(s'))p(s', r \mid s, a) = \sum_{s',r} (r + \gamma \sup_\pi v_\pi(s'))p(s', r \mid s, a),$$

which requires a proof. On the one hand we have, for any policy π,

$$\sum_{s',r} (r + \gamma v_\pi(s'))p(s', r \mid s, a) \leq \sum_{s',r} (r + \gamma v_*(s'))p(s', r \mid s, a),$$

so taking the supremum over π on both sides gives the inequality

$$\sup_\pi \sum_{s',r} (r + \gamma v_\pi(s'))p(s', r \mid s, a) \leq \sum_{s',r} (r + \gamma v_*(s'))p(s', r \mid s, a).$$

To prove the reverse inequality, take an arbitrary $\varepsilon > 0$ and select a policy π_ε such that $v_{\pi_\varepsilon}(s') + \varepsilon \geq v_*(s')$ for each $s' \in \mathcal{S}$. Then,

$$\begin{aligned}
\gamma\varepsilon + \sum_{s',r} (r + \gamma v_{\pi_\varepsilon}(s'))p(s', r \mid s, a) &= \sum_{s',r} (r + \gamma(\varepsilon + v_{\pi_\varepsilon}(s')))p(s', r \mid s, a) \\
&\geq \sum_{s',r} (r + \gamma v_*(s'))p(s', r \mid s, a).
\end{aligned}$$

It follows that

$$\begin{aligned}\gamma\varepsilon + \sup_\pi \sum_{s',r}(r+\gamma v_\pi(s'))p(s',r \mid s,a) &\geq \gamma\varepsilon + \sum_{s',r}(r+\gamma v_{\pi_\varepsilon}(s'))p(s',r \mid s,a)\\ &\geq \sum_{s',r}(r+\gamma v_*(s'))p(s',r \mid s,a),\end{aligned}$$

and since $\varepsilon > 0$ was arbitrary

$$\sup_\pi \sum_{s',r}(r+\gamma v_\pi(s'))p(s',r \mid s,a) \geq \sum_{s',r}(r+\gamma v_*(s'))p(s',r \mid s,a).$$

This completes the proof.

Similarly, we also have the Bellman optimality equation for the action value:

$$\begin{aligned}q_*(s,a) &= \mathrm{E}\left[R_{t+1} + \gamma v_*(S_{t+1}) \mid S_t = s, A_t = a\right]\\ &= \mathrm{E}\left[R_{t+1} + \gamma \sup_{a'} q_*(S_{t+1},a') \mid S_t = s, A_t = a\right].\end{aligned}$$

Defining the operator $T_* : \mathbb{R}^{|\mathcal{S}|} \to \mathbb{R}^{|\mathcal{S}|}$ by the right-hand side of the Bellman equation, i.e.

$$(T_* v)(s) = \sup_{a\in\mathcal{A}(s)} \mathrm{E}\left[R_{t+1} + \gamma v_*(S_{t+1}) \mid S_t = s, A_t = a\right], \tag{24}$$

the Bellman optimality equation can be written as $T_* v_* = v_*$. Note that, similar to the Bellman equation, Bellman's optimality equation can be interpreted as a fixed point equation, and fixed point methods may be considered for solving it; indeed, like T_π, T_* is also contractive, as we will also see.

19.2. **An example playing Tic-Tac-Toe.** This example is inspired by Section 1.5 in the book by Sutton and Barto. See there for more insights.

Consider playing the game Tic-Tac-Toe against an opponent with a somewhat imperfect strategy. At each of your turns, the state s consists of the information of which tiles are marked by you and your opponent's symbols, respectively. $\mathcal{A}(s)$ consists of the possible moves you can make, i.e. the tiles you can choose to mark. Hence, $|\mathcal{A}(s)|$ is equal to the number of free tiles. After you choose an action $a \in \mathcal{A}(s)$, assuming the game does not end, the opponent will make a move, informed by your action a and the rest of the board, s. The opponent's move is unknown and outside your control, and would perhaps not be the same in a repeated scenario, so we see it as random but, of course, influenced by (s,a). Hence, it generates the next state s', and also a reward r, according to a probability distribution $p(s',r \mid s,a)$. This function is the *dynamics* of the game. If the opponent (or you in your turn) makes a winning move or marks the last tile, the game is over, meaning the game goes to a terminal state $s' = \dagger$. Then you get the reward $+1$ if you win, -1 if you lose and 0 for draws. For all transitions $s \to s'$ that are not game ending (i.e. $s' \neq \dagger$), the reward is 0.

If we have a good understanding of our opponent's psychology or have watched them play for long enough, we might be able to specify or approximate $p(s',r \mid s,a)$, which gives us a model for the environment. For any policy π, we can then evaluate its value function v_π with policy evaluation. We can also learn an optimal policy π_* with policy iteration, or directly learn v_* with value iteration.

Lecture 13

Literature:

- Sutton and Barto, Chapters 3 and 4.

20. Markov decision problems and policy evaluation, improvement, and iteration

Let us recall the setup of a Markov decision process (MDP). We consider a finite state-space $\mathcal{S} = \{s_1, \dots, s_n\}$. In each state $s \in \mathcal{S}$ the agent can take actions in a finite set $\mathcal{A}(s)$, where the available actions depend on the state s. If, at time $t \geq 0$, the agent is in state $S_t = s \in \mathcal{S}$ and picks action $A_t = a \in \mathcal{A}(s)$ the state S_{t+1} and reward R_{t+1} are updated according to the probability distribution

$$p(s', r \mid s, a) = \Pr(S_{t+1} = s', R_{t+1} = r \mid S_t = s, A_t = a).$$

From this probability distribution we can express the expected reward for an action $A_t = a$ as

$$r(s, a) = \mathrm{E}[R_{t+1} \mid S_t = s, A_t = a] = \sum_{s', r} r p(s', r \mid s, a),$$

and the transition probability

$$p(s' \mid s, a) = \Pr(S_{t+1} = s' \mid S_t = s, A_t = a) = \sum_r p(s', r \mid s, a).$$

A sequence $\pi = (\pi_t)_{t \geq 0}$ where, for each $t \geq 0$, $\pi_t(a \mid s)$ is the probability to take action a, when in state s at time t, is called a *policy*. An agent starting at time t in state s will select its action at time t according to the probability $\pi_t(\cdot \mid s)$. If the outcome is a, then the reward, r, and new state at time $t+1$, s', is sampled from $p(s', r \mid s, a)$ and from there the agent continues to select the action at $t+1$ according to $\pi_{t+1}(\cdot \mid s')$ etc. A policy which does not depend on time $\pi_0 = \pi_1 = \dots$ is called a *stationary policy*.

The *state value* of a policy π is given by the expected future discounted rewards

$$v_\pi(s) = \mathrm{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \mid S_t = s \right],$$

where $0 < \gamma < 1$ is the discount factor. The corresponding *action value* is given by

$$q_\pi(s, a) = \mathrm{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \mid S_t = s, A_t = a \right].$$

We concluded previously that the state value of a stationary policy π is the solution to the *Bellman equation*

$$v_\pi(s) = \mathrm{E}_\pi \left[R_{t+k+1} + \gamma v_\pi(S_{t+1}) \mid S_t = s \right]. \tag{25}$$

The optimal state value is $v_*(s) = \sup_\pi v_\pi(s)$, where the supremum is taken over all stationary policies. A policy π_* is called optimal if $v_*(s) = v_{\pi_*}(s)$ for every $s \in \mathcal{S}$. The optimal action value is $q_*(s, a) = \sup_\pi q_\pi(s, a)$. Similarly to the Bellman equation (21), we derived an equation for the optimal value function. The *Bellman optimality equation* is

$$v_*(s) = \sup_{a \in \mathcal{A}(s)} \mathrm{E} \left[R_{t+1} + \gamma v_*(S_{t+1}) \mid S_t = s, A_t = a \right]. \tag{26}$$

Note that the right-hand side can also be written as

$$\sup_{a\in\mathcal{A}(s)}\left[r(s,a)+\gamma\sum_{s'}v_*(s')p(s'\mid s,a)\right].$$

Similarly, we have the Bellman optimality equation for the action value

$$\begin{aligned}q_*(s,a)&=\mathrm{E}\left[R_{t+1}+\gamma v_*(S_{t+1})\mid S_t=s,A_t=a\right]\\&=\mathrm{E}\left[R_{t+1}+\gamma\sup_{a'}q_*(S_{t+1},a')\mid S_t=s,A_t=a\right]\end{aligned}$$

20.1. **Policy evaluation.** For a policy π, recall the Bellman equation (21) for computing the state value

$$\begin{aligned}v_\pi(s)&=\mathrm{E}_\pi\left[R_{t+1}+\gamma v_\pi(S_{t+1})\mid S_t=s\right]\\&=\sum_a\Big(\sum_{s',r}(r+\gamma v_\pi(s'))p(s',r\mid s,a)\Big)\pi(a\mid s)\end{aligned}$$

If the dynamics $p(s',r\mid s,a)$ are completely known, then computing v_π by solving the Bellman equation is a matter of solving a system of $|\mathcal{S}|$ linear equations.

Here, we will consider iterative methods. For a policy π, let $T_\pi:\mathbb{R}^{|\mathcal{S}|}\to\mathbb{R}^{|\mathcal{S}|}$ be the linear operator defined by

$$T_\pi v(s)=\sum_a\Big(\sum_{s',r}(r+\gamma v(s'))p(s',r\mid s,a)\Big)\pi(a\mid s). \tag{27}$$

The Bellman equation can be written as the fixed point equation $v_\pi=T_\pi v_\pi$. Moreover, T_π is a contractive operator (called the Bellman operator, see exercise below), with fixed point v_π, which implies that for any initial approximation v_0 the updates

$$v_k=T_\pi v_{k-1},\quad k\geq 1, \tag{28}$$

converges to v_π as $k\to\infty$. We refer to (28) as *policy evaluation*.

Exercise 21. Recall that $T:\mathbb{R}^{|\mathcal{S}|}\to\mathbb{R}^{|\mathcal{S}|}$ is a contraction if $d(Tv_1,Tv_0)\leq\gamma d(v_1,v_0)$ for all $v_0,v_1\in\mathbb{R}^{|\mathcal{S}|}$. Show that, with $\gamma\in(0,1)$ the operator T_π in (27) is a contraction with respect to the L_∞-metric (sup-norm) given by

$$d_1(v_1,v_0)=|v_1-v_0|_\infty=\sup_{s\in\mathcal{S}}|v_1(s)-v_0(s)|.$$

20.2. **Policy improvement.** Take a policy π, with state value v_π. Suppose π' is a policy improvement such that, for all $s\in\mathcal{S}$,

$$T_{\pi'}v_\pi(s)=\sum_a q_\pi(s,a)\pi'(a\mid s)\geq v_\pi(s). \tag{29}$$

Further, one can show an inductive statement: for any k, if $T_{\pi'}^{(k-1)}v_\pi\leq T_{\pi'}^{(k)}v_\pi$, then $T_{\pi'}^{(k)}v_\pi\leq T_{\pi'}^{(k+1)}v_\pi$, since if the former holds, then for all $s\in\mathcal{S}$,

$$\begin{aligned}(T_{\pi'}^{(k)}v_\pi)(s)&=\mathrm{E}_{\pi'}[R_{t+1}+\gamma(T_{\pi'}^{(k-1)}v_\pi)(S_{t+1})\mid S_t=s]\\&\leq\mathrm{E}_{\pi'}[R_{t+1}+\gamma(T_{\pi'}^{(k)}v_\pi)(S_{t+1})\mid S_t=s]=(T_{\pi'}^{(k+1)}v_\pi)(s).\end{aligned} \tag{30}$$

By induction, since the statement holds for $k=0$ by assumption, $v_\pi=T_{\pi'}^{(0)}v_\pi=T_{\pi'}^{(1)}v_\pi\leq\cdots\leq T_{\pi'}^{(k)}v_\pi$. Since $T_{\pi'}^{(k)}v_\pi\to v_{\pi'}$ by contractility, we conclude that $v_\pi\leq v_{\pi'}$, i.e.,

$$v_\pi(s) \le v_{\pi'}(s), \tag{31}$$

for each $s \in \mathcal{S}$. Thereby, the criterion in (29) is sufficient for saying that π' is a better policy than π. Next, we will see how to choose π' such that (29) and then (31) hold.

20.3. **Policy iteration.** By alternating *policy improvement* and *policy evaluation*, we can construct a sequence of gradually improving policies. That is, starting from π_0 and v_0, let, for any $k \ge 1$, π_k be a policy improvement such that

$$T_{\pi_k} v_{\pi_{k-1}} \ge v_{\pi_{k-1}}.$$

Then compute v_{π_k} (policy evaluation) by iterating over i,

$$v_i = T_{\pi_i} v_{i-1},$$

until convergence, starting at e.g. $v_0 = v_{\pi_{k-1}}$.

A good proposal for policy improvement is to take a *greedy* policy

$$\pi'(s) = \text{argsup}_{a\in\mathcal{A}(s)} \sum_{s',r} (r + \gamma v_\pi(s'))p(s', r \mid s, a).$$

It is guaranteed to give a policy improvement unless

$$v_{\pi'}(s) = \sup_{a\in\mathcal{A}(s)} \sum_{s',r} (r + \gamma v_{\pi'}(s'))p(s', r \mid s, a),$$

which is the Bellman optimality equation, in which case no further improvement is possible. Then, alternating between policy improvement and policy evaluation leads to a sequence of increasingly better strategies. That is, start an initial guess for the value function v_0 and iterate:

$$\begin{aligned} \pi_k(s) &= \text{argsup}_{a\in\mathcal{A}(s)} \sum_{s',r} (r + \gamma v_{k-1}(s'))p(s', r \mid s, a) \quad \text{for all } s \in \mathcal{S}, \\ v_i &= T_{\pi_k} v_{i-1}, \text{ iterated over } i \text{ in a sub-loop until convergence,} \end{aligned} \tag{32}$$

for $k \ge 1$. We refer to (32) as *policy iteration*.

20.4. **Value iteration.** By defining the operator $T_* : \mathbb{R}^{|\mathcal{S}|} \to \mathbb{R}^{|\mathcal{S}|}$ as the contractive operator

$$T_* v(s) = \sup_{a\in\mathcal{A}(s)} \Big(\sum_{s',r} (r + \gamma v(s'))p(s', r \mid s, a) \Big),$$

we see that v_* solves the fixed point equation $v = T_* v$. For any initial guess v_0 we can do a *value iteration* by

$$v_k = T_* v_{k-1}, \quad k \ge 1,$$

which converges to v_*; $\lim_{k\to\infty} v_k = v_*$. Given v_* we can define an optimal policy as

$$\pi_*(s) = \text{argsup}_{a\in\mathcal{A}(s)} \sum_{s',r} (r + \gamma v_*(s'))p(s', r \mid s, a),$$

provided that the supremum is attained.

Literature:

- Sutton and Barto, Chapter 5.

21. Monte Carlo methods in reinforcement learning

When dealing with Monte Carlo methods in reinforcement learning, we will consider episodic tasks, that is, tasks that terminate in finite time (an episode) and then can be restarted. The typical example is playing a game until the game terminates (an episode) and then restarting to play the game again (a new episode). To describe this situation we add a terminal state † to the state space such that the action set at † is a single null action (i.e. no choice), $\mathcal{A}(\dagger) = \{\emptyset\}$, and $\Pr(S_{t+1} = \dagger, R_{t+1} = 0 \mid S_t = \dagger, A_t = \emptyset) = 1$. Let

$$\tau = \inf\{t \geq 0 : S_t = \dagger\}$$

be the random terminal time. We assume that τ is bounded above by some $T > 0$ and call the times $t = 0, \ldots, \tau$ an episode.

The state value and action values are given by

$$v_\pi(s) = \mathrm{E}_\pi\left[\sum_{k=0}^{\tau-1} \gamma^k R_{k+1} \mid S_0 = s\right], \text{ and}$$
$$q_\pi(s,a) = \mathrm{E}_\pi\left[\sum_{k=0}^{\tau-1} \gamma^k R_{k+1} \mid S_0 = s, A_0 = a\right],$$

respectively, where $\gamma > 0$ is the discount factor. Since the sum has finitely many terms with probability 1 by assumption, it is not necessary to assume $\gamma < 1$.

In contrast to the previous section, we do not assume that the transition probabilities $p(s', r \mid s, a)$ are known, but only that we can sample from $p(s', r \mid s, a)$. We will begin by describing policy evaluation using Monte Carlo. That is, the calculation of v_π and q_π.

Given a policy π and generating independent episodes $S_1^{(i)}, R_1^{(i)}, \ldots, S_{\tau^{(i)}}^{(i)}, R_{\tau^{(i)}}^{(i)}$, $i = 1, \ldots, N$, the state value and action value can be estimated by sample averages. More precisely, for the ith episode, let

$$\tau_s^{(i)} = \inf\{t \geq 0, S_t^{(i)} = s\}$$

be the first visit to s in episode i and $\tau_s^{(i)} = \infty$ if $S_t \neq s$ for $t = 0, \ldots, \tau^{(i)} - 1$. Let

$$N(s) = \sum_{i=1}^{N} I\{\tau_s^{(i)} < \tau^{(i)}\}$$

be the number of episodes that visit the state s and estimate v_π by

$$\hat{v}_\pi(s) = \frac{1}{N(s)} \sum_{i=1}^{N} I\{\tau_s^{(i)} < \tau^{(i)}\} \left[\sum_{k=0}^{\tau^{(i)} - \tau_s^{(i)} - 1} \gamma^k R^{(i)}_{\tau_s^{(i)}+k+1}\right].$$

Similarly, we may estimate action values. With

$$\tau_{s,a}^{(i)} = \inf\{t \geq 0, S_t = s, A_t = a\},$$

and

$$N(s,a) = \sum_{i=1}^{N} I\{\tau_{s,a}^{(i)} < \tau^{(i)}\},$$

we can estimate $q_\pi(s,a)$ by

$$\hat{q}_\pi(s,a) = \frac{1}{N(s,a)} \sum_{i=1}^{N} I\{\tau_{s,a}^{(i)} < \tau^{(i)}\} \left[\sum_{k=0}^{\tau^{(i)} - \tau_{s,a}^{(i)} - 1} \gamma^k R^{(i)}_{\tau_{s,a}^{(i)}+k+1} \right].$$

Some care is needed here. For instance, the number of combinations (s,a) with $s \in \mathcal{S}$, $a \in \mathcal{A}(s)$, can be very large, and there is no guarantee that a particular (s,a) will ever be visited. For instance, if π is deterministic and we always start from $S_0 = s_0$ it may not be possible to reach (s,a). Therefore, we must assume that the initial value of an episode has a chance to take any combination (s,a) with positive probability. We call this assumption *exploring starts*. Then, by the law of large numbers, as $N \to \infty$, $\hat{v}_\pi$ and $\hat{q}_\pi$ converge to v_π and q_π, respectively.

When computing the estimates $\hat{v}_\pi$ and $\hat{q}_\pi$, it is convenient to note that the running arithmetic mean of some numbers $x_1, x_2, \ldots,$

$$m_n = \frac{1}{n} \sum_{i=1}^{n} x_i,$$

can be updated as

$$m_n = m_{n-1} + \frac{1}{n}(x_n - m_{n-1}).$$

Let $\hat{v}_\pi^{(i-1)}(s)$, $\hat{q}_\pi^{(i-1)}(s,a)$ be the estimates after $i-1$ episodes, $N_i(s), N_i(s,a)$ the number of the first i episodes that visits s and (s,a), respectively, and

$$G^{(i)}_{\tau_s^{(i)}} = \sum_{k=0}^{\tau^{(i)} - \tau_s^{(i)} - 1} \gamma^k R^{(i)}_{\tau_s^{(i)}+k+1},$$
$$G^{(i)}_{\tau_{s,a}^{(i)}} = \sum_{k=0}^{\tau^{(i)} - \tau_{s,a}^{(i)} - 1} \gamma^k R^{(i)}_{\tau_{s,a}^{(i)}+k+1},$$

the returns following a (first) visit to s and (s,a), respectively, in the ith episode. Then, we can use the incremental updates

$$\hat{v}_\pi^{(i)}(s) = \hat{v}^{(i-1)}(s) + \frac{1}{N_i(s)}(G^{(i)}_{\tau_s^{(i)}} - \hat{v}_\pi^{(i-1)}(s)), \tag{33}$$
$$\hat{q}_\pi^{(i)}(s,a) = \hat{q}_\pi^{(i-1)}(s,a) + \frac{1}{N_i(s,a)}(G^{(i)}_{\tau_{s,a}^{(i)}} - \hat{q}_\pi^{(i-1)}(s,a)),$$

for every s and (s,a), respectively, that appears in the ith episode.

21.1. **Policy improvement.** Similarly to policy improvement in the previous section, we may perform policy improvement based on the current Monte Carlo estimate of the action value $\hat{q}_{i-1}$. Assuming exploring starts, we may update greedily:

$$\pi_i(s) = \text{argmax}_{a \in \mathcal{A}(s)} \hat{q}_{i-1}(s,a),$$

and, proceed by generating a new episode $S_1^{(i)}, R_1^{(i)}, \dots, S_{\tau^{(i)}}^{(i)}, R_{\tau^{(i)}}^{(i)}$ using π_i. Then, the action value estimates are updated as

$$\hat{q}_i(s,a) = \hat{q}_{i-1}(s,a) + \frac{1}{N_i(s,a)}(G^{(i)}_{\tau^{(i)}_{s,a}} - \hat{q}_{i-1}(s,a)).$$

21.2. **On-policy Monte Carlo control.** To remove the assumption of exploring starts, we need some other way of ensuring that every state-action pair has positive probability. To this end we define the ϵ-*greedy* policy, which selects a greedy action with probability $1 - \epsilon + \epsilon/|\mathcal{A}(s)|$ and any random action in $\mathcal{A}(s)$ with probability $\epsilon/|\mathcal{A}(s)|$ (a policy such that $a \in \mathcal{A}(s)$ has at least probability $\epsilon/|\mathcal{A}(s)|$ is called ϵ-*soft*). We may initialize the algorithm with an arbitrary action value function q_0 and ϵ-soft policy π_0. In each iteration i, for each $s \in \mathcal{S}$, let

$$a_*(s) = \text{argmax}_{a \in \mathcal{A}(s)} \hat{q}_{i-1}(s,a),$$

and, for all $a \in \mathcal{A}(s)$,

$$\pi_i(a \mid s) = \begin{cases} 1 - \epsilon + \epsilon/|\mathcal{A}(s)|, & \text{if } a = a_*(s), \\ \epsilon/|\mathcal{A}(s)|, & \text{if } a \neq a_*(s). \end{cases}$$

An episode $S_1^{(i)}, R_1^{(i)}, \dots, S_{\tau^{(i)}}^{(i)}, R_{\tau^{(i)}}^{(i)}$ is generated using π_i and $\hat{q}_i$ is updated in all visited pairs (s,a) as:

$$\hat{q}_i(s,a) = \hat{q}_{i-1}(s,a) + \frac{1}{N_i(s,a)}(G^{(i)}_{\tau^{(i)}_{s,a}} - \hat{q}_{i-1}(s,a)). \tag{34}$$

The limiting policy cannot be optimal globally, but only optimal among all ϵ-soft policies.

The factor $\frac{1}{N_i(s,a)}$ appearing in (34) may be interpreted as a chosen step-size, which is decaying. One could also consider other step-sizes here (generically called α), perhaps decaying slower or not at all (which of course modifies the convergence properties). Setting a constant $\alpha > 0$ gives a *constant-α MC* algorithm. This should be compared to the TD methods introduced in the next section, as well as to the Robbins-Monro algorithm from Section 13.

Lecture 14

Literature:

- Sutton and Barto, Chapter 6 and 9.
- Kushner and Yin, Section 2.4.1

21.3. **SARSA and Q-learning.** Let us recall the Bellman equation (21), which we can write in several steps as

$$v_\pi(s) = \mathrm{E}_\pi[G_t \mid S_t = s] \tag{35}$$
$$= \mathrm{E}_\pi[R_{t+1} + \gamma v_\pi(S_{t+1}) \mid S_t = s] \tag{36}$$
$$= \sum_a \pi(a \mid s) \sum_{r,s'} (r + \gamma v_\pi(s')) p(s', r \mid s, a). \tag{37}$$

The policy evaluation in (28) is based on (37) and uses explicit knowledge of $p(s', r \mid s, a)$. The Monte Carlo policy evaluation in the previous section, (33), is based on Monte Carlo approximation of (35). A potential problem using (35) is that the gain G_t cannot be evaluated until the episode terminates. This implies that the estimate of v_π can only be updated after each episode. Now we will consider another approach based on Monte Carlo approximation of (36). Updating using (36) requires only R_{t+1} and S_{t+1}, which implies the estimate can be updated after every time step. Such methods are called temporal difference (TD) methods. Following the recipe for incremental updates, the state value can be updated as follows. Given v_t, a state S_t, and a policy π, we can generate R_{t+1} and S_{t+1} using π and update

$$v_{t+1}(S_t) = v_t(S_t) + \alpha \left[R_{t+1} + \gamma v_t(S_{t+1}) - v_t(S_t) \right], \tag{38}$$

where α is a generic notation for the step-size. The update method in (38) is called *TD(0)*. The corresponding algorithm for the action value is as follows. Given q_t, a state S_t, and a policy π, we can generate an action A_t using $\pi(\cdot \mid S_t)$, and then R_{t+1} and S_{t+1} and, finally, A_{t+1} from $\pi(\cdot \mid S_{t+1})$, and proceed by updating

$$q_{t+1}(S_t, A_t) = q_t(S_t, A_t) + \alpha \left[R_{t+1} + \gamma q_t(S_{t+1}, A_{t+1}) - q_t(S_t, A_t) \right].$$

This updating is based on generating $S_t, A_t, R_{t+1}, S_{t+1}, A_{t+1}$, which has given rise to the method's name: *SARSA*.

21.3.1. *Policy iteration using SARSA.* The SARSA method for policy evaluation can also be used in policy iteration in the following way. Assume a current state S_t and action A_t, and current action value estimates q_t (it will be clear how A_t and q_t were created using the previous estimates q_{t-1}, soon). From the action A_t, generate a reward and the next state, R_{t+1}, S_{t+1}.

In the new state S_{t+1}, select the next action A_{t+1} ϵ-greedily based on the last estimates q_t. In other words, letting

$$a_*(s) = \mathrm{argmax}_{a \in \mathcal{A}(s)} q_t(s, a),$$

and, for all $a \in \mathcal{A}(s)$,

$$\pi_{t+1}(a \mid s) = \begin{cases} 1 - \epsilon + \epsilon/|\mathcal{A}(s)|, & \text{if } a = a_*(s), \\ \epsilon/|\mathcal{A}(s)|, & \text{if } a \neq a_*(s), \end{cases}$$

draw A_{t+1} from $\pi_{t+1}(\cdot \mid S_{t+1})$. Now apply the SARSA update for the q-estimate at (S_t, A_t):

$$q_{t+1}(S_t, A_t) = q_t(S_t, A_t) + \alpha[R_{t+1} + \gamma q_t(S_{t+1}, A_{t+1}) - q_t(S_t, A_t)].$$

This takes us back to the starting point: having the estimates q_{t+1} and a state-action pair (S_{t+1}, A_{t+1}), and the procedure is iterated.

21.3.2. *Q-learning.* An alternative to SARSA is to directly use Bellman's optimality equation for the action value, instead of policy iteration. This is called *Q-learning*. It still requires a policy to draw action values, A_t, to be updated, but does not need the action A_{t+1} when updating the action value at A_t. The Q-learning algorithm proceeds as follows.

Given the action value q_t, select the action A_t using, say, the ϵ-greedy algorithm. That is, given $S_t = s$, let

$$a_*(s) = \text{argmax}_{a\in\mathcal{A}(s)} q_t(s, a),$$

and, for all $a \in \mathcal{A}(s)$,

$$\pi_t(a \mid s) = \begin{cases} 1 - \epsilon + \epsilon/|\mathcal{A}(s)|, & \text{if } a = a_*(s), \\ \epsilon/|\mathcal{A}(s)|, & \text{if } a \neq a_*(s). \end{cases}$$

Draw A_t from $\pi_t(\cdot \mid s)$ and generate R_{t+1}, S_{t+1}. Then update

$$q_{t+1}(S_t, A_t) = q_t(S_t, A_t) + \alpha \left[R_{t+1} + \gamma \sup_{a\in\mathcal{A}(S_{t+1})} q_t(S_{t+1}, a) - q_t(S_t, A_t) \right]. \quad (39)$$

The Q-learning algorithm is very popular and also admits convergence proofs based on stochastic approximation.

Let us now take a closer look at the step-size α in the Q-learning algorithm. Let $\{\alpha_n\}$ be a sequence of positive real numbers such that $\sum \alpha_n = \infty$ (often taken as $\alpha_n = c\, n^{-\omega}$ for some $\omega \in [0, 1]$) and let $w(s, a)$ be positive real numbers. Define $\alpha_n(s, a) = w(s, a)\alpha_n$. The step-size used at the current iteration depends on the number of times that the particular current state-action pair has been seen before. If the current state-action pair is (s, a) and it has been seen n times before in the simulation, then the step-size used in the current update is $\tilde{\alpha}_t(s, a) = \alpha_{n+1}(s, a)$. More precisely, the Q-learning algorithm described above can be further specified by replacing the update (39) by

$$\begin{aligned} q_{t+1}(S_t, A_t) = {} & q_t(S_t, A_t) \\ & + \tilde{\alpha}_t(S_t, A_t) \left[R_{t+1} + \gamma \sup_{a\in\mathcal{A}(S_{t+1})} q_t(S_{t+1}, a) - q_t(S_t, A_t) \right]. \end{aligned}$$

Next, we provide some additional details on the connection to Robbins-Monro and stochastic approximation algorithms, discussed in Section 13. Let T_* be the Bellman optimality operator, defined by

$$\begin{aligned} T_* q(s, a) &= \mathrm{E}[R_{t+1} + \gamma \sup_{a'\in\mathcal{A}(S_{t+1})} q(S_{t+1}, a')) \mid S_t = s, A_t = a, S_{t-1}, A_{t-1}, \ldots, S_0, A_0] \\ &= \mathrm{E}[R_{t+1} + \gamma \sup_{a'\in\mathcal{A}(S_{t+1})} q(S_{t+1}, a')) \mid S_t = s, A_t = a], \end{aligned}$$

where the second equality follows from the Markov property. With

$$Y_{t+1} = R_{t+1} + \gamma \sup_{a'\in\mathcal{A}(S_{t+1})} q_t(S_{t+1}, a') - T_* q_t(S_t, A_t),$$

we can write the Q-learning update as

$$q_{t+1}(S_t, A_t) = q_t(S_t, A_t) + \tilde{\alpha}_t(S_t, A_t) \left[T_* q_t(S_t, A_t) - q_t(S_t, A_t) + Y_{t+1} \right], \quad (40)$$

where

$$\mathrm{E}[Y_{t+1} \mid S_t, A_t, \ldots, S_0, A_0] = 0.$$

Ignoring the random variables (S_t, A_t), (40) can be seen as the Robbins-Monro algorithm for root finding of the function $(T_* - \mathrm{id})$, in the variable q. With $\gamma < 1$, the operator T_* is a contraction with respect to the sup-norm and q_* is the unique fixed point.

21.4. **Learning with function approximation.** So far we have considered the state value v_π and the action value q_π as vectors, where each component in the vector is a state, or a state-action pair. The updating mechanisms considered above update each state, or each state-action pair, separately. For instance, in the Q-learning algorithm, each update updates the action value at S_t, A_t. If the number of state-action pairs is large, it will take a long time to update all the states.

An alternative is to approximate the action value as a function, depending on some parameters θ. Rather than updating the action value at each state–action pair, the parameter θ is updated, which affects the action function at all state-action pairs.

Let us consider function approximation for policy evaluation. Suppose we want to approximate the state value v_π for a given policy π, using functions $\hat{v}(\cdot, \theta)$. We may, for instance, try to minimize the weighted mean-squared error

$$\inf_\theta \sum_s w(s)(v_\pi(s) - \hat{v}(s,\theta))^2,$$

using gradient descent, where $w(s)$ are non-negative weights with $\sum_s w(s) = 1$. Then, the gradient descent with step-size α leads to an iteration of the form

$$\begin{aligned}\theta_{t+1} &= \theta_t - \alpha \sum_s w(s) \nabla_\theta (v_\pi(s) - \hat{v}(s,\theta))^2 \\ &= \theta_t + 2\alpha \sum_s w(s)(v_\pi(s) - \hat{v}(s,\theta)) \nabla_\theta \hat{v}(s,\theta).\end{aligned}$$

The gradient descent in the previous display cannot be used in this form, since we do not know v_π. A stochastic gradient descent version can be obtained by updating, using a single sampled S_t in each update and replacing $v_\pi(s)$ with a random quantity, whose expectation is $v_\pi(s)$.

For example, combining the Monte Carlo policy evaluation (33) with function approximation leads to the update, after each episode,

$$\theta_{i+1} = \theta_i + \alpha[G^{(i)}_{\tau_s^{(i)}} - \hat{v}(s,\theta_i)] \nabla_\theta \hat{v}(s,\theta_i),$$

for each state s visited in the jth episode.

Similarly, for the TD(0) method for policy evaluation, given θ_{t-1} and the current state $S_{t-1} = s$, we use π to generate A_{t-1}, R_t and S_t and update

$$\theta_{t+1} = \theta_t + \alpha \left[R_{t+1} + \gamma \hat{v}(S_{t+1}, \theta_t) - \hat{v}(S_t, \theta_t)\right] \nabla_\theta \hat{v}(S_t, \theta_t). \tag{41}$$

Additionally, one can proceed similarly to function approximation for the action values. If one approximates q_π with $\hat{q}$ in a similar fashion to the above, the SARSA update rule has the same structure as (41), but with $\hat{q}$ instead of $\hat{v}$. Popular forms

of the approximating functions are

$$\hat{v}(s,\theta) = \sum_{i=1}^{p} \theta_i \phi_i(s),$$

for some basis functions ϕ_i. Such approximations are linear in the weights, which makes the gradient $\nabla_\theta \hat{v}(s,\theta)$ particularly easy to evaluate. Function approximation of this type is called *linear approximation*. Alternatively, one can use deep feedforward neural networks to approximate $\hat{v}(s,\theta)$. This case is often referred to as deep reinforcement learning, and in this case, θ are the weight parameters in the network. The gradient $\nabla_\theta \hat{v}(s,\theta)$ can, in this case, be computed using backpropagation. For more details, see Sutton and Barto, Chapter 9.

Lecture 15

Literature:

- S. Janson, *Gaussian Hilbert Spaces*, Cambridge University Press, 1997 (We use the theorems in Appendix F).

22. Reproducing kernel Hilbert spaces and the kernel trick

A (real) vector space H is called an *inner product space* if there is an *inner product* $\langle \cdot, \cdot \rangle_H : H \times H \to \mathbb{R}$ satisfying

(i) $\langle f, g \rangle_H = \langle g, f \rangle_H$, for each $f, g \in H$ (symmetry),
(ii) $\langle \alpha f + \beta g, h \rangle_H = \alpha \langle f, h \rangle_H + \beta \langle g, h \rangle_H$, for each $f, g, h \in H$, $\alpha, \beta, \in \mathbb{R}$ (linearity),
(iii) $\langle f, f \rangle_H \geq 0$, for each $f \in H$, with equality only if $f = 0$.

If H is complete, that is, every Cauchy sequence $\{f_n\}$ converges to a point in H, then H is a *Hilbert space*. A *reproducing kernel Hilbert space* (RKHS) is a Hilbert space H of functions on a set $\mathcal{X}$ such that the point evaluation, $f \mapsto f(x)$, is a continuous linear functional for each $x \in \mathcal{X}$.

Example 29. Here are three standard examples of Hilbert spaces.

(i) The Euclidean space $\mathbb{R}^d$ with inner product $\langle u, v \rangle = x^{\mathrm{T}} y$ is a Hilbert space. Each $u \in \mathbb{R}^d$ can be identified with a function on $\mathcal{X} = \{1, \dots, d\}$ by $u(i) = u_i = \langle u, e_i \rangle$, which is continuous for each $i \in \mathcal{X}$. Hence, it is an RKHS.
(ii) The space $\ell^2 := \{f = (f_1, f_2, \dots) : \sum_n f_n^2 < \infty\}$ with the inner product $\langle f, g \rangle_{\ell^2} = \sum_n f_n g_n$ is an RKHS, since $\mathcal{X} = \{1, 2, \dots\}$ and $f \mapsto f_n$ is continuous and linear for each n.
(iii) The space $L^2(\mathbb{R}) := \{f : \mathbb{R} \to \mathbb{R} : \int f^2(x) dx < \infty\}$ with the inner product $\langle f, g \rangle_{L^2} = \int f(x) g(x) dx$ is a Hilbert space. But, point evaluation $f \mapsto f(x)$ is not continuous, because $f_n \to f$ in $L^2(\mathbb{R})$ does not imply $f_n(x) \to f(x)$ pointwise, so $L^2(\mathbb{R})$ is not an RKHS.

A *reproducing kernel* associated to an RKHS H, is a function $\mathcal{X} \times \mathcal{X} \to R$ satisfying the reproducing property:

$$f(x) = \langle f, k(\cdot, x) \rangle_H, \quad \text{for each } x \in \mathcal{X}, f \in H.$$

Note that, since the point evaluation $f \mapsto f(x)$ is a continuous linear functional on a Hilbert space, there is (by the *Riesz representation theorem*) an element $h_x(\cdot) \in H$, which is a function on $\mathcal{X}$, such that $f(x) = \langle f, h_x(\cdot) \rangle_H$. Then, $k(\cdot, x) = h_x(\cdot)$ is the reproducing kernel. Note also that, conversely, if a reproducing kernel exists for any Hilbert space H, H is an RKHS, making this an alternative definition of RKHS.

The following properties of an RKHS are useful.

Theorem 13. *Let H be a reproducing kernel Hilbert space of functions on $\mathcal{X}$. Then, there is a unique kernel $k : \mathcal{X} \times \mathcal{X} \to \mathbb{R}$ such that*

(i) $k(x, x') = k(x', x)$.
(ii) The kernel k is positive semi-definite: $\sum_{i,j=1}^n \alpha_i \alpha_j k(x_i, x_j) \geq 0$, for each $\{x_i\}_{i=1}^n \subset \mathcal{X}$ and $\{\alpha_i\}_{i=1}^n \subset \mathbb{R}$.
(iii) The function $x \mapsto k(x, x')$ belongs to H for each $x' \in \mathcal{X}$.
(iv) $f(x) = \langle f, k(\cdot, x) \rangle_H$, for each $x \in \mathcal{X}$, and $f \in H$.
(v) $|k(\cdot, x)|_H^2 = k(x, x)$ and $\langle k(\cdot, x), k(\cdot, x') \rangle = k(x, x')$ for any $x, x' \in \mathcal{X}$.
(vi) The set $\{k(\cdot, x)\}_{x \in \mathcal{X}}$ is total in H, that is, if $\langle f, k(\cdot, x) \rangle_H = 0$ for each $x \in \mathcal{X}$, then $f = 0$).

Reproducing kernel Hilbert spaces are often constructed in the following general way.

Theorem 14. *Let H_0 be a Hilbert space and $\{\tilde{h}_x\}_{x\in\mathcal{X}}$ a total set in H_0. Define a linear map $\Phi : H_0 \to \mathbb{R}^{\mathcal{X}}$ by $\Phi h(x) = \langle h, \tilde{h}_x\rangle_{H_0}$. Then Φ is injective and we may define the inner product on $H = \Phi(H_0) = \{\Phi h : h \in H_0\}$ by*

$$\langle f, g\rangle_H = \langle \Phi^{-1} f, \Phi^{-1} g\rangle_{H_0},$$

which makes $\Phi : H_0 \to H$ an isometry. Then H is an RKHS with kernel $k(x, x') = \langle h_x, h_{x'}\rangle$.

Example 30. A simple example is when $\mathcal{X} = H_0 = \mathbb{R}^d$ and $\tilde{h}_x = x$. Then, $\Phi h_0 = \langle h_0, \tilde{h}_\cdot\rangle_{\mathbb{R}^d} = \langle h_0, \cdot\rangle_{\mathbb{R}^d} \in \mathbb{R}^{\mathcal{X}}$. Thus, $H = \Phi(H_0)$ consists of the functions $f_{x'}(\cdot) = \langle x', \cdot\rangle_{\mathbb{R}^d}$, $x' \in \mathbb{R}^d$, and

$$\langle f_x, f_{x'}\rangle_H = \langle \Phi^{-1} f_x, \Phi^{-1} f_{x'}\rangle_{H_0} = \langle x, x'\rangle_{\mathbb{R}^d}.$$

A linear predictor of the form $\langle \theta, x\rangle_{\mathbb{R}^d}$ can be associated with a corresponding linear functional $f_\theta : H \to \mathbb{R}$, given by an element $\langle \theta, \cdot\rangle_{\mathbb{R}^d} \in H$, with point evaluation $f_\theta(x) = \langle \langle \theta, \cdot\rangle_{\mathbb{R}^d}, \langle x, \cdot\rangle_{\mathbb{R}^d}\rangle_H = \langle \theta, x\rangle_{\mathbb{R}^d}$.

Example 31. A common setup in machine learning (e.g. classification and/or regression) is that decisions are based on a latent function $f : \mathcal{X} \to \mathbb{R}$ of the form $f(x) = \theta^{\mathrm{T}}\phi(x)$, where $\phi(x) = (\phi_1(x), \dots, \phi_m(x))^{\mathrm{T}}$ is a vector of feature functions. In this case, let $H_0 = \mathbb{R}^m$ and $\{\tilde{h}_x\}_{x\in\mathcal{X}} = \{\phi(x)\}_{x\in\mathcal{X}}$ the total set. We can define $\Phi : H_0 \to \mathbb{R}^{\mathcal{X}}$ by $\Phi h(\cdot) = \langle h, \phi(\cdot)\rangle_{\mathbb{R}^m}$. Then $H = \Phi(H_0)$ is an RKHS with kernel $k(x, x') = \langle \tilde{h}_x, \tilde{h}_{x'}\rangle_{H_0} = \langle \phi(x), \phi(x')\rangle_{\mathbb{R}^m}$. One may note, in particular, that $\mathcal{X}$ need not be a vector space, but merely a set on which the feature functions are defined. A linear predictor of the form $\langle \theta, \phi(x)\rangle_{\mathbb{R}^m}$ can be associated with a corresponding linear functional $f_\theta : H \to \mathbb{R}$, given by an element $\langle \theta, \phi(\cdot)\rangle_{\mathbb{R}^m} \in H$, $\theta \in H_0$, with point evaluation,

$$f_\theta(x) = \big\langle \langle \theta, \phi(\cdot)\rangle_{\mathbb{R}^m}, k(x, \cdot)\big\rangle_H = \big\langle \langle \theta, \phi(\cdot)\rangle_{\mathbb{R}^m}, \langle \phi(x), \phi(\cdot)\rangle_{\mathbb{R}^m}\big\rangle_H = \langle \theta, \phi(x)\rangle_{\mathbb{R}^m}.$$

An extension of the previous example is obtained by omitting the feature functions, ϕ_j, and instead directly defining a kernel $k : \mathcal{X} \times \mathcal{X} \to \mathbb{R}$, and interpreting the features of x as the element $k(\cdot, x)$ in the RKHS. First, let us show the existence of an RKHS for a given kernel k.

Theorem 15 (Moore-Aronszajn theorem)**.** *Suppose $\mathcal{X}$ is a set and $k : \mathcal{X} \times \mathcal{X} \to \mathbb{R}$ is a positive semi-definite function such that $k(x, y) = k(y, x)$, $x, y \in \mathcal{X}$. Then there exists a unique RKHS, H, with reproducing kernel k.*

In light of Theorem 15, given a kernel k, there exists an RKHS, H, and we may consider the feature map $\mathcal{X} \to H$ given by $x \mapsto k(\cdot, x)$, and interpret $k(\cdot, x)$ as representing the features of x. This is called the *kernel trick*. A linear predictor can be associated with a linear functional given by an element $f \in H$ with point evaluation $f(x) = \langle f, k(\cdot, x)\rangle_H$.

22.1. **Kernels and operators.** Let $L^2(\mathcal{X})$ denote the square-integrable functions on $\mathcal{X}$. Given a square-integrable kernel $k : \mathcal{X} \times \mathcal{X} \to \mathbb{R}$ we may associate an integral operator $K : L^2(\mathcal{X}) \to L^2(\mathcal{X})$ by

$$Kf(x) = \int_{\mathcal{X}} k(x, x')f(x')dx', \qquad x \in \mathcal{X}.$$

We may define the eigenvalues and eigenfunctions of K as the pairs $\{(\lambda_n, \phi_n)\}_{n\geq 1}$ satisfying

$$K\phi_n = \lambda_n \phi_n, \qquad k \geq 1.$$

If k is positive definite, so is K (the converse is also true), and the eigenvalues of K are non-negative.

Theorem 16 (Mercer's theorem)**.** *Assume that the kernel k is continuous and that $\mathcal{X}$ is compact. With $\{(\lambda_n, \phi_n)\}_{n\geq 1}$ being the eigenvalues and orthonormalized eigenvectors of K (meaning that $\int_{\mathcal{X}} \phi_m(x)\phi_n(x)dx = \delta_{mn}$), the representation*

$$k(x, x') = \sum_n \lambda_n \phi_n(x)\phi_n(x'), \qquad x, x' \in \mathcal{X},$$

holds.

The proof uses that K is a self-adjoint Hilbert-Schmidt operator, which implies that it is compact, and the spectral theorem for self-adjoint compact operators.

One can actually define the RKHS associated with k as the space spanned by the eigenfunctions $\{\phi_n\}$. Then,

$$H = \{f : f(x) = \sum_n \alpha_n \phi_n(x),\ |f|_H^2 = \sum_n \frac{\alpha_n^2}{\lambda_n} < \infty\},$$

and the inner product is defined by

$$\langle f, g\rangle_H = \langle \sum_n \alpha\phi_n, \sum_n \beta_n \phi_n\rangle_H = \sum_n \frac{\alpha_n \beta_n}{\lambda_n}.$$

A remarkable property of RKHSs is their connection to certain optimization problems appearing in machine learning. Consider an arbitrary loss function L and a monotonically increasing regularization function $g : \mathbb{R} \to \mathbb{R}$ and a set of data $\{(x_i, y_i)\}_{i=1}^n$.

Theorem 17 (Representer theorem)**.** *Let H be an RKHS with kernel k. Then, a function of the form $\hat{f}(x) = \sum_{i=1}^n \alpha_i k(x, x_i)$ is a minimizer in H of*

$$L((f(x_1), y_1), \ldots, (f(x_n), y_n)) + g(|f|_H).$$

The idea of the proof is to take an arbitrary $f \in H$ and represent it as a sum of its projection onto the subspace $\text{span}\{k(x_i, \cdot), i = 1, \ldots, n\}$ and its orthogonal complement in H.

Proof. Take an arbitrary $f \in H$ and write it as $f = f_1 + f_2$ where f_1 is the projection of f onto $\text{span}\{k(\cdot, x_i), i = 1, \ldots, n\}$ and f_2 its orthogonal complement, i.e. $\langle f_1, f_2\rangle_H = 0$. Then,

$$|f|_H^2 = |f_1|_H^2 + |f_2|_H^2 + 2\langle f_1, f_2\rangle_H = |f_1|_H^2 + |f_2|_H^2 \geq |f_1|_H^2.$$

Since g is increasing $g(|f|_H) \geq g(|f_1|_H)$. Moreover, by the reproducing property and $\langle f_2, k(\cdot, x_i)\rangle_H = 0$ for each i,

$$f(x_i) = \langle f, k(\cdot, x_i)\rangle_H = \langle f_1, k(\cdot, x_i)\rangle_H + \langle f_2, k(\cdot, x_i)\rangle_H = \langle f_1, k(\cdot, x_i)\rangle_H = f_1(x_i),$$

for each i. This implies that $L((f(x_1), y_1), \ldots, (f(x_n), y_n)) = L((f_1(x_1), y_1), \ldots, (f_1(x_n), y_n))$ and it follows that the minimizer belongs to $\text{span}\{k(\cdot, x_i), i = 1, \ldots, n\}$ and therefore is of the form $\hat{f}(x) = \sum_{i=1}^n \alpha_i k(x, x_i)$. □

Lecture 16

Literature:

- Goodfellow, Bengio, and Courville, Deep Learning, Chapters 6 and 9

23. Deep Neural Networks for Classification and Regression

In this section we will introduce deep neural networks for classification and regression. Let us begin by introducing (feed-forward) neural networks.

Let $f : \mathcal{X} \to \mathbb{R}^{d_n}$ be a composition of functions $f^{(i)}$, $i = 1, \dots, n$, of the form

$$f(x) = f^{(n)} \circ \cdots \circ f^{(1)}(x) = f^{(n)}(f^{(n-1)}(\dots f^{(1)}(x))). \tag{42}$$

Each $f^{(i)}$ is interpreted as the transition between hidden layers and $f^{(i)}$ is often of a simple form:

$$f^{(i)}(h^{(i-1)}) = g^{(i)}(W^{(i)}h^{(i-1)} + b^{(i)}). \tag{43}$$

In this notation $h^{(i-1)}$ is a vector, the value at the $(i-1)$th layer, $h^{(i-1)} = f^{(i-1)} \circ \cdots \circ f^{(1)}(x)$, W^i is a matrix, $b^{(i)}$ is a vector, called the bias, and $g^{(i)}$ is an activation function applied to each component of $W^{(i)}h^{(i-1)} + b^{(i)}$. A layer of the form (43) in a neural network is called a *fully connected layer* or a *dense layer*, because all the outputs of the layer are dependent on (connected to) all the inputs to the layer. The neural network f is parametrized by all the weight matrices $W^{(i)}$ and the bias vectors $b^{(i)}$ and we write $\theta = \{\theta^{(i)}\}_{i=1}^n = \{(W^{(i)}, b^{(i)})\}_{i=1}^n$, and f_θ for the corresponding neural network.

23.1. **Activation functions.** There are many choices in setting up the architecture of a feed-forward neural network, such as the depth (the number of layers), the width of each layer (the dimension of $h^{(i)}$), the choice of the activation functions $g^{(i)}$ and the structure of the matrices $W^{(i)}$.

In this section we briefly discuss frequently used activation functions. Note that if the activation functions $g^{(i)}$ are linear, then f is also linear and the whole feedforward neural network reduces to a linear function. Thus, the non-linearities captured by neural networks relies entirely on the activation functions being non-linear. Popular choices of activation functions are given in Table 23.1.

Name	$g^{(i)}(z)$
ReLU	$\max(z, 0)$
leaky ReLU	$zI\{z > 0\} + \alpha z I\{z < 0\}$
absolute value	$\lvert z \rvert$
tanh	$\tanh(z)$
sigmoid	$\mathrm{sigm}(z)$

Table 23.1. Frequently used activation functions.

In addition to capturing non-linearities the choice of activation function plays a major role in the training of the networks, usually done by gradient descent, which requires the derivative of $g^{(i)}$. Rectified linear units are attractive in that they are similar to linear units, the gradient is constant whenever the unit is active (the argument is positive) and therefore does not introduce second-order effects. In contrast, the sigmoid function saturates for large values of the argument, which implies that the gradient is close to zero and parameter updates may be slow.

23.2. **Neural networks for classification.** Consider first the binary classification problem. Suppose we have training data in the form of $\{(x_i, y_i)\}_{i=1}^n$ that are outcomes of independent pairs $\{(X_i, Y_i)\}_{i=1}^n$ where X_i represents the explanatory variables and Y_i the corresponding label. We also have $X_{n+1} = x_{n+1}$ and want to predict the label Y_{n+1} that takes values in $\mathcal{Y} = \{-1, 1\}$. Let $X = (X_1, Y_1, \ldots, X_n, Y_n, X_{n+1})$ be the observations and $Y = Y_{n+1}$ the label to predict.

The conditional probabilities are modeled as

$$\Pr(Y_i = y_i \mid \Theta = \theta, X_i = x_i) = \operatorname{sigm}\Big(y_i f_\theta(x_i)\Big).$$

where $\mathcal{D} = \{f_\theta, \theta \in \Theta\}$ is a neural network with a single output: $f_\theta : \mathcal{X} \to \mathbb{R}$.

To perform classification with neural networks we could put a prior p_Θ on the parameters and proceed with the Bayesian approach outlined above to compute the predictive class probabilities. As indicated in Section 16.2 wide one-hidden-layer neural networks with independent prior can be approximated by Gaussian processes.

However, often the prior is completely ignored and instead the maximum likelihood estimate $\theta^{\mathrm{ML}} = \operatorname{argmax}_\theta p_{X|\Theta}(x \mid \theta)$ is used as the "true" value of θ and classification decisions are based entirely on

$$p_{Y|X,\Theta}(y \mid x, \theta^{\mathrm{ML}}).$$

The log-likelihood becomes

$$\begin{aligned}\log p_{X|\Theta}(x \mid \theta) &= \log p_{Y_1,\ldots,Y_n|\Theta,X_1,\ldots,X_{n+1}}(y_1, \ldots, y_n \mid \theta, x_1, \ldots, x_{n+1}) \\ &\quad + \log p_{X_1,\ldots,X_{n+1}}(x_1, \ldots, x_{n+1}) \\ &= \sum_{i=1}^n \log \operatorname{sigm}\Big(y_i f_\theta(x_i)\Big) + \sum_{i=1}^{n+1} \log p_{X_i}(x_i).\end{aligned}$$

Maximizing the log-likelihood is equivalent to minimizing the function

$$-\sum_{i=1}^n \log \operatorname{sigm}(y_i f_\theta(x_i)),$$

over the parameters $\theta = \{(W^{(i)}, b^{(i)})\}_{i=1}^m$. The resulting decision rule is to classify $\delta(x) = 1$ if $p_{Y_{n+1}|X_{n+1},\Theta}(1 \mid x_{n+1}, \theta^{\mathrm{ML}}) \geq 1/2$, or equivalently if $f_{\theta^{\mathrm{ML}}}(x_{n+1}) \geq 0$.

Note that, the same decision rule is obtained if we parametrize the decision rules by the parameters, θ, of the neural network as $\delta_\theta(x_i) = f_\theta(x_i)$, where the actual classification is to select the label as $2I\{f_\theta(x_{n+1}) \geq 0\} - 1$, and train the decision rules by minimizing the empirical risk

$$R^{\mathrm{emp}}(\delta_\theta) = \frac{1}{n}\sum_{i=1}^n L(y_i, f_\theta(x_i)),$$

where the loss function is $L(y, a) = -\log \operatorname{sigm}(ya)$.

23.2.1. *Multi-class classification.* In classification problems with multiple classes $\{1, \ldots, C\}$ we may represent the label y_i as a *one-hot-vector*

$$y_i = (y_i^1, \ldots, y_i^C),$$

where all but one elements are zero and $y_i^c = 1$ for the correct class. As a generalization of the sigmoid the class probabilities can be written as

$$\Pr(Y_i = a_c \mid \Theta = \theta, X_i = x_i) = \operatorname{softmax}(f_\theta(x_i))_c = \frac{e^{f_\theta^c(x_i)}}{\sum_{c'=1}^C e^{f_\theta^{c'}(x_i)}},$$

where a_c refers to the one-hot-vector corresponding to class c and $f_\theta : \mathcal{X} \to \mathbb{R}^C$ is a neural network with C-dimensional output. The conditional class probabilities are (recalling that $y_i^c = 1$ for exactly one value of c)

$$p_{Y_i|X_i,\Theta}(y_i \mid x_i, \theta) = \prod_{c=1}^C \operatorname{softmax}(f_\theta(x_i))_c^{y_i^c}.$$

Now we can proceed just as in binary classification. Again, the prior is completely ignored and instead the maximum likelihood estimate $\theta^{\mathrm{ML}} = \operatorname{argmax}_\theta p_{X|\Theta}(x \mid \theta)$ is used as the "true" value of θ and classification decisions are based entirely on

$$p_{Y|X,\Theta}(y \mid x, \theta^{\mathrm{ML}}).$$

The likelihood is

$$\begin{aligned}
p_{X|\Theta}(x \mid \theta) &= p_{Y_1,\ldots,Y_n|\Theta,X_1,\ldots,X_{n+1}}(y_1,\ldots,y_n \mid \theta, x_1,\ldots,x_{n+1}) p_{X_1,\ldots,X_{n+1}}(x_1,\ldots,x_{n+1}) \\
&= \prod_{i=1}^n \Big(\prod_{c=1}^C \operatorname{softmax}\big(f_\theta(x_i)\big)_c^{I\{y_i=a_c\}} \Big) \prod_{i=1}^{n+1} p_{X_i}(x_i) \\
&= \prod_{i=1}^n \Big(\prod_{c=1}^C \Big(\frac{e^{f_\theta^c(x_i)}}{\sum_{c'=1}^C e^{f_\theta^{c'}(x_i)}} \Big)^{I\{y_i=a_c\}} \Big) \prod_{i=1}^{n+1} p_{X_i}(x_i),
\end{aligned}$$

and the log-likelihood

$$\log p_{X|\Theta}(x \mid \theta) = \sum_{i=1}^n \sum_{c=1}^C y_i^c \log \operatorname{softmax}(f_\theta(x_i))_c + \sum_{i=1}^n \log p_{X_i}(x_i).$$

Maximizing the log-likelihood is equivalent to minimizing

$$\sum_{k=1}^n \sum_{c=1}^C -y_i^c \log \pi_i^c.$$

where $\pi_i = (\pi_i^1, \ldots, \pi_i^C)$ is the probability distribution on $\{1,\ldots,C\}$ with $\pi_i^c = \operatorname{softmax}(f(x_i))_c$.

A one-hot-vector $y_i = (y_i^1, \ldots, y_i^C)$ may be interpreted as a probability distribution on $\{1,\ldots,C\}$ with unit mass on y_i^c, which implies

$$\sum_{c=1}^C -y_i^c \log \pi_i^c = H(y_i \mid \pi_i),$$

H is the relative entropy,

$$H(q \mid p) = \sum_{c=1}^C -q^c \log(q^c/p^c),$$

for two probabilities p and q on $\{1,\ldots,C\}$.

The resulting decision rule is to classify $\delta(x) = c$ if $p_{Y_{n+1}|X_{n+1},\Theta}(a_c \mid x_{n+1}, \theta^{\mathrm{ML}})$ is maximal, or equivalently if $f_{\theta^{\mathrm{ML}}}^c(x_{n+1})$ is maximal.

Note that, just as in the binary classification, the same decision rule is obtained if we parametrize the decision rules by the parameters, θ, of the neural network as $\delta_\theta(x_i) = \text{softmax}(f_\theta(x_i))$, where the actual classification is to select the label as $\text{argmax}_c \, \text{softmax}(f_{\theta^{\text{ML}}}(x_{n+1}))$, and train the decision rules by minimizing the empirical risk

$$R^{\text{emp}}(\delta_\theta) = \frac{1}{n}\sum_{i=1}^{n} L(y_i, \delta_\theta(x_i)),$$

where the loss function is $L(y, a) = H(y \mid a)$ is the relative entropy. This loss function is usually referred to as the *cross-entropy loss*.

23.3. **Neural networks for regression.** Consider a regression problem with additive noise. Suppose we have training data in the form of $\{(x_i, y_i)\}_{i=1}^n$ that are outcomes of independent pairs $\{(X_i, Y_i)\}_{i=1}^n$ where X_i represents the explanatory variables and Y_i the response variable. We also have $X_{n+1} = x_{n+1}$ and want to predict the response Y_{n+1}. As usual we put $X = (X_1, Y_1, \ldots, X_n, Y_n, X_{n+1})$ and $Y = Y_{n+1}$.

The regression model is,

$$Y_i = f_\theta(X_i) + \varepsilon_i.$$

where f_θ is a neural network, parametrized by θ, with output in $\mathcal{Y}$.

To perform regression with neural networks we could put a prior p_Θ on the parameters and proceed with the Bayesian approach outlined above to compute the predictive distribution (or predictive mean). However, often the prior is completely ignored and instead the maximum likelihood estimate $\theta^{\text{ML}} = \text{argmax}_\theta p_{X|\Theta}(x \mid \theta)$ is used as the "true" value of θ and prediction is based entirely on

$$p_{Y|X,\Theta}(y \mid x, \theta^{\text{ML}}).$$

If the squared loss is used, then a formal Bayes rule is the conditional expectation $\text{E}[Y \mid X = x]$, which is approximated by $\text{E}[Y \mid X = x, \Theta = \theta^{\text{ML}}] = f_{\theta^{\text{ML}}}(x_{n+1})$.

It remain only to maximize the likelihood. With $X = (X_1, Y_1, \ldots, X_n, Y_n, X_{n+1})$ the log-likelihood becomes

$$\begin{aligned}\log p_{X|\Theta}(x \mid \theta) &= \log p_{Y_1,\ldots,Y_n|\Theta,X_1,\ldots,X_{n+1}}(y_1, \ldots, y_n \mid \theta, x_1, \ldots, x_{n+1}) \\ &\quad + \log p_{X_1,\ldots,X_{n+1}}(x_1, \ldots, x_{n+1}) \\ &= \sum_{i=1}^{n} \log p_\varepsilon\Big(y_i - f_\theta(x_i)\Big) + \sum_{i=1}^{n+1} \log p_{X_i}(x_i).\end{aligned}$$

Maximizing the log-likelihood is equivalent to minimizing

$$-\sum_{i=1}^{n} \log p_\varepsilon(y_i - f_\theta(x_i)),$$

over the parameters $\theta = \{(W^{(i)}, b^{(i)})\}$. For instance, if p_ε is the density of a Gaussian random variable with known variance σ^2, then the loss function becomes

$$-\sum_{i=1}^{n} \log p_\varepsilon(y_i - f_\theta(x_i)) = \text{constant} + \frac{1}{\sigma^2}\sum_{i=1}^{n}(y_i - f_\theta(x_i))^2.$$

Minimizing this loss function is equivalent to minimizing the empirical squared loss. More precisely, by parametrizing the decision rules by $\delta_\theta(x_i) = f_\theta(x_i)$ the empirical risk is

$$R^{\text{emp}}(\delta_\theta) = -\sum_{i=1}^{n} \log p_\varepsilon(y_i - f_\theta(x_i)),$$

where the loss function is

$$L(y, a) = -\log p_\varepsilon(y - a).$$

The squared loss corresponds to p_ε being the density of a Gaussian random variable with known variance whereas the ℓ_1-loss given by $L(y, a) = \sum_{j=1}^{d} |y_j - a_j|$ corresponds to the additive noise having a density

$$p_\varepsilon(z) \propto \exp\left\{ -\sum_{j=1}^{d} |z_i| \right\}.$$

24. Convolutional neural networks

A convolutional neural network is a feed-forward neural network, consisting of convolutional layers, pooling layers, fully connected layers, etc. They are used frequently and with great success in image analysis and became increasingly popular after (Krizhevsky et al, 2010) used them to significantly improve the performance on the image classification challenge on ImageNet.

A typical architecture of a convolutional neural network (CNN) consists of one or several blocks of layers, where each block consists of a convolutional layer, a detection layer and a pooling layer, combined with a fully connected layer at the end of the CNN. Below we describe the basic structure of each of these types of layers.

24.1. **Convolutional layers.** Let us first describe the basics of a simple convolutional layer in a 2D image with one channel. The image I is a $n \times m$ matrix where each entry $I(i, j)$ corresponds to the value (color-code) of the corresponding pixel in the image. Let us, for illustration, assume that the image is the 3×4 matrix

$$I = \begin{bmatrix} I_{00} & I_{01} & I_{02} & I_{03} \\ I_{10} & I_{11} & I_{12} & I_{13} \\ I_{20} & I_{21} & I_{22} & I_{23} \end{bmatrix} = \begin{bmatrix} 4 & 1 & 3 & 5 \\ 2 & 3 & 2 & 3 \\ 1 & 7 & 6 & 5 \end{bmatrix}.$$

The convolutional layer has a specified number of *filters* where each filter is associated with a *kernel* W. Each kernel is a weight matrix of size (smaller that the image), say 2×2. Suppose that a filter has the kernel

$$W = \begin{bmatrix} W_{00} & W_{01} \\ W_{10} & W_{11} \end{bmatrix} = \begin{bmatrix} 1 & 2 \\ -2 & 0 \end{bmatrix}.$$

The result of applying the filter is the convolution of the image and the kernel

$$H(i, j) = \sum_{k=1}^{2} \sum_{l=0}^{1} W(k, l) I(i + k, j + k), \quad 0 \le i \le 1,\ 0 \le j \le 2.$$

We can interpret the output H as a hidden layer, which is the 2×3 matrix

$$H = \begin{bmatrix} H_{00} & H_{01} & H_{02} \\ H_{10} & H_{11} & H_{12} \end{bmatrix} = \begin{bmatrix} 2 & 1 & 9 \\ 6 & -7 & -4 \end{bmatrix}. \tag{44}$$

24.2. **Detection layer.** A detection layer is often used after a convolutional layer to introduce a non-linearity, just as an activation function is used in a conventional feed-forward neural network. The activation function serves the purpose to discard between strong signals and weak signals by increasing the nonlinearity of the network. A kernel in the convolutional layer corresponds to a particular feature to be detected in the image. If that feature is present at a certain location the input to the detection layer will be large, whereas it is negative or zero if that feature is not present at that location. The detection layer keeps only large positive values of the input, which indicates that the feature is present.

The activation function g is applied pointwise to each filter in a layer. That is, if the input of the detection layer is

$$H^{in} = \begin{bmatrix} H_{00} & H_{01} & H_{02} \\ H_{10} & H_{11} & H_{12} \end{bmatrix} = \begin{bmatrix} 2 & 1 & 9 \\ 6 & -7 & -4 \end{bmatrix},$$

and the activation function is a ReLU, $g(x) = \max(x, 0)$, then the output of the detection layer is

$$H^{out} = \begin{bmatrix} g(H_{00}) & g(H_{01}) & g(H_{02}) \\ g(H_{10}) & g(H_{11}) & g(H_{12}) \end{bmatrix} = \begin{bmatrix} 2 & 1 & 9 \\ 6 & 0 & 0 \end{bmatrix}.$$

24.3. **Pooling layers.** A pooling layers reduces the output of a convolutional layer by replacing the layer at a location with a summary statistic of nearby locations. Max-pooling and average pooling are the most common pooling layers.

In a max-pooling layer the input is a hidden layer H, such as in the above example:

$$H^{in} = \begin{bmatrix} g(H_{00}) & g(H_{01}) & g(H_{02}) \\ g(H_{10}) & g(H_{11}) & g(H_{12}) \end{bmatrix} = \begin{bmatrix} 2 & 1 & 9 \\ 6 & 0 & 0 \end{bmatrix}.$$

The max-pooling layer of size 2×2 reduces the layer by keeping only the maximum value in each 2×2 square. Mathematically it can be described as

$$H^{out}(i, j) = \max_{k,l=0,1} H^{in}(i + k, j + l).$$

After applying the max-pooling layer the output is

$$H^{out} = \begin{bmatrix} \max(2, 1, 6, 0) & \max(1, 9, 0, 0) \end{bmatrix} = \begin{bmatrix} 6 & 9 \end{bmatrix}.$$

If, the max-pooling layer is replaced by an average pooling layer, the output is instead given by

$$H^{out}(i, j) = \frac{1}{4} \sum_{k,l=0,1} H^{in}(i + k, j + l),$$

resulting in

$$H^{out} = \begin{bmatrix} (2 + 1 + 6 + 0)/4 & (1 + 9 + 0 + 0)/4 \end{bmatrix} = \begin{bmatrix} 9/4 & 10/4 \end{bmatrix}.$$

An effect of the pooling layer is that the information from the previous layer is *downsampled*, so fewer parameters are used to encode the information.

Pooling layers tend to make the representation of an image roughly invariant to small translations of the input. That is, say we translate the input by shifting it

one pixel to the right. Then, the output of the pooling layer does not change much. For example, if the input of a 3×2 max pooling layer is

$$H^{in} = \begin{bmatrix} 2 & 1 & 9 & 2 & 0 & 1 \\ 6 & 0 & 0 & 6 & 7 & 1 \end{bmatrix}.$$

then, the output is

$$H^{out} = \begin{bmatrix} 9 & 9 & 9 & 7 \end{bmatrix}.$$

If the input is translated one column to the right:

$$H^{in} = \begin{bmatrix} 0 & 2 & 1 & 9 & 2 & 0 \\ 1 & 6 & 0 & 0 & 6 & 7 \end{bmatrix}.$$

then, the output is

$$H^{out} = \begin{bmatrix} 6 & 9 & 9 & 9 \end{bmatrix}.$$

24.4. **Fully connected layers.** A fully connected layer is a layer in which the computation of any element in the output layer requires all the elements of the input layer (as is the case in a standard feed-forward neural network). Fully connected layers are often used at the end of a CNN to produce a classifier or regressor. The idea being that the stack of convolutional, detection and pooling layers has resulted in an abstract representation of the image, where features at a variety of scales are detected, and all this information is used as input to a classifier (the fully connected layer).

If H is an $m \times n$ matrix of inputs to the fully connected layer with weight tensor W, which is a $m \times n \times p$ matrix, and we want to perform classification based on, say, the softmax function into p classes, then the classification probabilities are

$$\pi_k = \text{softmax}(H * W_{\bullet\bullet k}),$$

where $H * W_{\bullet\bullet k}$ is the convolution of H with the matrix $W_{\bullet\bullet k}$.

24.5. **Properties of convolutional neural networks.** In a standard feed-forward neural network such as (42) each node in the hidden layer is connected to each node in the previous layer (fully connected). A convolutional neural network is constructed to reduce the number of parameters in the network, using *sparse interactions and parameter sharing*.

A network has sparse interactions if it does not include all possible connections as in a fully connected layer, but only a fraction of all possible connections in each layer. Sparse interactions implies fewer parameters to store, fewer operations to compute and improved statistical efficiency, at the risk of losing some flexibility. It is particularly useful in image analysis as detecting image features such as edges only require a small number of parameters. A convolutional layer has sparse interactions because each output node is only connected to a fraction of the input nodes.

A network with *parameter sharing* use the same parameter (e.g. weight) for more than one function. The advantage of using parameter sharing is that there are fewer parameters to estimate. Moreover, using parameter sharing in the convolutional layer implies that the network does not have to learn features at every location, but can learn features that could be anywhere in the image.

24.6. **Tensors and multi-dimensional convolutional networks.** In the basic description above we introduced CNNs to two-dimensional images, but often the input to a CNN can be multidimensional and therefore represented by tensors rather than matrices. For instance, a color image usually has three channels (red, green, blue), one for each color, so each image is represented by a 3D-tensor. Often, one would work with a mini-batch of training examples simultaneously, rather than one example at a time, resulting in a 4D-tensor, where the last dimension is the index of the image in the mini-batch. When considering 3D-images, there is an additional dimensional for the z-coordinate of the image, and there can also be a time-coordinate for videos, etc.

When working with multi-dimensional tensors you may specify along which dimensions convolution should take place. For instance, suppose you have a 3D-image tensor $I_{i,j,k}$ representing the image intensity in channel i at row j, column k, in the image, and $W_{i,l,m,n}$ is a 4D-tensor giving the connection weight between a unit in channel i of the output and a unit in channel l of the input, with an offset of m rows and n columns between the output and input units. Then, the output of the convolutional layer is

$$H^{\text{out}}_{i,j,k} = \sum_{l,m,n} W_{i,l,m,n} I_{l,j+m,k+n}$$

24.6.1. *Stride and padding.* To further reduce the number of computations we may skip some positions of the kernel, while passing it across the image. If we sample only every s_1 pixel in each row and every s_2 pixel in column, then the output of the convolutional layer is

$$H^{\text{out}}_{i,j,k} = \sum_{l,m,n} W_{i,l,m,n} I_{l,j\times s_1+m,k\times s_2+n},$$

where s is called the stride. If the strides are $s_1 = 1$ and $s_2 = 2$ in our example leading to (44) then the corresponding output is

$$H = \begin{bmatrix} 2 & 9 \\ 6 & -4 \end{bmatrix}.$$

As we saw in the example of a convolutional layer leading to (44) the convolution naturally reduces the size of the output. To avoid reduction in the output size, one can use *padding* to increase the image size before the convolutional layer. If the image is

$$I = \begin{bmatrix} I_{00} & I_{01} & I_{02} & I_{03} \\ I_{10} & I_{11} & I_{12} & I_{13} \\ I_{20} & I_{21} & I_{22} & I_{23} \end{bmatrix} = \begin{bmatrix} 4 & 1 & 3 & 5 \\ 2 & 3 & 2 & 3 \\ 1 & 7 & 6 & 5 \end{bmatrix}.$$

and the kernel is

$$W = \begin{bmatrix} W_{00} & W_{01} \\ W_{10} & W_{11} \end{bmatrix} = \begin{bmatrix} 1 & 2 \\ -2 & 0 \end{bmatrix}.$$

then we can pad the image with zeros to get the zero-padded matrix

$$I^{\text{pad}} = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & I_{00} & I_{01} & I_{02} & I_{03} & 0 \\ 0 & I_{10} & I_{11} & I_{12} & I_{13} & 0 \\ 0 & I_{20} & I_{21} & I_{22} & I_{23} & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 4 & 1 & 3 & 5 & 0 \\ 0 & 2 & 3 & 2 & 3 & 0 \\ 0 & 1 & 7 & 6 & 5 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}.$$

One can also pad with something different than zeros, for instance by reflection.

Lecture 17

Literature:

- Goodfellow, Bengio, and Courville, Deep Learning, Chapter 6 and 8.

25. Training Neural Networks

25.1. **Backpropagation and derivatives on computational graphs.** Consider the stochastic gradient descent algorithm for minimizing the loss function $L(x, y; \theta)$ of a feed-forward neural network with parameters $\theta = \{(W^i, b^{(i)})\}$. The empirical risk can be written as

$$\frac{1}{n}\sum_{k=1}^{N} L(y_k, f_\theta(x_k)),$$

where f_θ is the output of the neural network. When considering the derivative with respect to a parameter, $W_{kl}^{(i)}$ say, in the ith layer it is convenient to use the chain rule to express that derivative. More generally, we may consider the chain rule on directed graphs to obtain a fast method to compute derivatives (or subgradients). This is the idea behind the backpropagation algorithm used for training neural networks. We will begin with a simple description of the chain rule on a computational graph.

25.1.1. *The chain rule on a computational graph.* Consider a directed graph $\mathcal{G}$ with 7 nodes denoted by $h_1, \dots, h_7$. Suppose, for now, that h_7 is the only leaf of the graph and $h_1, \dots, h_4$ are the ancestors (they do not have parents). See Figure 1 below.



Figure 1. Computational graph

The interpretation of the graph is that $h_7 = h_7(h_5, h_6)$, a function of h_5 and h_6, $h_5 = h_5(h_1, h_2, h_3)$ is a function of h_1, h_2 and h_3, etc. Let us consider computing the partial derivative $\frac{\partial h_7}{\partial h_3}$. Since $h_7 = h_7(h_5(h_1, h_2, h_3), h_6(h_3, h_4))$ the chain rule

yields

$$\frac{\partial h_7}{\partial h_3} = \frac{\partial h_7}{\partial h_5}\frac{\partial h_5}{\partial h_3} + \frac{\partial h_7}{\partial h_6}\frac{\partial h_6}{\partial h_3}.$$

If we also want to compute $\frac{\partial h_7}{\partial h_1}$, $\frac{\partial h_7}{\partial h_2}$ and $\frac{\partial h_7}{\partial h_4}$ we will need to recompute $\frac{\partial h_7}{\partial h_5}$ and $\frac{\partial h_7}{\partial h_6}$. It would be convenient to compute $\frac{\partial h_7}{\partial h_5}$ and $\frac{\partial h_7}{\partial h_6}$ first, and store them in the memory and then proceed to compute the partial derivatives with respect to their parents in the graph: $\frac{\partial h_6}{\partial h_3}$, $\frac{\partial h_6}{\partial h_4}$ and $\frac{\partial h_5}{\partial h_1}$, $\frac{\partial h_5}{\partial h_2}$, $\frac{\partial h_5}{\partial h_3}$.

More generally, for a graph with n nodes denoted by $h_1, \ldots, h_n$, where h_n is a scalar output, the leaf. and $h_1, \ldots, h_p$ are the ancestors (the input). The following algorithm computes the partial derivatives. For any $i = 1, \ldots,$, let Parent(i) denote the set of parents to the node i in the graph. For example, in Figure 1, Parent$(5) = \{1, 2, 3\}$. If i is an ancestor, Parent(i) is the empty set. For a subset $\mathcal{L} \subset \{1, \ldots, n\}$, Parent$(\mathcal{L}) = \cup_{i\in\mathcal{L}}$Parent$(i)$. We let $\mathcal{L}_0 = \{n\}$ denote the output layer and $\mathcal{L}_k = \text{Parent}(\mathcal{L}_{k-1}), k \geq 1$, the previous layers. We may suppose the computational graph is constructed so that each node i belongs to exactly one layer.

(1) Initiate the calculation by feeding the values into $h_1, \ldots, h_p$ and make a forward pass to compute the values of $h_1, \ldots, h_n$.
(2) Put $\frac{\partial h_n}{\partial h_n} = 1$ and let $\mathcal{L} = \{n\}$.
(3) For each $j \in \text{Parent}(\mathcal{L})$, compute

$$\frac{\partial h_n}{\partial h_j} = \sum_{i: j\in\text{Parent}(i)} \frac{\partial h_n}{\partial h_i}\frac{\partial h_i}{\partial h_j}$$

and store the values. Update $\mathcal{L} = \text{Parent}(\mathcal{L})$ and repeat until $\mathcal{L}$ is the empty set.

In the training of a feed-forward neural network we may represent h_n as the scalar loss over a mini-batch of training examples and h_i as the hidden variables. In this case we want to compute the derivatives with respect to the parameters in $W^{(i)}$ and $b^{(i)}$, which implies that we need to include the derivatives of the hidden nodes with respect to the parameters as well. For further details, and extensions to tensors, see Section 6.5 in Bengio, Goodfellow: Deep Learning.

Exercise 22 (Backpropagation for a two-hidden-layers neural network)**.** Consider the empirical risk of a two-hidden-layer, fully connected, neural network for multi-class classification. The empirical risk is given by

$$R^{\text{emp}}(\theta) = \frac{1}{n}\sum_{k=1}^{n} L(y_k, \text{softmax}(f_\theta(x_k))),$$

where $L(y, a)$ is the cross-entropy loss. The function $f_\theta(x)$ is given by

$$f_\theta(x) = \left(W^{(2)}(W^{(1)}x + b^{(1)})_+ + b^{(2)}\right)_+,$$

where $\theta = (W^{(1)}, b^{(1)}, W^{(2)}, b^{(2)})$.

(a) Draw the directed computational graph associated with computing $L(y_k, \text{softmax}(f_\theta(x_k)))$.
(b) Provide a step-by-step calculation, according to the backpropagation algorithm, for computing the derivative of $L(y_k, \text{softmax}(f_\theta(x_k)))$ with respect to an arbitrary weight $W_{ij}^{(1)}$ in the first hidden layer.

25.2. **Stochastic gradient descent.** Let us consider the training of a feedforward neural network, which amounts to minimizing the empirical risk. For brevity we write $\theta = \{(W^{(i)}, b^{(i)})\})$ for the parameters. As the risk function is complicated we usually minimize the risk by gradient descent. That is, given an initial point θ_0, the value of θ is updated by following the gradient

$$\theta_{t+1} = \theta_t - \lambda \nabla_\theta R^{\text{emp}}(x, y; \theta_t),$$

where λ is the *learning rate*.

In the setting where the data is assumed to be IID, as in the examples of regression and classification above, the empirical risk can be written as an average over the data,

$$R^{\text{emp}}(\theta) = \frac{1}{n} \sum_{k=1}^{n} L(x_k, y_k; \theta),$$

and consequently the gradient is a

$$\nabla_\theta R^{\text{emp}}(x, y; \theta) = \frac{1}{n} \sum_{k=1}^{n} \nabla_\theta L(x_k, y_k; \theta). \tag{45}$$

The computation of each gradient update requires the computation of n different gradients, which may be slow. To overcome this computational cost stochastic gradient descent (SGD) samples a single observation (x_i, y_i) with replacement from the data and computes the gradient update

$$\theta_{t+1} = \theta_t - \lambda \nabla_\theta L(x_i, y_i; \theta_t).$$

The advantage of SGD is that each update is much faster, which implies that many more gradient updates (training steps) can be taken for the same computational time. The disadvantage is that the computation of the gradient has high variance causing the objective function to fluctuate. The key insight is that the expected value of $\nabla_\theta L(x_i, y_i; \theta_t)$ when sampling with replacement is precisely the full gradient (45), so the SGD update is correct in expectation.

One can also do mini-batch updates, in which a small sample of size n', denoted $(x'_1, y'_1), \ldots, (x'_{n'}, y'_{n'})$, is sampled with replacement from the full data and the gradient update is

$$\theta_{t+1} = \theta_t - \lambda \frac{1}{n'} \sum_{i=1}^{n'} \nabla_\theta L(x'_i, y'_i; \theta_t).$$

Choosing the learning rate is a delicate issue. If the learning rate is high the gradient steps are larger, but there is a risk that the methods overshoots the local minima. In contrast, if the learning rate is small, it may take a long time to reach the local minima. One may try to adapt the learning rate, taking a larger learning rate in the beginning of the training and reducing it as one approaches a local minima.

A further alternative is to introduce a momentum, where the gradient updates are given by

$$\begin{aligned} m_{t+1} &= \gamma m_t + \lambda \nabla_\theta L(x, y; \theta_t), \\ \theta_{t+1} &= \theta_t - m_{t+1}. \end{aligned}$$

As the name suggests, it gives the updates a momentum from recent updates with reduces the variance of the updates.

A more recent method which has become popular is the ADAM optimizer (Adaptive Moment Estimator) which computes adaptive learning rates for each parameter. For more details on various optimization methods for training deep neural network we refer to Chapter 8 of *Deep Learning*.

25.3. **Relevant papers on convolutional neural networks.** Below we list some relevant papers for the progress of convolutional neural networks in image classification.

- Krizhevsky, A., Sutskever, I., and Hinton, G. E. ImageNet classification with deep convolutional neural networks, 2010, `https://www.cs.toronto.edu/~fritz/absps/imagenet.pdf`
- Szegedy, C., Liu, W., Yangqing, J., Sermnanet, P., Reed, S., Anguelov, D., Erhan, D:, Vanhoucke, V., and Rabinovic, A.: Going deeper with convolutions, 2014, `https://arxiv.org/pdf/1409.4842.pdf`
- Srivastava, N., Hinton, G.E., Krizhevsky, A., Sutskever, I., and Salakthudinov, R.: Dropout: a simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15, 1929–1958, 2014.
- Simonyan, K. and Zisserman, A.: Very deep convolutional networks for large-scale image recognition. ICLR 2015. `https://arxiv.org/pdf/1409.1556.pdf`
- Ioffe, S. and Szegedy, C.: Batch normalization: accelerating deep networks training by reducing internal covariate shift, 2015. `https://arxiv.org/pdf/1502.03167.pdf`

Literature:

- Goodfellow et al. *Deep Learning*, Chapter 10.

26. Recurrent neural networks

Recurrent neural networks are constructed for dealing with sequential modeling, such as sound, voice, text, etc. It is convenient to view recurrent neural networks from the point of view of hidden Markov models.

Consider a hidden Markov model, where $\{h_t\}$ is an unobserved (hidden) Markov chain with transition probability $p(h_t \mid h_{t-1})$ and $\{y_t\}$ are observations from $p(y_t \mid h_t)$, where $y_1, \dots, y_T$ are conditionally independent given $h_1, \dots, h_T$. We assume that the probability distribution has its own set of parameters, θ and ψ, respectively. That is,

$$h_t \mid h_{t-1} \sim p(h_t \mid h_{t-1}, \theta),$$
$$y_t \mid h_t \sim p(y_t \mid h_t, \psi).$$

Based on observations $\{y_t\}_{t=1}^T$ we may want to perform inference on θ, ψ, recover the hidden states $h_1, \dots, h_T$, or make predictions of y_{T+1}, etc.

We can extend the above formulation by introducing an additional observed input x_t into the dynamics of the hidden sequence $\{h_t\}$. Consider the system

$$h_t \mid h_{t-1}, x_t \sim p(h_t \mid h_{t-1}, x_t, \theta),$$
$$y_t \mid h_t \sim p(y_t \mid h_t, \psi).$$

The inclusion of the observed input x_t does not change much from a theoretical point of view.

If we consider estimating θ and ψ using maximum likelihood we run into trouble, because the likelihood needs to be marginalized over the hidden states.

$$\begin{aligned} p(y_1,\dots,y_T \mid x_1,\dots,x_T,\theta,\psi) &= \int p(y_1,\dots,y_T,h_1,\dots,h_T \mid \{x_t\},\theta,\psi)dh_1\cdots dh_T \\ &= \int p(y_1,\dots,y_T \mid h_1,\dots,h_T,\theta,\psi)p(h_1,\dots,h_T \mid \{x_t\},\theta,\psi)dh_1\cdots dh_T \\ &= \int \prod_{t=1}^{T} p(y_t \mid h_t,\psi)p(h_t \mid h_{t-1},x_t,\theta)dh_1\cdots dh_T. \end{aligned}$$

The likelihood is therefore intractable and to approach the inference problem one needs to apply advanced computational methods such as the EM-algorithm, particle filters, MCMC, etc.

Here we take a much more elementary approach and assume that the transition from h_{t-1} to h_t is given by a deterministic function f. That is, we have the system

$$\begin{aligned} h_t &= f(h_{t-1},x_t;\theta), \\ y_t \mid h_t &\sim p(y_t \mid h_t,\psi). \end{aligned} \tag{46}$$

Then, $h_1,\dots,h_T$ are functions of $x_1,\dots,x_T$ and therefore the likelihood can be written as

$$\begin{aligned} p(\{y_t\} \mid \{x_t\},\theta,\psi) &= p(\{y_t\} \mid \{x_t\},\{h_t\},\theta,\psi) \\ &= \prod_{t=1}^{T} p(y_t \mid h_t,\psi). \end{aligned}$$

Maximizing the likelihood is then equivalent to minimizing the negative log-likelihood. If, for instance,

$$y_t \mid h_t \sim N(g(h_t;\psi),\sigma^2 I),$$

where I is the identity matrix, then maximizing the likelihood is equivalent to minimizing the loss function

$$L(\{x_t,y_t\};\theta,\psi) = \frac{1}{2\sigma^2}\sum_{t=1}^{T}|y_t - g(h_t;\psi)|^2,$$

where h_t depends on θ, through (46).

If y_t is categorical, taking values in the classes $\{1,\dots,C\}$, and we represent y_t as a one-hot-vector $y_t = (y_t^1,\dots,y_t^C)$, then we could model

$$p(y_t \mid h_t) = \prod_c g^c(h_t;\psi)^{y_t^c},$$

where

$$g(h,\psi) = (g^1(h,\psi),\dots,g^C(h,\psi)),$$

are class probabilities. Maximizing the likelihood is equivalent to minimizing the cross-entropy loss

$$L(\{x_t,y_t\};\theta,\psi) = -\sum_{t=1}^{T}\sum_{c=1}^{C} y_t^c \log g_c(h_t;\psi).$$

In many cases of interest we can write the loss as

$$L(\{x_t, y_t\}; \theta, \psi) = \sum_{t=1}^{T} L_t(y_t, g(h_t; \psi))).$$

Minimizing the loss by SGD requires to compute the gradients with respect to θ and ψ. In principle, this is not more complicated than the backpropagation algorithm.

26.1. **Backproagation through time (BPTT).** To illustrate the backpropagation through time, let us for simplicity assume that all the variables are one-dimensional. The extension to vector valued or tensor valued functions are straightforward, but requires some additional care. Suppose that we can write the loss as

$$L(\{x_t, y_t\}; \theta, \psi) = \sum_{t=1}^{T} L_t(y_t, g(h_t; \psi)).$$

We want to minimize the loss with respect to θ and ψ using SGD. For this reason we want to compute the derivatives of the loss with respect to θ and ψ. Given the current values of θ and ψ we make a forward pass through the network to compute $h_t, g(h_t; \psi)$ for $t = 1, \dots, T$. The computation of the derivative with respect to ψ is direct:

$$\begin{aligned}\frac{\partial}{\partial \psi} L(\{x_t, y_t\}; \theta, \psi) &= \sum_{t=1}^{T} \frac{\partial}{\partial \psi} L_t(y_t, g(h_t; \psi)) \\ &= \sum_{t=1}^{T} \frac{\partial L_t(y_t, g(h_t; \psi))}{\partial g} \frac{\partial g(h_t; \psi)}{\partial \psi}.\end{aligned}$$

Note that each term in the sum can be computed separately. The derivative with respect to θ is more involved and can be performed by backpropagation. For notational convenience we introduce dummy variables θ_t, $t = 1, \dots, T$, all taking the value θ, such that $h_t = f(h_{t-1}, x_t; \theta_t)$. Then,

$$\begin{aligned}\frac{\partial}{\partial \theta} L(\{x_t, y_t\}; \theta, \psi) &= \sum_{t=1}^{T} \frac{\partial}{\partial \theta} L_t(y_t, g(h_t; \psi)) \\ &= \sum_{t=1}^{T} \frac{\partial L_t(y_t, g(h_t; \psi))}{\partial g} \frac{\partial g(h_t; \psi)}{\partial h_t} \frac{\partial h_t}{\partial \theta}.\end{aligned}$$

Since h_t depends on θ both through the function $f(h_{t-1}, \theta)$ and, because h_{t-1} is also a function of θ, we can use the dummy variables to write

$$\frac{\partial h_t}{\partial \theta} = \frac{\partial f(h_{t-1}, x_t; \theta_t)}{\partial \theta_t}\Big|_{\theta_t = \theta} + \frac{\partial f(h_{t-1}, x_t; \theta_t)}{\partial h_{t-1}} \frac{\partial h_{t-1}}{\partial \theta}$$

Beginning at $t = T$ we can iterate backwards to compute all the derivatives $\frac{\partial h_t}{\partial \theta}$ for $t = T, \dots, 1$.

26.2. **Variations of recurrent neural networks.** The conventional RNN for categorical data y has the form:

$$\begin{aligned}h_t = f(h_{t-1}, x_t; \theta) &= \tanh\left(b + W h_{t-1} + U x_t\right), \\ g(h_t; \psi) &= \operatorname{softmax}(c + V h_t),\end{aligned}$$

where the parameters are $\theta = (b, W, U)$ and $\psi = (c, V)$ with b, c being vectors and W, U, V matrices.

If we replace the functions f and g by deep feed-forward neural networks, then we get a so-called deep RNN (Pascanu et al., "How to construct deep recurrent neural networks", 2014, `arXiv:1312.6026v5`).

Note that training a deep RNN can also be done with the BPTT algorithm. The main difference is that we need to use the standard backpropagation algorithm to compute the gradients/Jacobians associated with

$$\frac{\partial}{\partial \psi} g(h;\psi),\ \frac{\partial}{\partial h} g(h;\psi),\ \frac{\partial}{\partial \theta} f(h,x;\theta),\ \frac{\partial}{\partial h} f(h,x;\theta).$$

26.2.1. *Faster training by replacing hidden-to-hidden.* The computation of the derivatives in BPTT is slow because we need to propagate the derivatives of the hidden-to-hidden connections backward. If we remove the hidden-to-hidden and instead add an output-to-hidden connection, the computation of gradients can be made faster. That is, we introduce the notation

$$o_t = g(h_t, \psi)$$

and replace $h_t = f(h_{t-1}, x_t; \theta)$ by

$$h_t = f(o_{t-1}, x_t; \theta).$$

Then, following the BPTT algorithm above, we can compute

$$\frac{\partial h_t}{\partial \theta} = \frac{\partial f(o_{t-1}, x_t; \theta)}{\partial \theta}$$

Note that o_{t-1} is computed in the forward pass and therefore the derivatives $\frac{\partial h_t}{\partial \theta}$, $t = 1, \dots, T$ can be computed in parallel.

26.3. **Long-short term memory model.** The long-short term memory model (LSTM) is a recurrent neural network that has become very popular. It has a more complex rule for updating of the hidden state. Recall that the conventional RNN updates the hidden state based on the previous hidden state h_{t-1} and the input x_t:

$$h_t = f(h_{t-1}, x_t; \theta) = \tanh\left(b + W h_{t-1} + U x_t\right).$$

In the LSTM this update is replaced by a LSTM-cell, which combines information from the previous hidden state, the input and an internal state s_t. The purpose of the internal state is to capture longer memory effects. The various effects are combined using gates taking values between 0 and 1. The LSTM-cell takes h_{t-1} and x_t as input and produces h_t as output. It is defined by

- the input gate: $g_t = (g_t^1, \dots, g_t^d)$,

$$g_t^i = \text{sigm}\left(b_i^g + \sum_j U_{ij}^g x_t^j + \sum_j W_{ij}^g h_{t-1}^j\right),$$

- the forget gate: $f_t = (f_t^1, \dots, f_t^d)$,

$$f_t^i = \text{sigm}\left(b_i^f + \sum_j U_{ij}^f x_t^j + \sum_j W_{ij}^f h_{t-1}^j\right),$$

- the outputgate: $q_t = (q_t^1, \ldots, q_t^d)$,

$$q_t^i = \text{sigm}\left(b_i^q + \sum_j U_{ij}^q x_t^j + \sum_j W_{ij}^q h_{t-1}^j\right),$$

- the internal state: $s_t = (s_t^1, \ldots, s_t^d)$,

$$s_t^i = f_t^i s_{t-1}^i + g_t^i \,\text{sigm}\left(b_i + \sum_j U_{ij} x_t^j + \sum_j W_{ij} h_{t-1}^j\right),$$

- output: $h_t = (h_t^1, \ldots, h_t^d)$,

$$h_t^i = q_t^i \tanh(s_t^i).$$

For more details on LSTM see Section 10.10.1 in Goodfellow et al. *Deep Learning*.

Exercise 23. Consider a recurrent neural network

$$h_t = f(h_{t-1}, x_t; \theta) = \tanh(b + W h_{t-1} + U x_t),$$
$$y_t \mid h_t \sim N(g(h_t; \psi), \sigma^2 I) = N(c + V h_t, \sigma^2 I),$$

where $\theta = (b, W, U)$ and $\psi = (c, V)$ are unknown parameters. To train the model we aim to minimize

$$L(\{x_t, y_t\}; \theta, \psi) = \sum_{t=1}^{T} L_t(y_t, g(h_t; \psi)),$$

where

$$L_t(x_t, y_t; \theta, \psi) = \frac{1}{2\sigma^2} |y_t - g(h_t; \psi)|^2.$$

We may also interpret the unobserved initial state h_0 as a parameter in the model.

(a) Draw the directed computational graph associated with computing $L(\{x_t, y_t\}, \theta, \psi)$.
(b) Let $T = 2$ and provide a step-by-step calculation, according to the backpropagation through time algorithm, for computing
   (a) the derivative of $L(\{x_t, y_t\}, \theta, \psi)$ with respect to an arbitrary component of h_0,
   (b) the derivative of $L(\{x_t, y_t\}, \theta, \psi)$ with respect to an arbitrary weight W_{ij} in the matrix W.

References

[1] Dempster, A.P., Laird, N.M. and Rubin, D.B. (1977) Maximum Likelihood from Incomplete Data via the EM Algorithm, *Annals of Statistics* 39, 1 – 38.

[2] MacLachlan, G.J. and Krishnan, T. (1997) *The EM algorithm and extensions*, John Wiley & Sons ISBN 0-471-12358-7.

(Henrik Hult) Department of Mathematics, KTH
Email address: hult@kth.se
URL: